

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

计算机组成原理



老汤

浙江 杭州

讲师：老汤

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

第四章：数据的表示与运算



第五章：指令系统



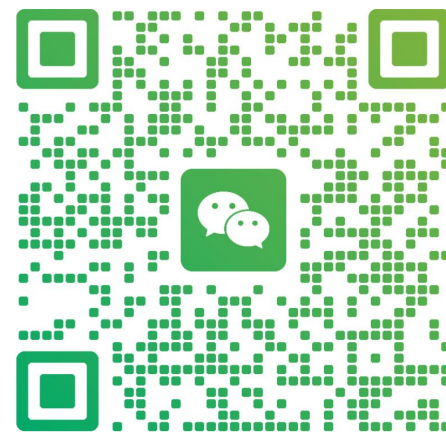
说明了计算机能做什么

第六章：中央处理器（CPU）



说明了计算机是如何做

第七章：输入输出（I/O）系统



扫一扫上面的二维码图案，加我为朋友。

指令系统：指令集体系结构 (ISA)

应用 (问题)
算法与数据结构
编程语言
操作系统
指令集体系结构 (ISA)
微体系结构
逻辑电路
器件

指令集体系结构 (Instruction Set Architecture, ISA)
是一种规定或结构规范，是软件和硬件之间的接口

- ① 定义了一台计算机可以执行的**所有指令的集合**
- ② 规定了每条指令对应计算机**执行什么操作**
- ③ 规定了指令处理的操作数**存放的位置**以及**操作数类型**

指令系统：指令集体系结构 (ISA)

应用 (问题)
算法与数据结构
编程语言
操作系统
指令集体系结构 (ISA)
微体系结构
逻辑电路
器件

指令集体系结构 (Instruction Set Architecture, ISA)
是一种规定或结构规范，是软件和硬件之间的接口

① Intel X86

支配着计算机的 PC 时代

② ARM

最流行的嵌入式设备指令系统

③ MIPS

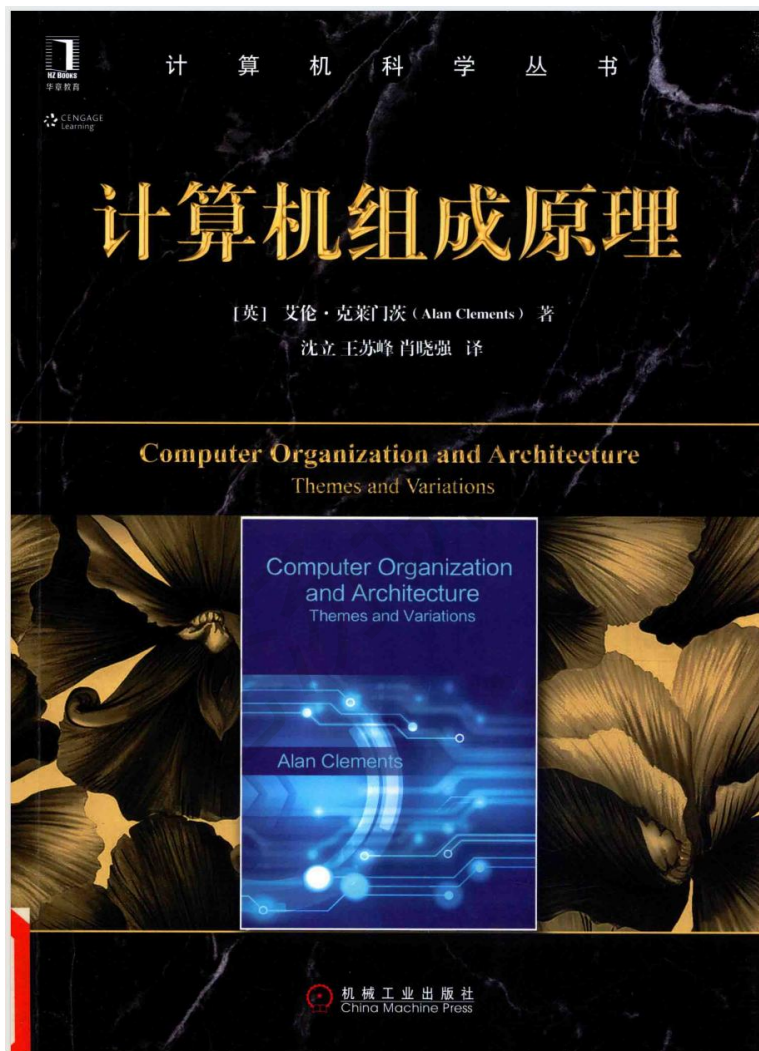
④ RISC-V

使得计算机的构建变的简单

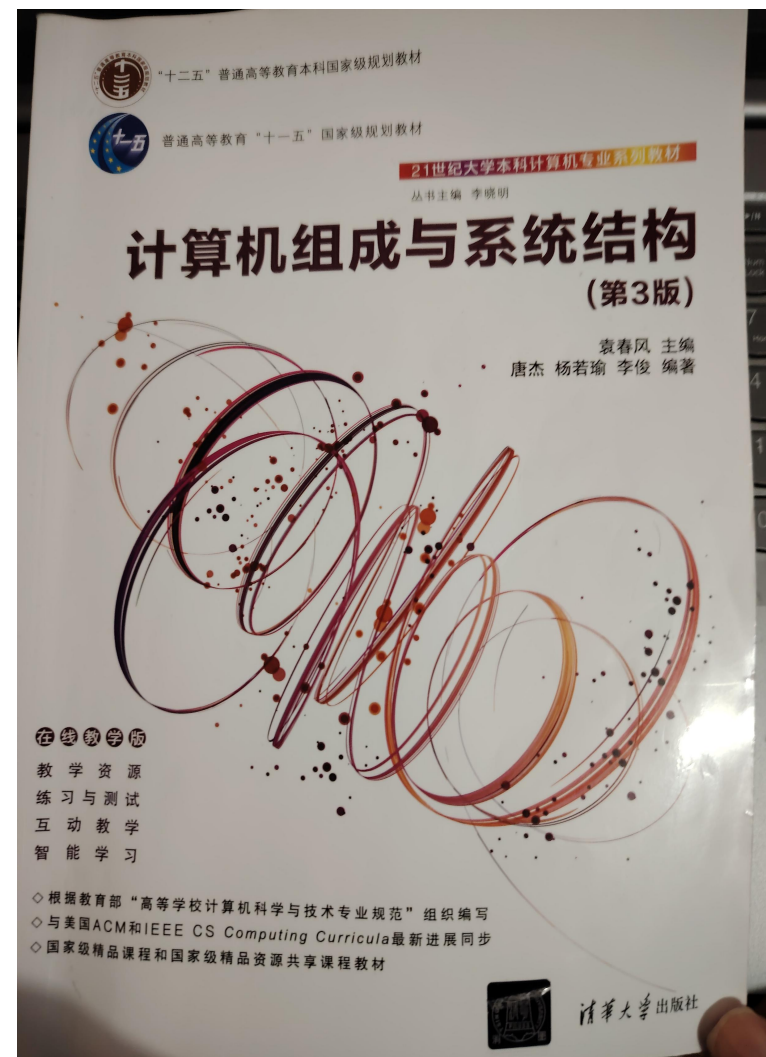
指令系统：指令集体系结构 (ISA)



第 3 章讲解的是 X86 指令系统

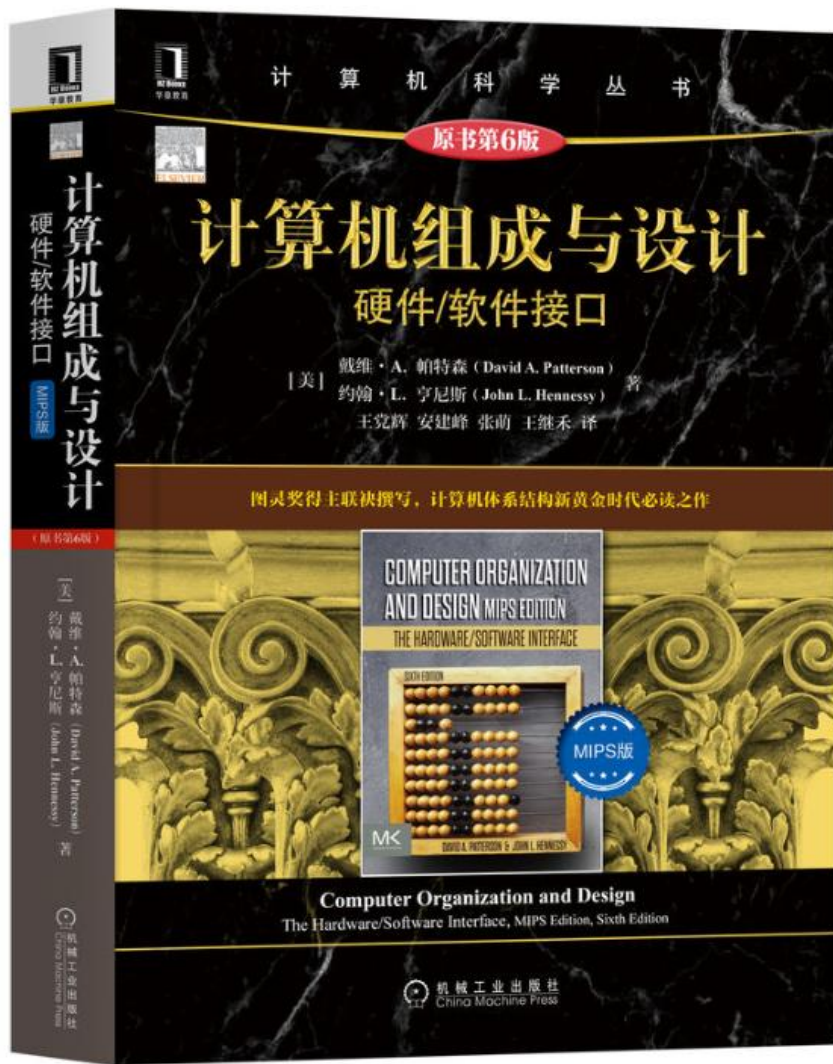


第 3 章讲解的是 ARM 指令系统

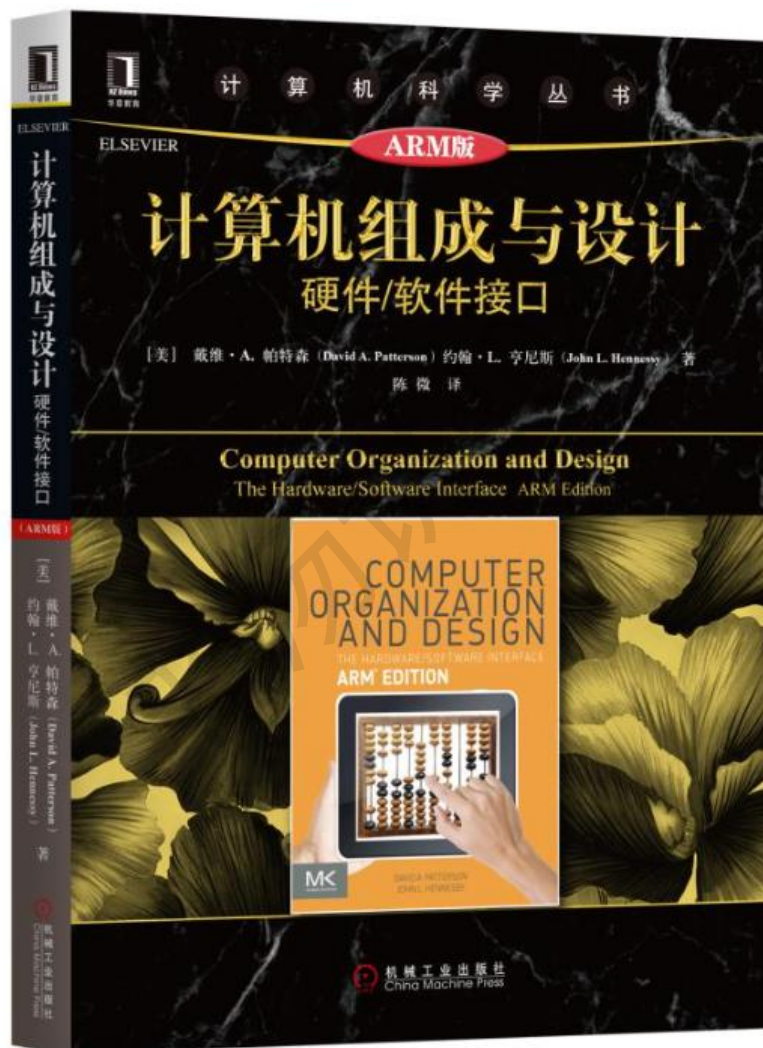


第 4 章讲解的是 MIPS 和 RISC-V

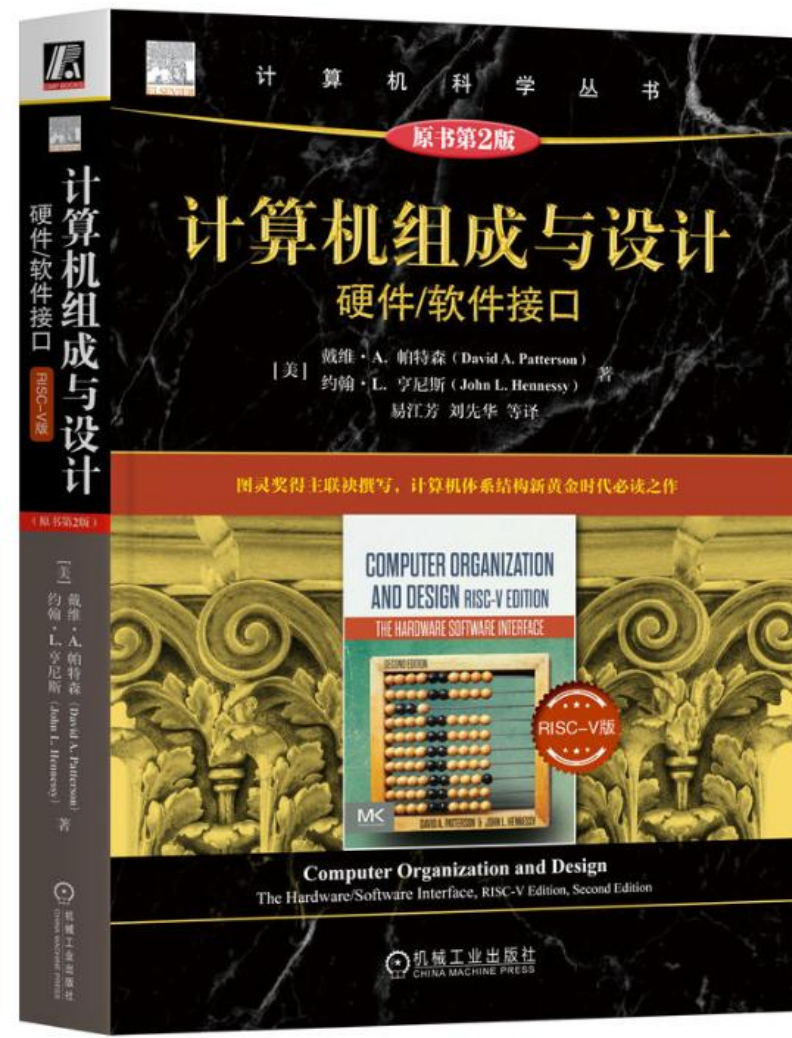
指令系统：指令集体系结构 (ISA)



MIPS 版本



ARM 版本



RISC-V 版本

指令系统：指令集体系结构 (ISA)

CISC (Complex Instruction Set Computer, 复杂指令集计算机) —→ Intel X86



RISC (Reduced Instruction Set Computer, 精简指令集计算机)

ARM

MIPS

RISC-V



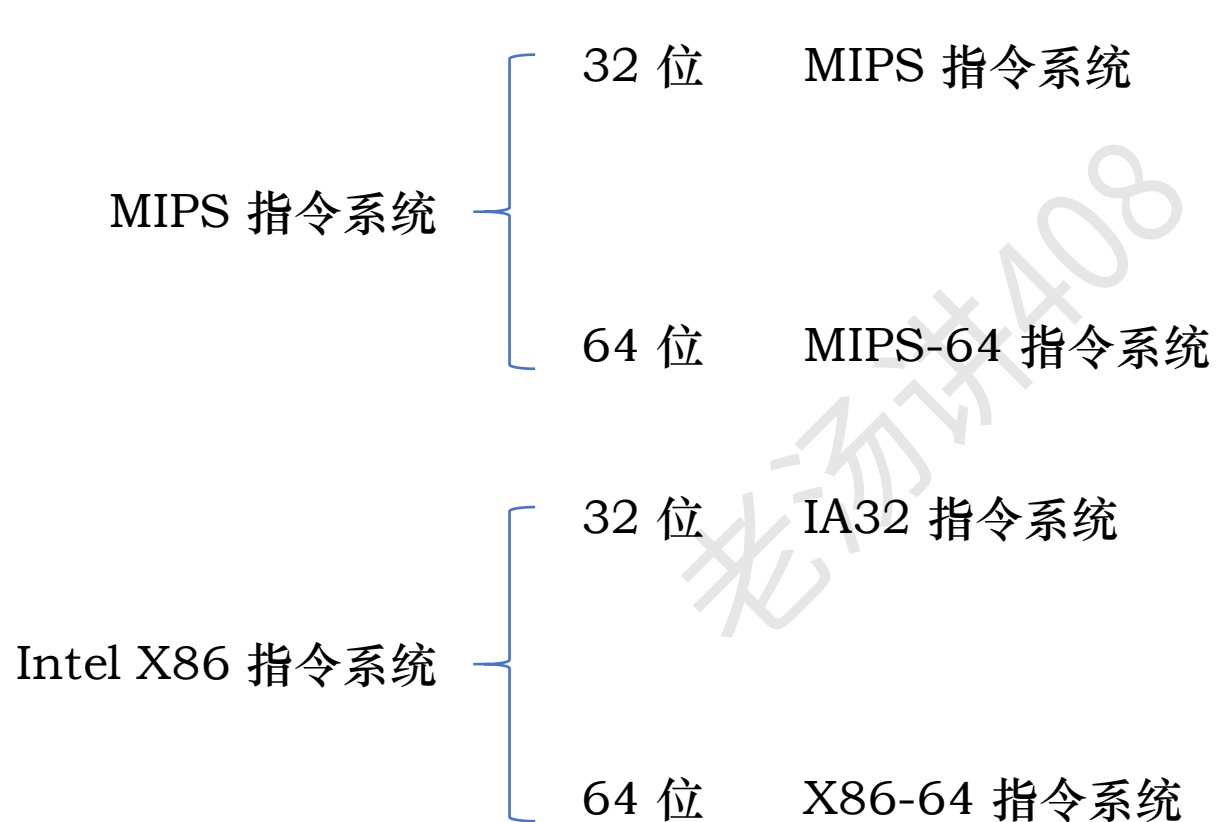
自定义指令系统：2010 年、2012 年、2013 年、2015 年

MIPS 指令系统：2014 年、2021 年 RISC-V 指令系统：2024 年

x86 指令系统：2017 年、2019 年、2023 年

主要目标：让你自己会设计一套简单的指令系统

指令系统：指令集体系结构 (ISA)



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

指令系统：指令集体系结构 (ISA)

应用 (问题)
算法与数据结构
编程语言
操作系统
指令集体系结构 (ISA)
微体系结构
逻辑电路
器件

指令集体系结构 (Instruction Set Architecture, ISA)

是对指令系统的一种规定或结构规范，是软件和硬件之间的接口

- ① 定义了一台计算机可以执行的**所有指令的集合**
- ② 规定了每条指令对应计算机**执行什么操作**
- ③ 规定了指令处理的操作数**存放的位置**以及**操作数类型**

MIPS 指令系统：指令集

类别	指令	示例	含义	注释
算术运算	加法	add \$s1, \$s2, \$s3	$\$s1 = \$s2 + \$s3$	三个寄存器操作数
	减法	sub \$s1, \$s2, \$s3	$\$s1 = \$s2 - \$s3$	三个寄存器操作数
	立即数加法	addi \$s1, \$s2, 20	$\$s1 = \$s2 + 20$	用于加常数数据
数据传输	取字	lw \$s1, 20(\$s2)	$\$s1 = \text{MEM}[\$s2 + 20]$	将一个字从内存中取到寄存器中
	存字	sw \$s1, 20(\$s2)	$\text{MEM}[\$s2 + 20] = \$s1$	将一个字从寄存器中取到内存中
	取半字	lh \$s1, 20(\$s2)	$\$s1 = \text{MEM}[\$s2 + 20]$	将半个字从内存中取到寄存器中
	取无符号半字	lhu \$s1, 20(\$s2)	$\$s1 = \text{MEM}[\$s2 + 20]$	将半个字从内存中取到寄存器中
	存半字	sh \$s1, 20(\$s2)	$\text{MEM}[\$s2 + 20] = \$s1$	将半个字从寄存器中取到内存中
	取字节	lb \$s1, 20(\$s2)	$\$s1 = \text{MEM}[\$s2 + 20]$	将一个字节从内存中取到寄存器中
	取无符号字节	lbu \$s1, 20(\$s2)	$\$s1 = \text{MEM}[\$s2 + 20]$	将一个字节从内存中取到寄存器中
	存字节	sb \$s1, 20(\$s2)	$\text{MEM}[\$s2 + 20] = \$s1$	将一个字节从寄存器中取到内存中
	取链接字	ll \$s1, 20(\$s2)	$\$s1 = \text{MEM}[\$s2 + 20]$	取字作为原子交换的前半部
	存条件字	sc \$s1, 20(\$s2)	$\text{MEM}[\$s2 + 20] = \$s1$ $\$s1 = 0 \text{ 或 } 1$	存字作为原子交换的前半部
	取立即数的高位	lui \$s1, 20	$\$s1 = 20 * 2^{16}$	取立即数并放到高 16 位

类别	指令	示例	含义	注释
逻辑运算	与	and \$s1, \$s2, \$s3	$\$s1 = \$s2 \& \$s3$	三个寄存器操作数（按位与）
	或	or \$s1, \$s2, \$s3	$\$s1 = \$s2 \mid \$s3$	三个寄存器操作数（按位或）
	或非	nor \$s1, \$s2, \$s3	$\$s1 = \sim(\$s2 \mid \$s3)$	三个寄存器操作数(按位或非)
	立即数与	andi \$s1, \$s2, 20	$\$s1 = \$s2 \& 20$	和常数按位与
	立即数或	ori \$s1, \$s2, 20	$\$s1 = \$s2 \mid 20$	和常数按位或
	逻辑左移	sll \$s1, \$s2, 20	$\$s1 = \$s2 \ll 20$	根据常数左移相应位
	逻辑右移	srl \$s1, \$s2, 20	$\$s1 = \$s2 \gg 20$	根据常数右移相应位
条件分支	相等时跳转	beq \$s1, \$s2, 25	if ($\$s1 == \$s2$) 跳转PC+4+100	相等检测；和 PC 相关跳转
	不相等时跳转	bne \$s1, \$s2, 25	if ($\$s1 \neq \$s2$) 跳转PC+4+100	不相等检测；和PC相关跳转
	小于时置位	slt \$s1, \$s2, \$s3	if($\$s2 < \$s3$) $\$s1 = 1$ else $\$s1 = 0$	比较是否小于
	小于时置位（无符号）	sltu \$s1, \$s2, \$s3	if($\$s2 < \$s3$) $\$s1 = 1$ else $\$s1 = 0$	比较是否小于无符号数
	小于立即数时置位	slti \$s1, \$s2, 20	if($\$s2 < 20$) $\$s1 = 1$ else $\$s1 = 0$	比较是否小于常数
	无符号小于立即数时置位	sltiu \$s1, \$s2, 20	if($\$s2 < 20$) $\$s1 = 1$ else $\$s1 = 0$	比较是否小于无符号常数
无条件跳转	跳转	j 2500	go to 10000	跳转到目标地址
	跳转到寄存器所指位置	jr \$ra	go to \$ra	用于 switch 和过程调用
	跳转并链接	jal 2500	$\$ra = PC + 4$; go to 10000	用于过程调用

Intel X86-64指令系统：指令集

类别	指令	示例	含义
算术运算	加法	add <reg>, <reg>	将两个寄存器的值相加，结果存放在第一个寄存器中
		add <reg>, <mem>	将寄存器值和指定内存值相加，结果存放在寄存器中
		add <mem>, <reg>	将指定内存值和寄存器值相加，结果存放在指令内存中
		add <reg>, <con>	将寄存器值和常数相加，结果存放在寄存器中
		add <mem>, <con>	将指定内存值和常数相加，结果存放在指定内存中
	减法	sub <reg>, <reg>	将两个寄存器的值相减，结果存放在第一个寄存器中
		sub <reg>, <mem>	将寄存器值和指定内存值相减，结果存放在寄存器中
		sub <mem>, <reg>	将指定内存值和寄存器值相减，结果存放在指令内存中
		sub <reg>, <con>	将寄存器值和常数相减，结果存放在寄存器中
		sub <mem>, <con>	将指定内存值和常数相减，结果存放在指定内存中
	自增/自减	inc <reg>	将寄存器值自增 1
		inc <mem>	将指定内存值自增 1
		dec <reg>	将寄存器值自减 1
		dec <mem>	将指定内存值自减 1

类别	指令	示例	含义
算术运算	乘法	imul <reg>, <reg>	将两个寄存器的值相乘，结果存放在第一个寄存器中
		imul <reg>, <mem>	将寄存器值和指定内存值相乘，结果存放在寄存器中
		imul <reg>, <reg>, <con>	将第二个寄存器的值和常数相乘，结果放在第一个寄存器中
		imul <reg>, <mem>, <con>	将指定内存的值和常数相乘，结果放在第一个寄存器中
	除法	idiv <reg>	被除数在 edx:eax 中，除数就是 idiv 后指定的寄存器值
		idiv <mem>	被除数在 edx:eax 中，除数就是 idiv 后指定内存中的值
数据传送	mov	mov <reg>, <reg>	将第二个寄存器的内容复制到第一个寄存器
		mov <reg>, <mem>	将指定内存的数据复制到第一个寄存器中
		mov <mem>, <reg>	将寄存器中的数据复制到指定内存中
		mov <reg>, <con>	将常数复制到第一个寄存器中
		mov <mem>, <con>	将常数复制到指定内存中
	push/pop	push <reg>	将寄存器中的内容压入内存的栈中
		push <mem>	将指定内存的内容压入内存的栈中
		push <con>	将常数压入内存的栈中
		pop <reg>	弹出内存栈顶的数据，存入寄存器中
		pop [<reg>]	弹出内存栈顶数据送到寄存器中指示的内存地址中

类别	指令	示例	含义
逻辑运算	逻辑与	and <reg>, <reg>	将两个寄存器的值相与，结果存放在第一个寄存器中
		and <reg>, <mem>	将寄存器值和指定内存值相与，结果存放在寄存器中
		and <mem>, <reg>	将指定内存值和寄存器值相与，结果存放在指令内存中
		and <reg>, <con>	将寄存器值和常数相与，结果存放在寄存器中
		and <mem>, <con>	将指定内存值和常数相与，结果存放在指定内存中
	逻辑或	or <reg>, <reg>	将两个寄存器的值相或，结果存放在第一个寄存器中
		or <reg>, <mem>	将寄存器值和指定内存值相或，结果存放在寄存器中
		or <mem>, <reg>	将指定内存值和寄存器值相或，结果存放在指令内存中
		or <reg>, <con>	将寄存器值和常数相或，结果存放在寄存器中
		or <mem>, <con>	将指定内存值和常数相或，结果存放在指定内存中
	逻辑异或	xor <reg>, <reg>	将两个寄存器的值相异或，结果存放在第一个寄存器中
		xor <reg>, <mem>	将寄存器值和指定内存值相异或，结果存放在寄存器中
		xor <mem>, <reg>	将指定内存值和寄存器值相异或，结果存放在指令内存中
		xor <reg>, <con>	将寄存器值和常数相异或，结果存放在寄存器中
		xor <mem>, <con>	将指定内存值和常数相异或，结果存放在指定内存中

类别	指令	示例	含义
逻辑运算	逻辑非	not <reg>	将寄存器中的值取反
		not <mem>	将指定内存中的值取反
	取负	neg <reg>	将寄存器中的值取负
		neg <mem>	将指定内存中的值取负
	逻辑左移	shl <reg>, <con>	将寄存器中的值逻辑左移给定位
		shl <mem>, <con>	将指定内存中的值逻辑左移给定位
		shl <reg>, <reg>	将寄存器中的值逻辑左移给定位
		shl <mem>, <reg>	将指定内存中的值逻辑左移给定位
	逻辑右移	shr <reg>, <con>	将寄存器中的值逻辑右移给定位
		shr <mem>, <con>	将指定内存中的值逻辑右移给定位
		shr <reg>, <reg>	将寄存器中的值逻辑右移给定位
		shr <mem>, <reg>	将指定内存中的值逻辑右移给定位
无条件转移	jmp <label>	跳转到目标地址	
	call <label>	函数调用指令	
	ret	子程序的返回	

类别	指令	示例	含义
条件分支	条件转移	je <label>	jump when equal
		jz <label>	jump when last result was zero
		jne <label>	jump when not equal
		jg <label>	jump when greater than
		jge <label>	jump when greater than or equal to
		jl <label>	jump when less than
		jle <label>	jump when less than or equal to
	cmp 指令的功能相当于 sub 指令，用于比较两个操作数的值	cmp <reg>,<reg>	比较两个寄存器的值
		cmp <reg>,<mem>	比较寄存器的值和指定内存的值
		cmp <mem>,<reg>	比较指定内存的值和寄存器的值
		cmp <reg>,<con>	比较寄存器的值和常数
	test 指令的功能相当于 and 指令，对两个操作数进行逐位与运算	test <reg>,<reg>	比较两个寄存器的值
		test <reg>,<mem>	比较寄存器的值和指定内存的值
		test <mem>,<reg>	比较指定内存的值和寄存器的值
		test <reg>,<con>	比较寄存器的值和常数

指令系统：指令集体系结构 (ISA)

```
void swap(int v[], int k) {  
    int temp;  
    temp = v[k];  
    v[k] = v[k + 1];  
    v[k + 1] = temp;  
}
```

MIPS 指令系统

```
sll    $t1, $a1, 2  
add    $t1, $t1, $a0  
lw     $t0, 0($t1)  
lw     $t2, 4($t1)  
sw     $t2, 0($t1)  
sw     $t0, 4($t1)  
jr     $31
```

X86-64 指令系统

```
movsx  rsi, esi  
lea    rdx, [rdi+rsi*4]  
mov     ecx, DWORD PTR [rdx]  
lea    rax, [rdi+4+rsi*4]  
mov     esi, DWORD PTR [rax]  
mov     DWORD PTR [rdx], esi  
mov     DWORD PTR [rax], ecx  
ret
```


指令系统：指令集体系结构 (ISA)

① 指令系统包含哪些内容

② 指令系统的设计原则

③ RISC 和 CISC 的区别

抽象：如何设计一个指令系统？



具体的指令系统实例：

MIPS

X86-64

三部分内容

寄存器组

指令格式

寻址方式

指令系统：指令集体系结构 (ISA)

① 指令系统包含哪些内容

② 指令系统的设计原则

③ RISC 和 CISC 的区别

抽象：如何设计一个指令系统？



具体的指令系统实例：

MIPS

X86-64

三部分内容

寄存器组

指令格式

寻址方式

程序的机器级表示

选择结构

循环结构

函数调用

指令系统：指令集体系结构 (ISA)

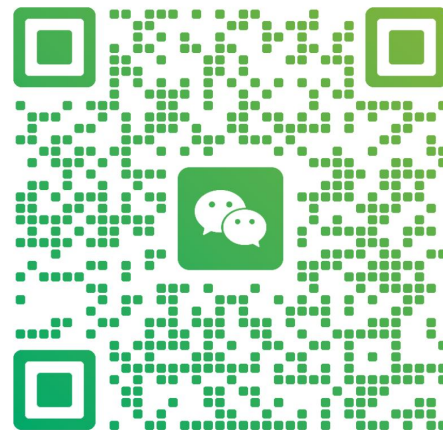
加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

<https://godbolt.org/>



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

指令系统：指令集体系结构 (ISA)

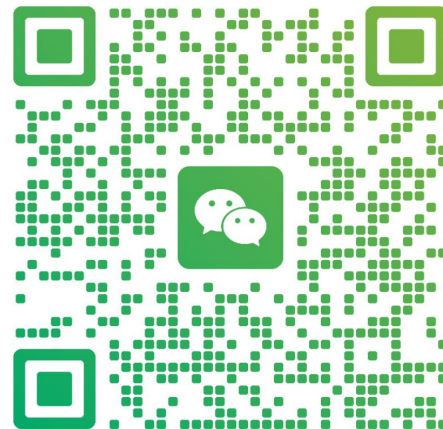
```
#include<stdio.h>
```

```
void swap(int v[], int k) {  
    int temp;  
    temp = v[k];  
    v[k] = v[k + 1];  
    v[k + 1] = temp;  
}  
  
int main() {  
    int arr[5] = {1, 2, 3, 4, 5};  
    swap(arr, 2);  
    return 0;  
}
```

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

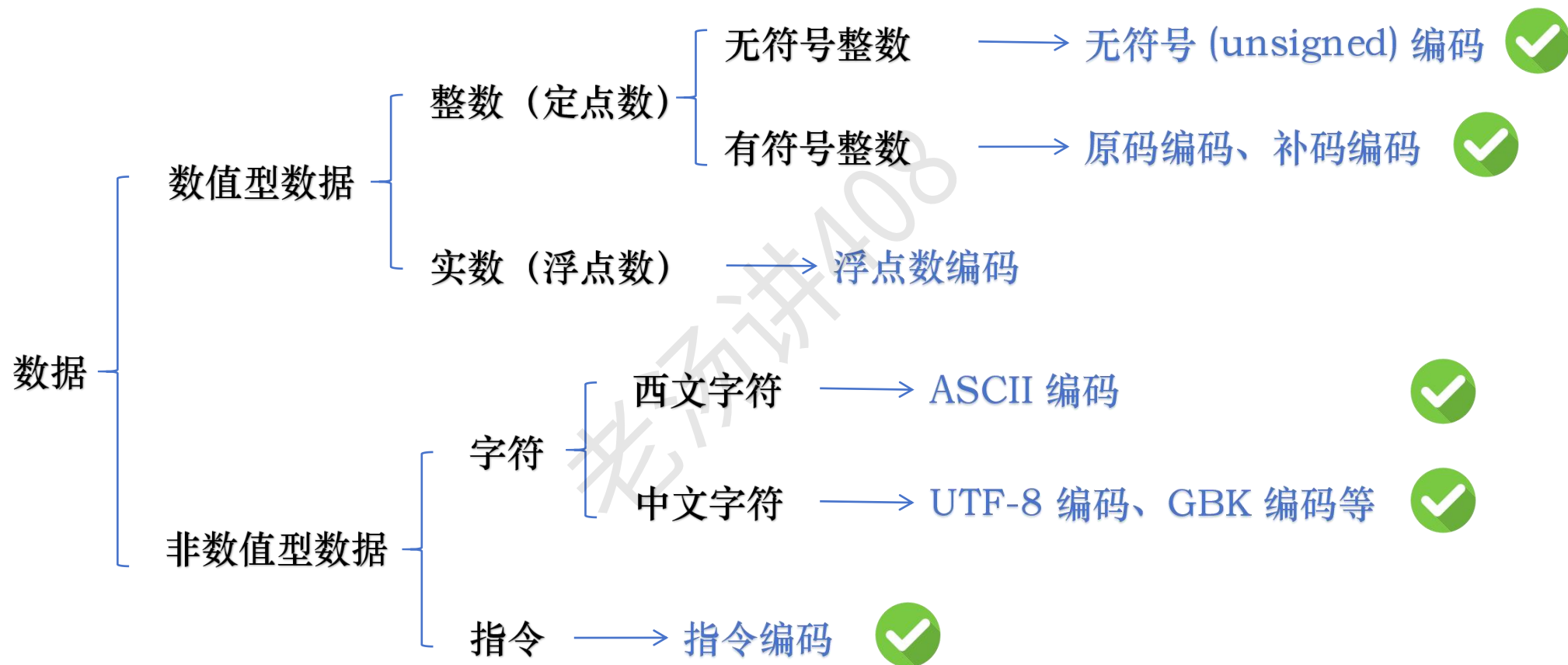


老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

数据存储：字节顺序和内存对齐



MIPS 指令系统

CISC (Complex Instruction Set Computer, 复杂指令集计算机) —→ Intel X86

RISC (Reduced Instruction Set Computer, 精简指令集计算机)

ARM

MIPS

RISC-V



MIPS 是 “Microprocessor without Interlocked Pipeline Stages” 的缩写，
即 “无互锁流水级的微处理器”

32 位的 MIPS 指令系统

MIPS 指令集体系结构 (ISA)

① 指令系统包含哪些内容

② 指令系统的设计原则

③ RISC 和 CISC 的区别

抽象：如何设计一个指令系统？



具体的指令系统实例：

MIPS

X86-64

三部分内容

寄存器组

指令格式

寻址方式

程序的机器级表示

选择结构

循环结构

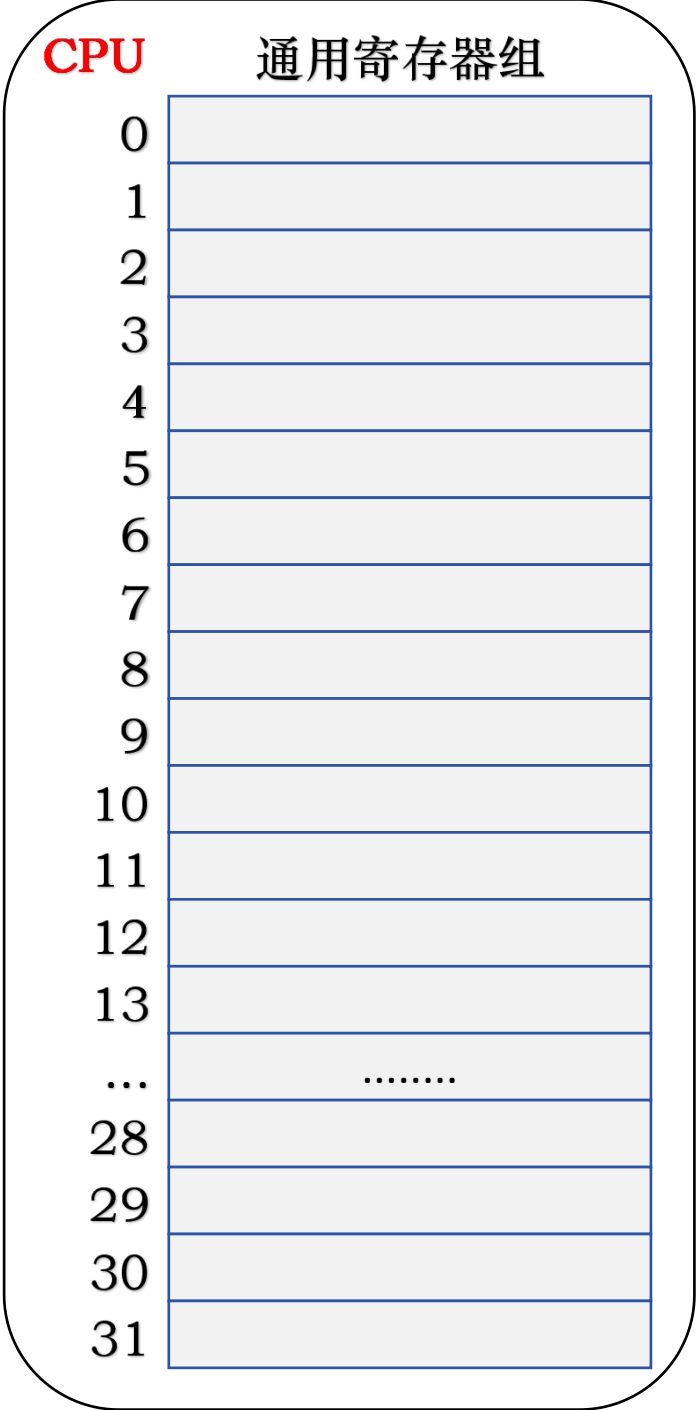
函数调用

MIPS 指令系统：寄存器组

总共 32 个通用寄存器

每个寄存器的大小是 32 位

32 位称为一个字 (word)



MIPS 指令系统：寄存器中的操作数

```
#include <stdio.h>

int add(int a, int b) {
    int c = a + b;
    return c;
}

int main() {
    int res = add(-10, 20);
    printf("res = %d", res);
    return 0;
}
```



```
li    $5, 20
li    $4, -10
jal   add
```

li: load immediate 的缩写，加载立即数（常数）

jal: jump and link 的缩写，实现函数的调用

CPU	通用寄存器组
0	
1	
2	
3	
4	0xFFFFFFFF6
5	0x00000014
6	
7	
8	
9	
10	
11	
12	
13	
...	
28	
29	
30	
31	

MIPS 指令系统：寄存器中的操作数

#include <stdio.h> addu: add unsigned 的缩写，无符号加法操作

```
int add(int a, int b) {  
    int c = a + b;  
    return c;  
}
```

➡ addu \$2,\$4,\$5

```
int main() {  
    int res = add(-10, 20);  
    printf("res = %d", res);  
    return 0;  
}
```

➡ li \$5,20
li \$4,-10
jal add

li: load immediate 的缩写，加载立即数（常数）

jal: jump and link 的缩写，实现函数的调用

CPU	通用寄存器组
0	
1	
2	0x0000000A
3	
4	0xFFFFFFFF6
5	0x00000014
6	
7	
8	
9	
10	
11	
12	
13	
...	
28	
29	
30	
31	

MIPS 指令系统：寄存器中的操作数

#include <stdio.h> addu: add unsigned 的缩写，无符号加法操作

```
int add(int a, int b) {  
    int c = a + b;  
    return c;  
}
```

➔ addu \$2,\$4,\$5

```
int main() {  
    int res = add(-10, 20);  
    printf("res = %d", res);  
    return 0;  
}
```

$$\begin{array}{r} 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 0110 \\ +\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001\ 0100 \\ \hline 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1010 \end{array}$$

CPU	通用寄存器组
0	
1	
2	0x0000000A
3	
4	0xFFFFFFFF6
5	0x00000014
6	
7	
8	
9	
10	
11	
12	
13	
...	
28	
29	
30	
31	

MIPS 指令系统：寄存器中的操作数

-20	1111 1111 1111 1111 1111 1111 1110 1100
10	+ 0000 0000 0000 0000 0000 0000 0000 1010
-10	1111 1111 1111 1111 1111 1111 1111 0110

MIPS 指令系统：寄存器中的操作数

#include <stdio.h> addu: add unsigned 的缩写，无符号加法操作

```
int add(int a, int b) {  
    int c = a + b;  
    return c;  
}
```

➡

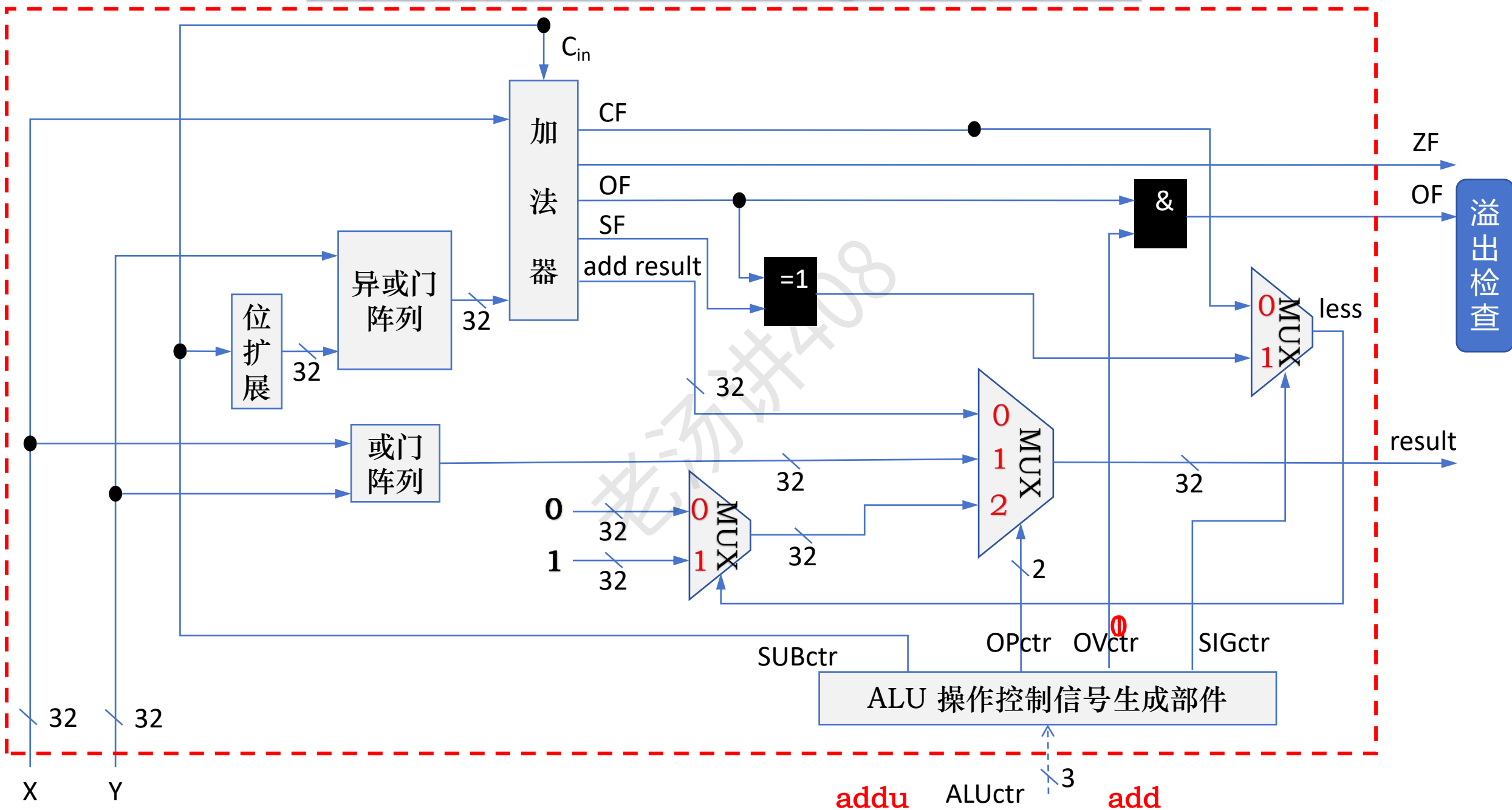
```
add    $2, $4, $5  
addu   $2, $4, $5
```

使用 addu 的原因：避免溢出检查带来的性能开销

```
int main() {  
    int res = add(-10, 20);  
    printf("res = %d", res);  
    return 0;  
}
```

CPU	通用寄存器组
0	
1	
2	0x0000000A
3	
4	0xFFFFFFFF6
5	0x00000014
6	
7	
8	
9	
10	
11	
12	
13	
...	
28	
29	
30	
31	

算术逻辑部件 (Arithmetic Logic Unit, ALU)



MIPS 指令系统：寄存器中的操作数

#include <stdio.h> addu: add unsigned 的缩写，无符号加法操作

```
int add(int a, int b) {  
    int c = a + b;  
    return c;  
}
```

➡

```
add    $2, $4, $5  
addu   $2, $4, $5
```

使用 addu 的原因：避免溢出检查带来的性能开销

```
int main() {  
    int res = add(-10, 20);  
    printf("res = %d", res);  
    return 0;  
}
```

CPU	通用寄存器组
0	
1	
2	0x0000000A
3	
4	0xFFFFFFFF6
5	0x00000014
6	
7	
8	
9	
10	
11	
12	
13	
...	
28	
29	
30	
31	

MIPS 指令系统：寄存器中的操作数

```
#include <stdio.h>
```

```
int add(int a, int b, int c, int d) {  
    int e = a + b + c + d;  
    return e;  
}
```

```
addu    $4,$4,$5  
addu    $4,$4,$6  
addu    $2,$4,$7
```

```
int main() {  
    int res = add(-10, 20, 12, 25);  
    printf("res = %d", res);  
    return 0;  
}
```

```
li      $7,25  
li      $6,12  
li      $5,20  
li      $4,-10  
jal     add
```

变量 e 和变量 res

CPU	通用寄存器组
0	
1	
2	47
3	
4	22
5	20
6	12
7	25
8	
9	
10	
11	
12	
13	
...	
28	
29	
30	
31	

对于 C 语言中某个变量，会分配一个寄存器，用于存储这个变量的值

MIPS 指令系统：寄存器中的操作数

```
#include <stdio.h>
```

```
int add(int a, int b, int c, int d) {  
    int e = (a + b) - (c + d);  
    return e;  
}
```

```
int main() {  
    int res = add(-10, 20, 12, 25);  
    printf("res = %d", res);  
    return 0;  
}
```

```
addu    $2,$4,$5  
addu    $6,$6,$7  
subu    $2,$2,$6
```

```
li      $7,25  
li      $6,12  
li      $5,20  
li      $4,-10  
jal     add
```

变量 e 和变量 res

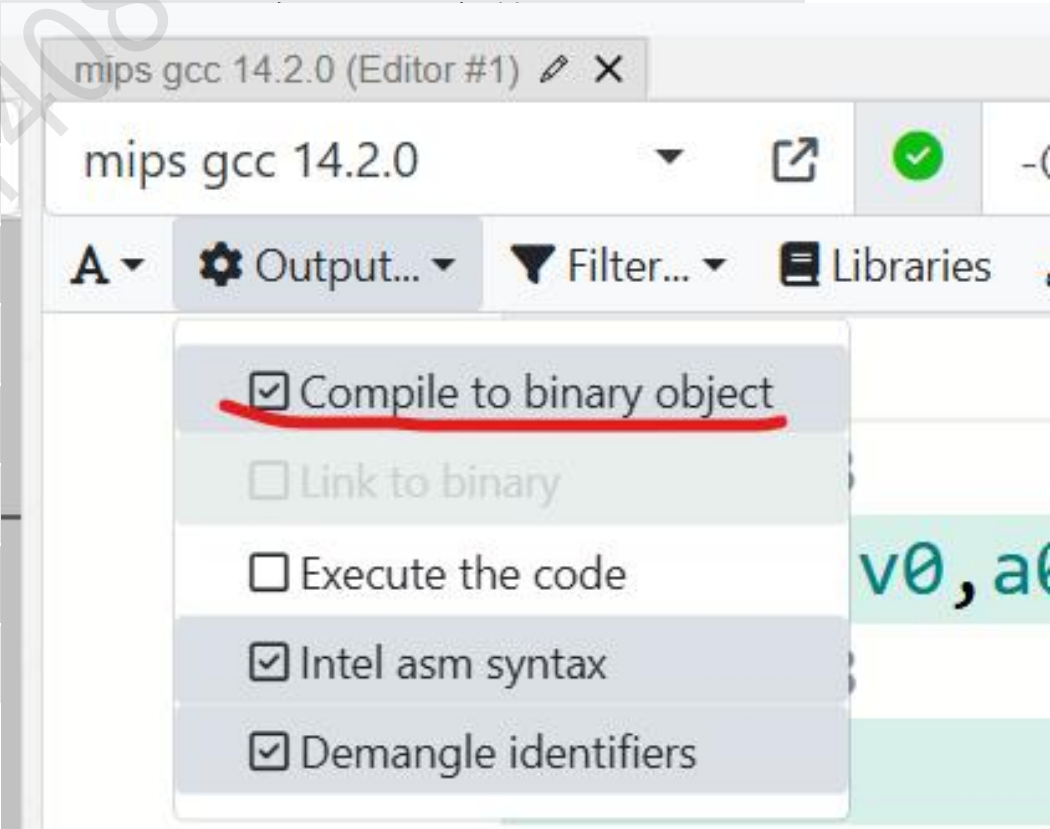
CPU	通用寄存器组
0	
1	
2	-27
3	
4	-10
5	20
6	37
7	25
8	
9	
10	
11	
12	
13	
...	
28	
29	
30	
31	

MIPS 的算术运算指令 addu 和 subu 的操作数必须来自寄存器，而且结果也必须写入到某个寄存器中

寄存器寻址

MIPS 指令系统：通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0 ~ v1	2 ~ 3	过程调用返回值
a0 ~ a3	4 ~ 7	
t0 ~ t7	8 ~ 15	
s0 ~ s7	16 ~ 23	
t8 ~ t9	24 ~ 25	
k0 ~ k1	26 ~ 27	
gp	28	
sp	29	
fp	30	
ra	31	



MIPS 指令系统：通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0 ~ v1	2 ~ 3	过程调用返回值
a0 ~ a3	4 ~ 7	过程调用参数
t0 ~ t7	8 ~ 15	临时变量，在被调用过程无需保存
s0 ~ s7	16 ~ 23	在被调用过程需保存
t8 ~ t9	24 ~ 25	临时变量，在被调用过程无需保存
k0 ~ k1	26 ~ 27	为 OS 保留
gp	28	全局指针
sp	29	栈指针
fp	30	帧指针
ra	31	过程调用返回地址

计算机体系结构中的 7 个伟大思想：

① 使用抽象分层简化设计

② 加速经常性事件

③ 通过并行提高性能

④ 通过流水线提高性能

⑤ 通过预测提高性能

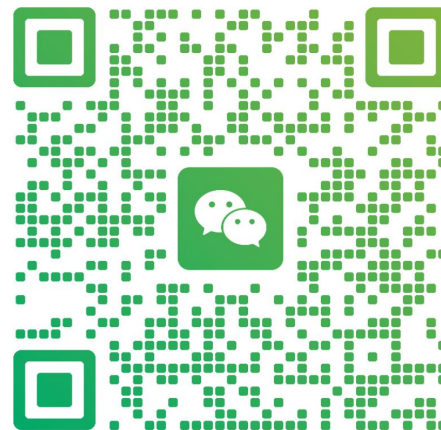
⑥ 存储层次

⑦ 通过冗余提高可靠性

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

MIPS 指令系统：通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0 ~ v1	2 ~ 3	过程调用返回值
a0 ~ a3	4 ~ 7	过程调用参数
t0 ~ t7	8 ~ 15	临时变量，在被调用过程无需保存
s0 ~ s7	16 ~ 23	在被调用过程需保存
t8 ~ t9	24 ~ 25	临时变量，在被调用过程无需保存
k0 ~ k1	26 ~ 27	为 OS 保留
gp	28	全局指针
sp	29	栈指针
fp	30	帧指针
ra	31	过程调用返回地址

简化指令集： addu \$2, \$4, \$0 ←———— move \$2, \$4 伪指令

MIPS 指令系统：通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0 ~ v1	2 ~ 3	过程调用返回值
a0 ~ a3	4 ~ 7	过程调用参数
t0 ~ t7	8 ~ 15	临时变量，在被调用过程无需保存
s0 ~ s7	16 ~ 23	在被调用过程需保存
t8 ~ t9	24 ~ 25	临时变量，在被调用过程无需保存
k0 ~ k1	26 ~ 27	为 OS 保留
gp	28	全局指针
sp	29	栈指针
fp	30	帧指针
ra	31	过程调用返回地址

MIPS 指令系统：寄存器中的操作数

```
#include <stdio.h>
```

```
int add(int a, int b, int c, int d) {  
    int e = (a + b) - (c + d);  
    return e;  
}
```

```
int main() {  
    int res = add(-10, 20, 12, 25);  
    printf("res = %d", res);  
    return 0;  
}
```

```
addu    $2,$4,$5  
addu    $6,$6,$7  
subu    $2,$2,$6
```

```
li      $7,25  
li      $6,12  
li      $5,20  
li      $4,-10  
jal     add
```

变量 e 和变量 res

CPU	通用寄存器组
0	
1	
2	-27
3	
4	-10
5	20
6	37
7	25
8	
9	
10	
11	
12	
13	
...	
28	
29	
30	
31	

MIPS 的算术运算指令 addu 和 subu 的操作数必须来自寄存器，而且结果也必须写入到某个寄存器中

寄存器寻址

MIPS 指令系统：通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0 ~ v1	2 ~ 3	过程调用返回值
a0 ~ a3	4 ~ 7	过程调用参数
t0 ~ t7	8 ~ 15	临时变量，在被调用过程无需保存
s0 ~ s7	16 ~ 23	在被调用过程需保存
t8 ~ t9	24 ~ 25	临时变量，在被调用过程无需保存
k0 ~ k1	26 ~ 27	为 OS 保留
gp	28	全局指针
sp	29	栈指针
fp	30	帧指针
ra	31	过程调用返回地址

MIPS 指令系统：寄存器中的操作数

```
#include <stdio.h>
```

```
int add(int a, int b, int c, int d) {  
    int e = (a + b) - (c + d);  
    return e;  
}
```

```
int main() {  
    int res = add(-10, 20, 12, 25);  
    printf("res = %d", res);  
    return 0;  
}
```

```
addu    $2,$4,$5  
addu    $6,$6,$7  
subu    $2,$2,$6
```

```
li      $7,25  
li      $6,12  
li      $5,20  
li      $4,-10  
jal     add
```

变量 e 和变量 res

CPU	通用寄存器组
0	
1	
2	-27
3	
4	-10
5	20
6	37
7	25
8	
9	
10	
11	
12	
13	
...	
28	
29	
30	
31	

MIPS 的算术运算指令 addu 和 subu 的操作数必须来自寄存器，而且结果也必须写入到某个寄存器中

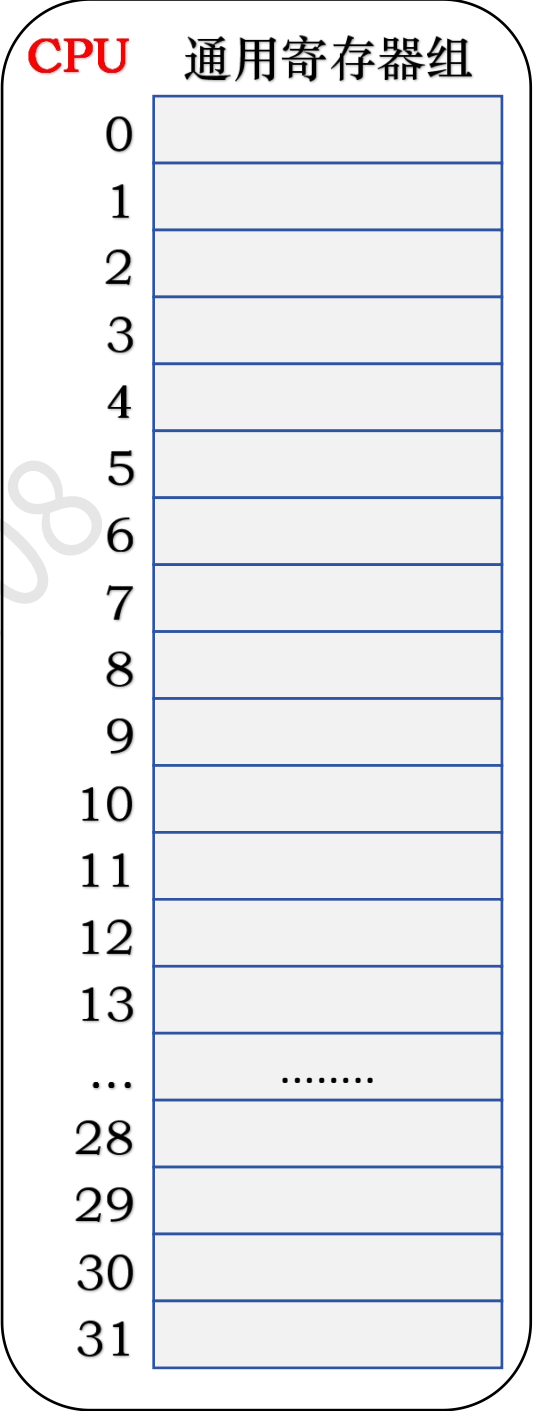
寄存器寻址

MIPS 指令系统：通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0 ~ v1	2 ~ 3	过程调用返回值
a0 ~ a3	4 ~ 7	过程调用参数
t0 ~ t7	8 ~ 15	临时变量，在被调用过程无需保存
s0 ~ s7	16 ~ 23	在被调用过程需保存
t8 ~ t9	24 ~ 25	临时变量，在被调用过程无需保存
k0 ~ k1	26 ~ 27	为 OS 保留
gp	28	全局指针
sp	29	栈指针
fp	30	帧指针
ra	31	过程调用返回地址

MIPS 指令系统：寄存器中的操作数

addu	\$2,\$4,\$5
addu	\$6,\$6,\$7
subu	\$2,\$2,\$6



MIPS 指令系统：主存储器中的操作数

```
#include <stdio.h>

int add_3(int A[], int h) {
    int g = h + A[3];
    return g;
}

int main() {
    int A[] = {1, 2, 3, 4, 5, 6};
    int res = add_3(A, 10);
    printf("res = %d\n", res);
    return 0;
}
```

→

li \$2, 1

sw \$2, 24(\$sp)

li \$2, 2

sw \$2, 28(\$sp)

li \$2, 3

sw \$2, 32(\$sp)

li \$2, 4

sw \$2, 36(\$sp)

li \$2, 5

sw \$2, 40(\$sp)

li \$2, 6

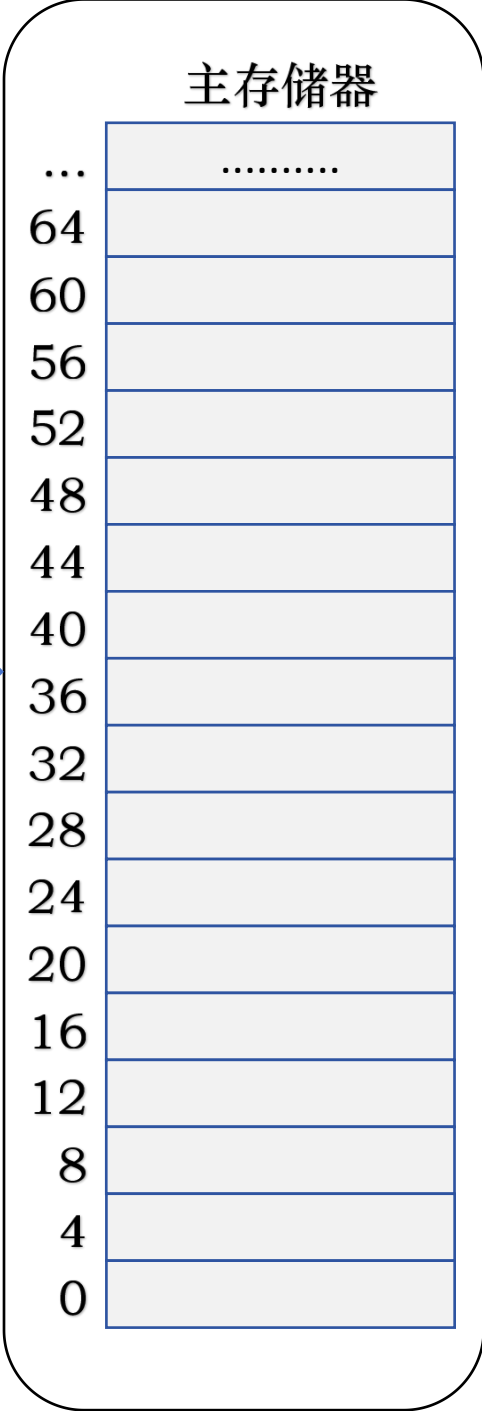
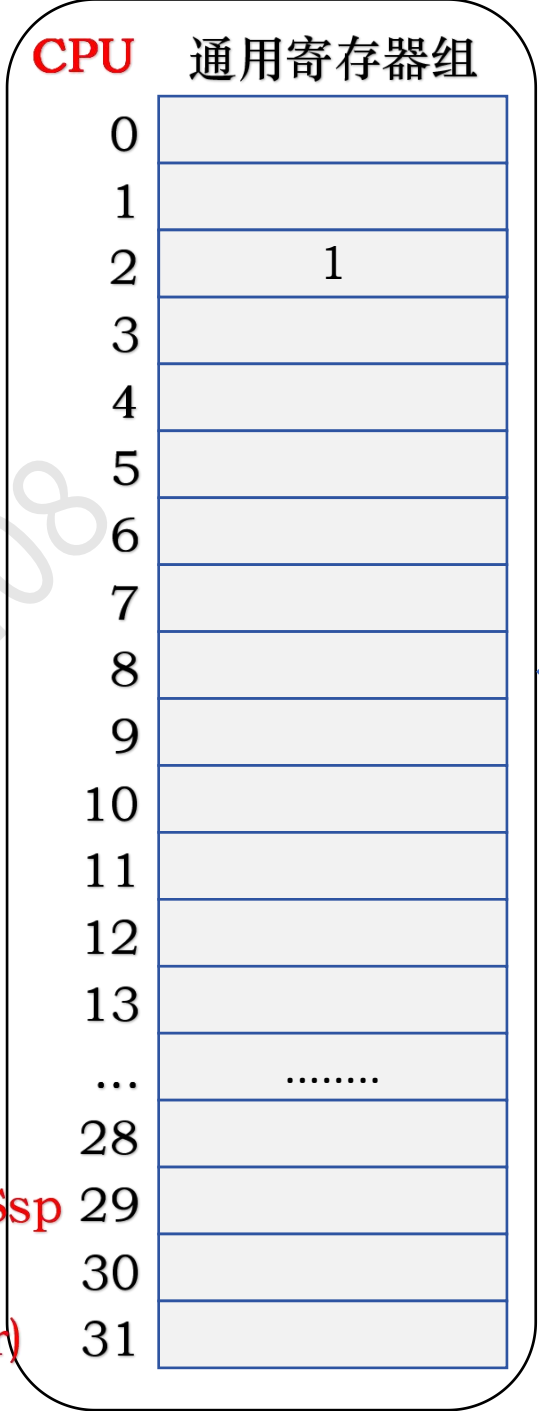
sw \$2, 44(\$sp)

li \$5, 10

addiu \$4, \$sp, 24

jal add_3

↗



\$sp
保存栈指针(stack pointer)

MIPS 指令系统：通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0 ~ v1	2 ~ 3	过程调用返回值
a0 ~ a3	4 ~ 7	过程调用参数
t0 ~ t7	8 ~ 15	临时变量，在被调用过程无需保存
s0 ~ s7	16 ~ 23	在被调用过程需保存
t8 ~ t9	24 ~ 25	临时变量，在被调用过程无需保存
k0 ~ k1	26 ~ 27	为 OS 保留
gp	28	全局指针
sp	29	栈指针
fp	30	帧指针
ra	31	过程调用返回地址

MIPS 指令系统：主存储器中的操作数

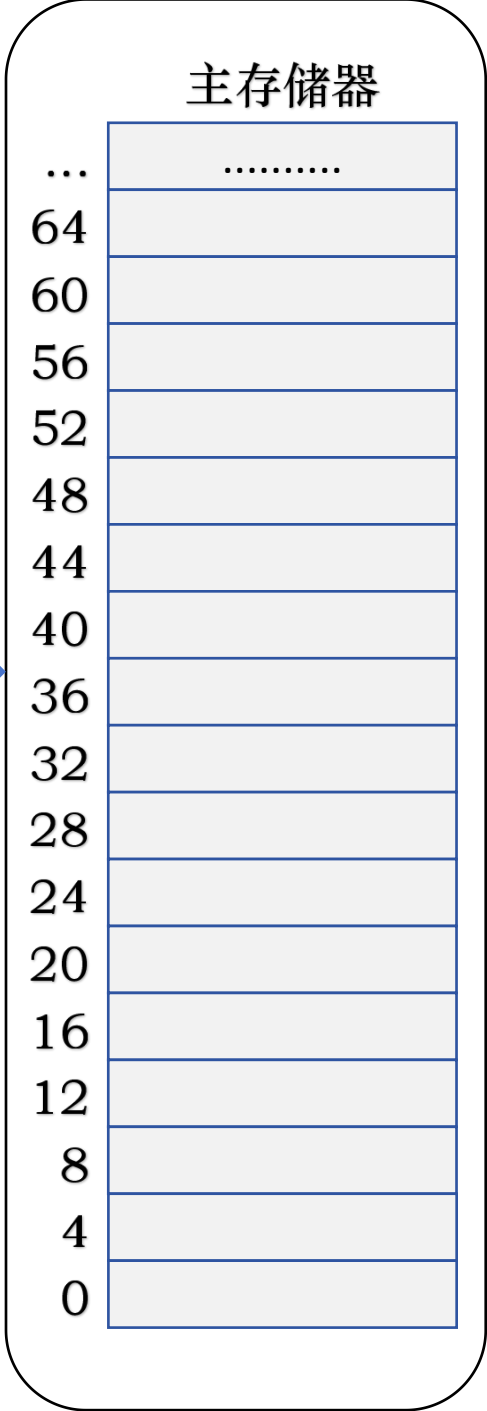
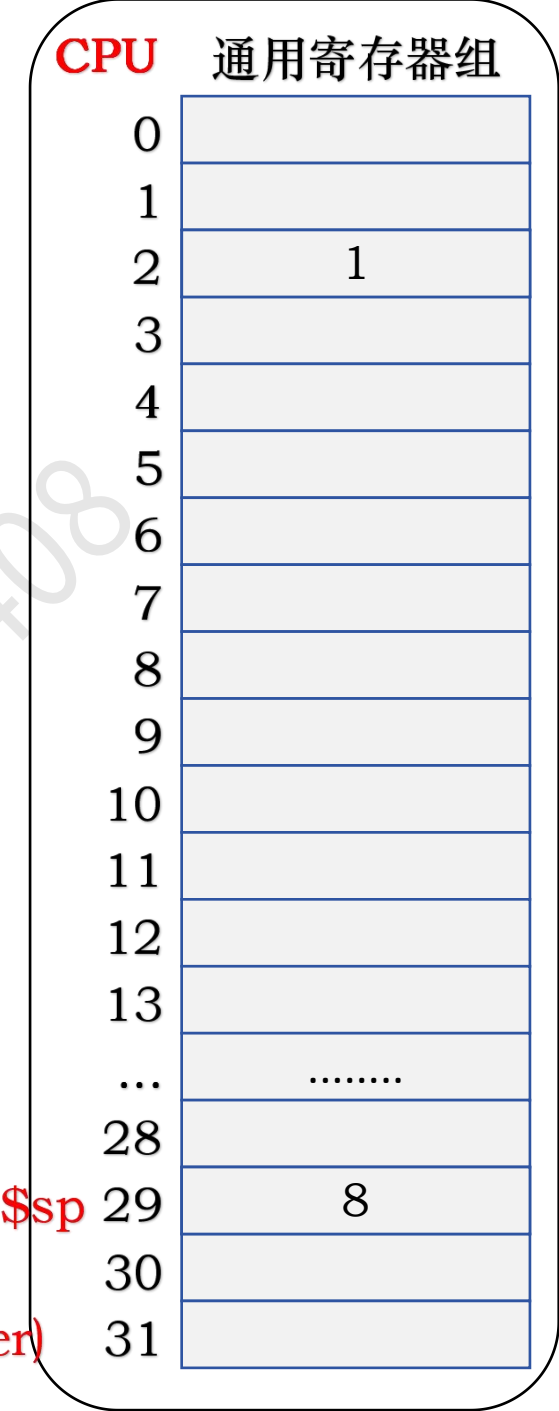
```
#include <stdio.h>

int add_3(int A[], int h) {
    int g = h + A[3];
    return g;
}

int main() {
    int A[] = {1, 2, 3, 4, 5, 6};
    int res = add_3(A, 10);
    printf("res = %d\n", res);
    return 0;
}
```



```
li    $2,1
sw    $2,24($sp)
li    $2,2
sw    $2,28($sp)
li    $2,3
sw    $2,32($sp)
li    $2,4
sw    $2,36($sp)
li    $2,5
sw    $2,40($sp)
li    $2,6
sw    $2,44($sp)
li    $5,10
addiu $4,$sp,24
jal   add_3
```



\$sp
保存栈指针(stack pointer)

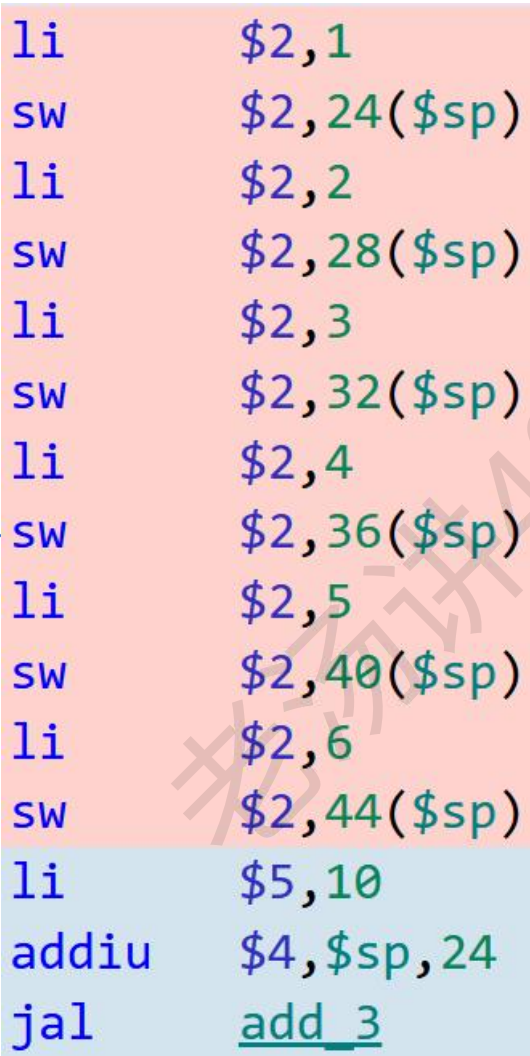
MIPS 指令系统：主存储器中的操作数

```
#include <stdio.h>

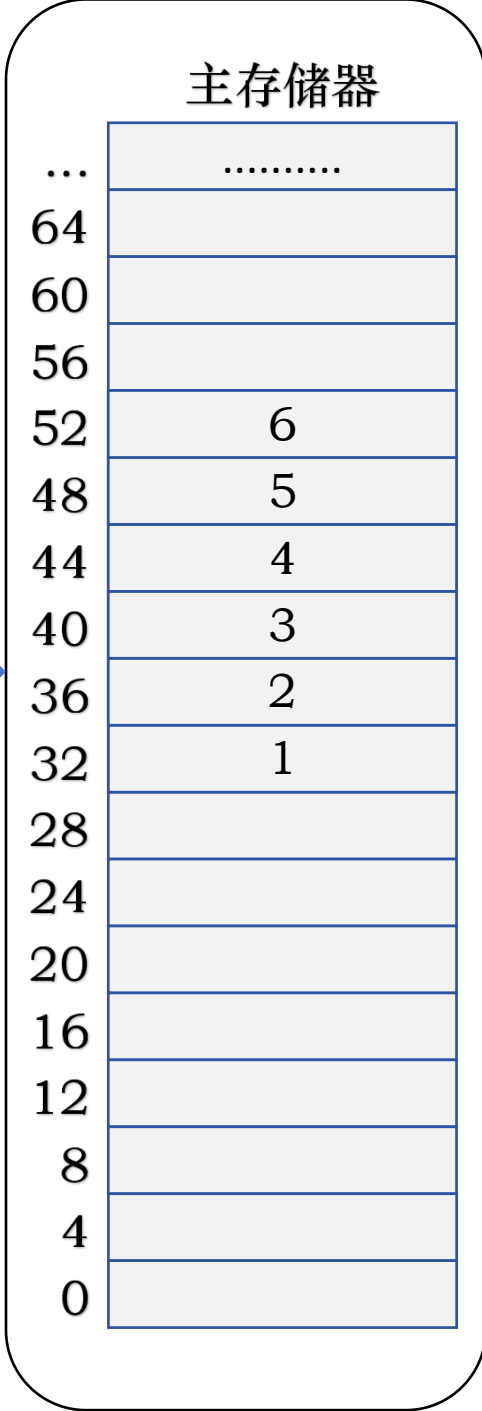
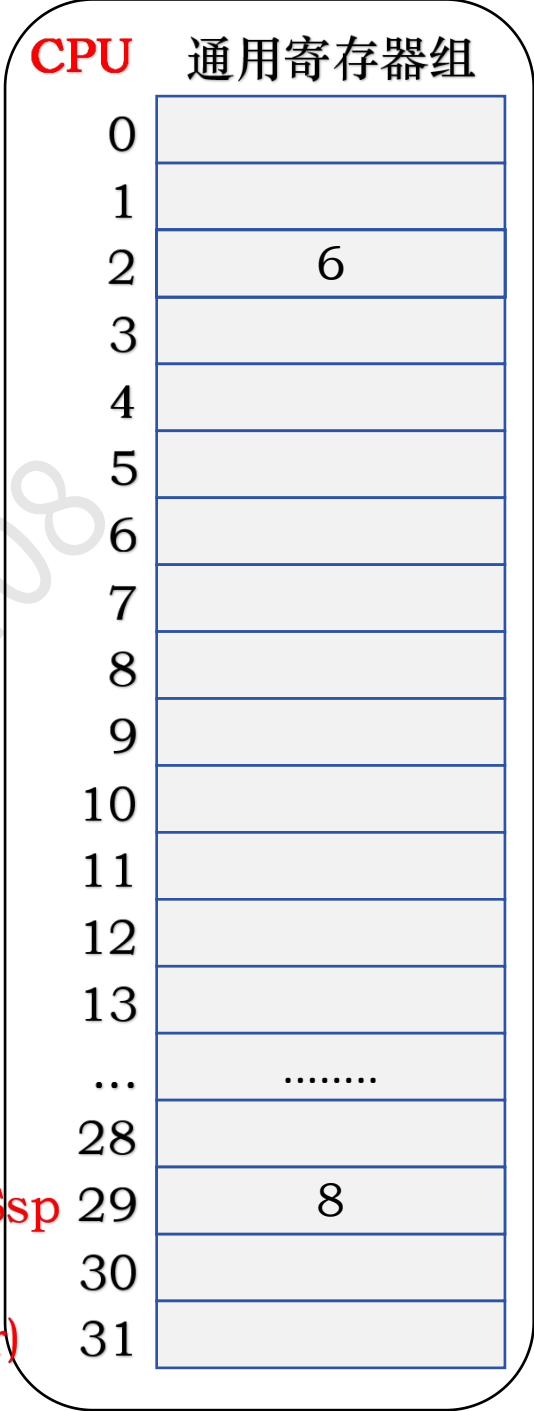
int add_3(int A[], int h) {
    int g = h + A[3];
    return g;
}

int main() {
    int A[] = {1, 2, 3, 4, 5, 6};
    int res = add_3(A, 10);
    printf("res = %d\n", res);
    return 0;
}
```

sw: store word 的缩写，存数指令，
将寄存器中的一个字的数据（也就是 4
字节的数据）存到指定内存单元中



\$sp
保存栈指针(stack pointer)



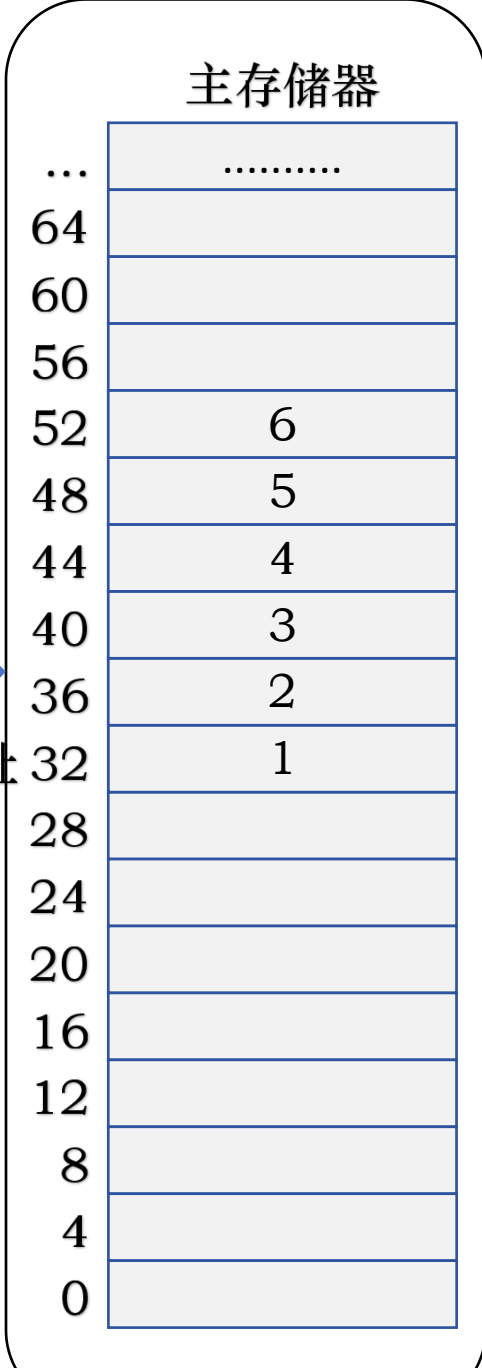
MIPS 指令系统：主存储器中的操作数

```
#include <stdio.h>

int add_3(int A[], int h) {
    int g = h + A[3];
    return g;
}

int main() {
    int A[] = {1, 2, 3, 4, 5, 6};
    int res = add_3(A, 10);
    printf("res = %d\n", res);
    return 0;
}
```

```
li    $2, 1
sw    $2, 24($sp)
li    $2, 2
sw    $2, 28($sp)
li    $2, 3
sw    $2, 32($sp)
li    $2, 4
sw    $2, 36($sp)
li    $2, 5
sw    $2, 40($sp)
li    $2, 6
sw    $2, 44($sp)
li    $5, 10
addiu $4, $sp, 24
jal   add_3
```



基地址

addiu: add immediate unsigned 的缩写，加法指令，
将寄存器的值和立即数做无符号加法

保存栈指针(stack pointer)

MIPS 指令系统：主存储器中的操作数

```
#include <stdio.h>
```

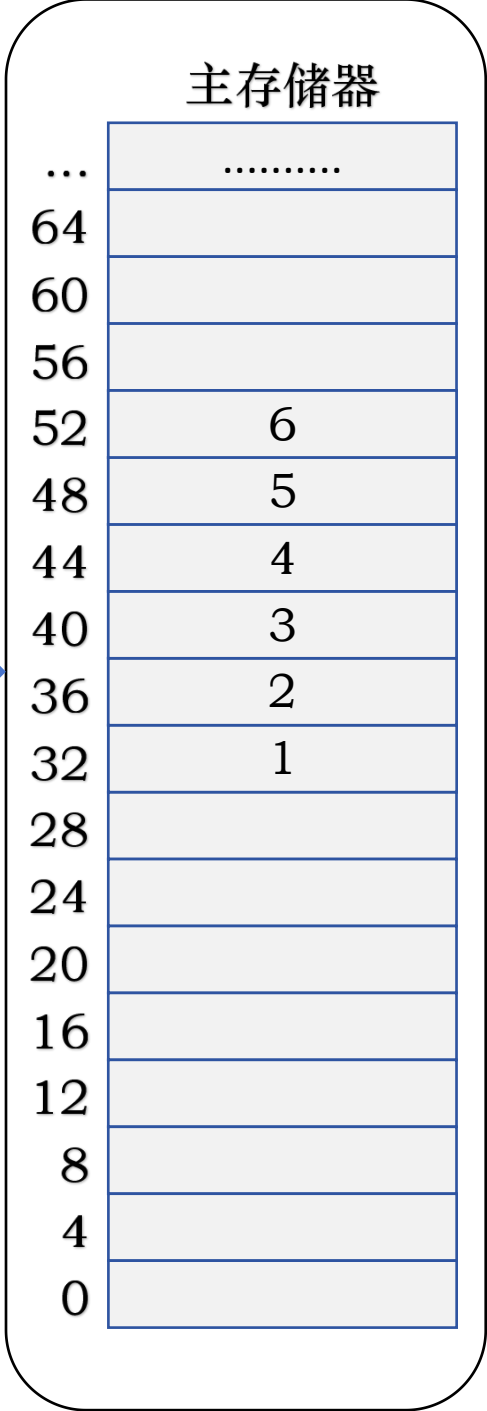
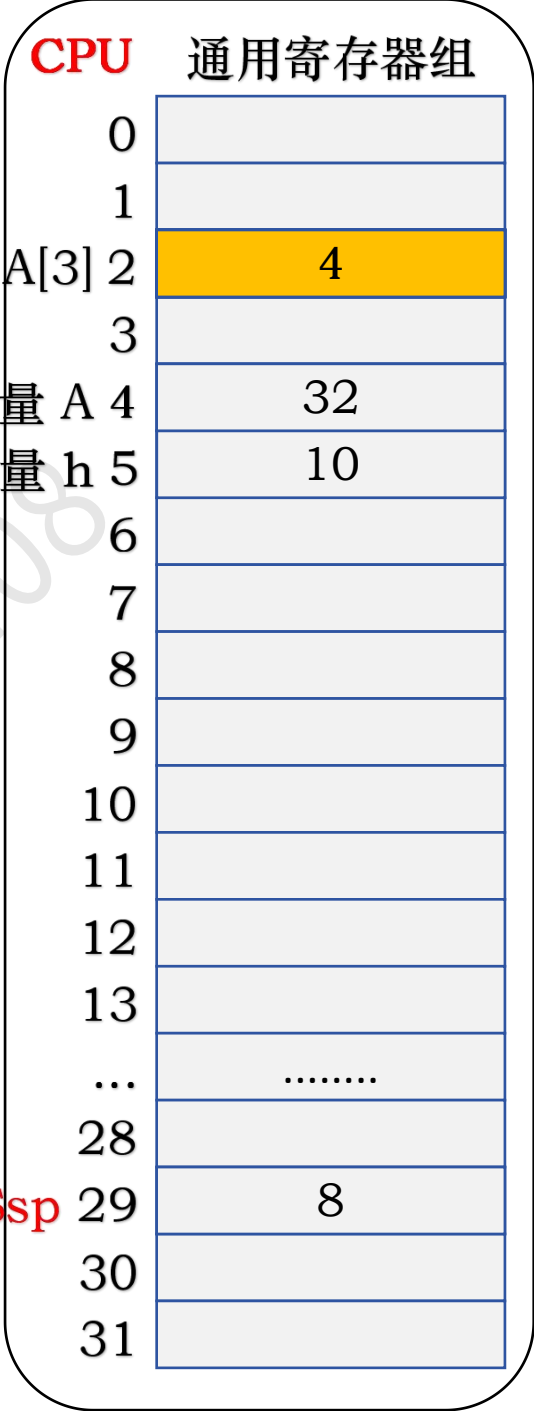
```
int add_3(int A[], int h) {  
    int g = h + A[3];  
    return g;  
}
```

3 × 4 字节 = 12 字节

```
lw    $2, 12($4)  
nop  
addu  $2, $2, $5
```

```
int main() {  
    int A[] = {1, 2, 3, 4, 5, 6};  
    int res = add_3(A, 10);  
    printf("res = %d\n", res);  
    return 0;  
}
```

lw: load word 的缩写，取数指令，
从指定内存地址存储单元加载一个字
(也就是 4 字节数据) 到指定寄存器

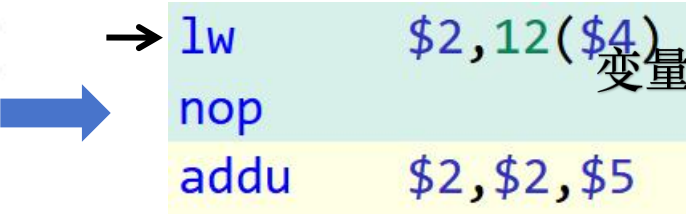


保存当前栈顶指针

MIPS 指令系统：主存储器中的操作数

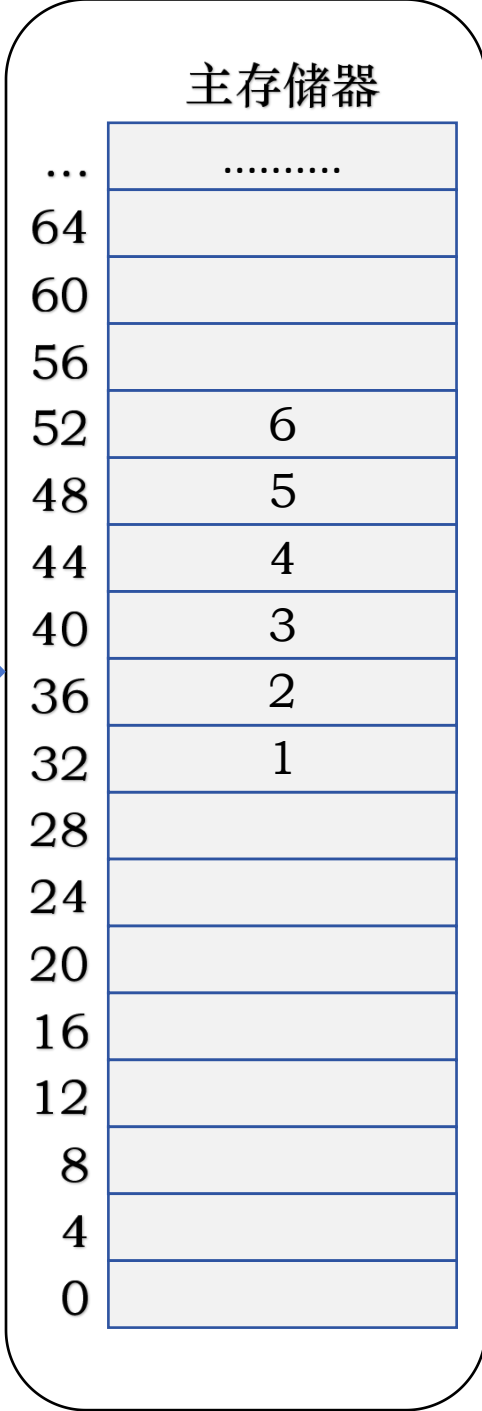
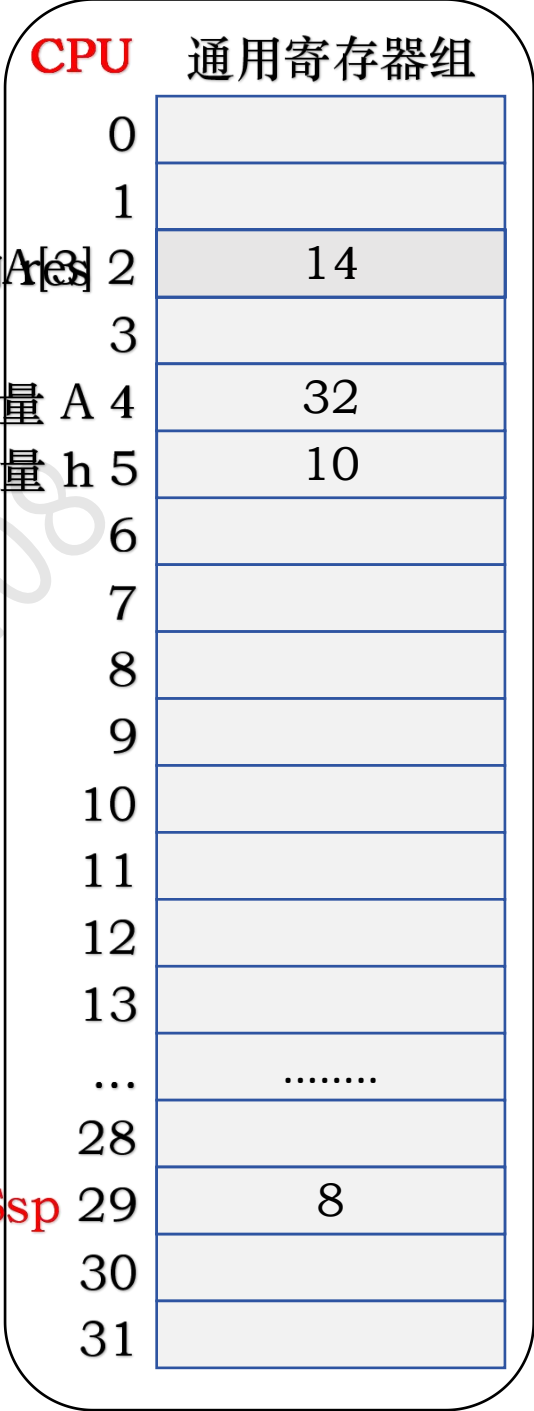
```
#include <stdio.h>
```

```
int add_3(int A[], int h) {  
    int g = h + A[3];  
    return g;  
}
```



```
int main() {  
    int A[] = {1, 2, 3, 4, 5, 6};  
    int res = add_3(A, 10);  
    printf("res = %d\n", res);  
    return 0;  
}
```

nop: no operation 的缩写，空操作指令，
作用一：用于满足内存对齐的要求（指令对齐）
作用二：在流水线中，解决指令之间的相关性问题



MIPS 指令系统：主存储器中的操作数

数据传送指令： 在主存储器和寄存器之间传送数据指令

sw: store word, 用于将指定寄存器中的一个字 (32 位) 数据传送到指定内存地址的内存单元中

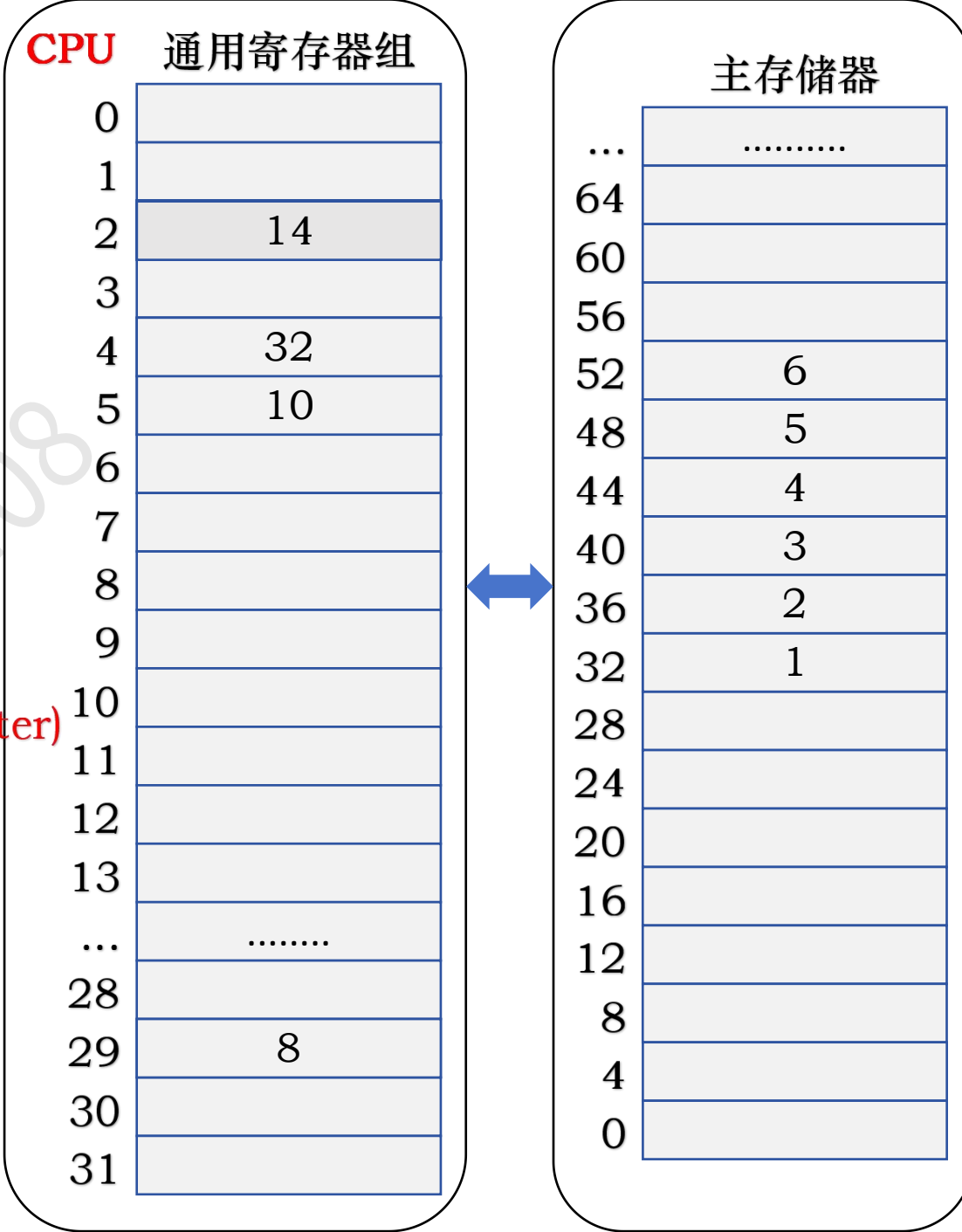
sw \$2, 24(\$sp) 寄存器 \$2 → 内存地址 (\$sp) + 24

偏移量 (offset) \$sp 也可以称为基址寄存器 (base register)

lw \$2, 12(\$sp) 寄存器 \$2 ← 内存地址 (\$sp) + 12

lw: load word, 用于将指定内存地址的一个字 (32 位) 数据传送到指定寄存器中

内存寻址范围：0 ~ 2³² - 1 字节，也就是 4GB



MIPS 指令系统：主存储器中的操作数

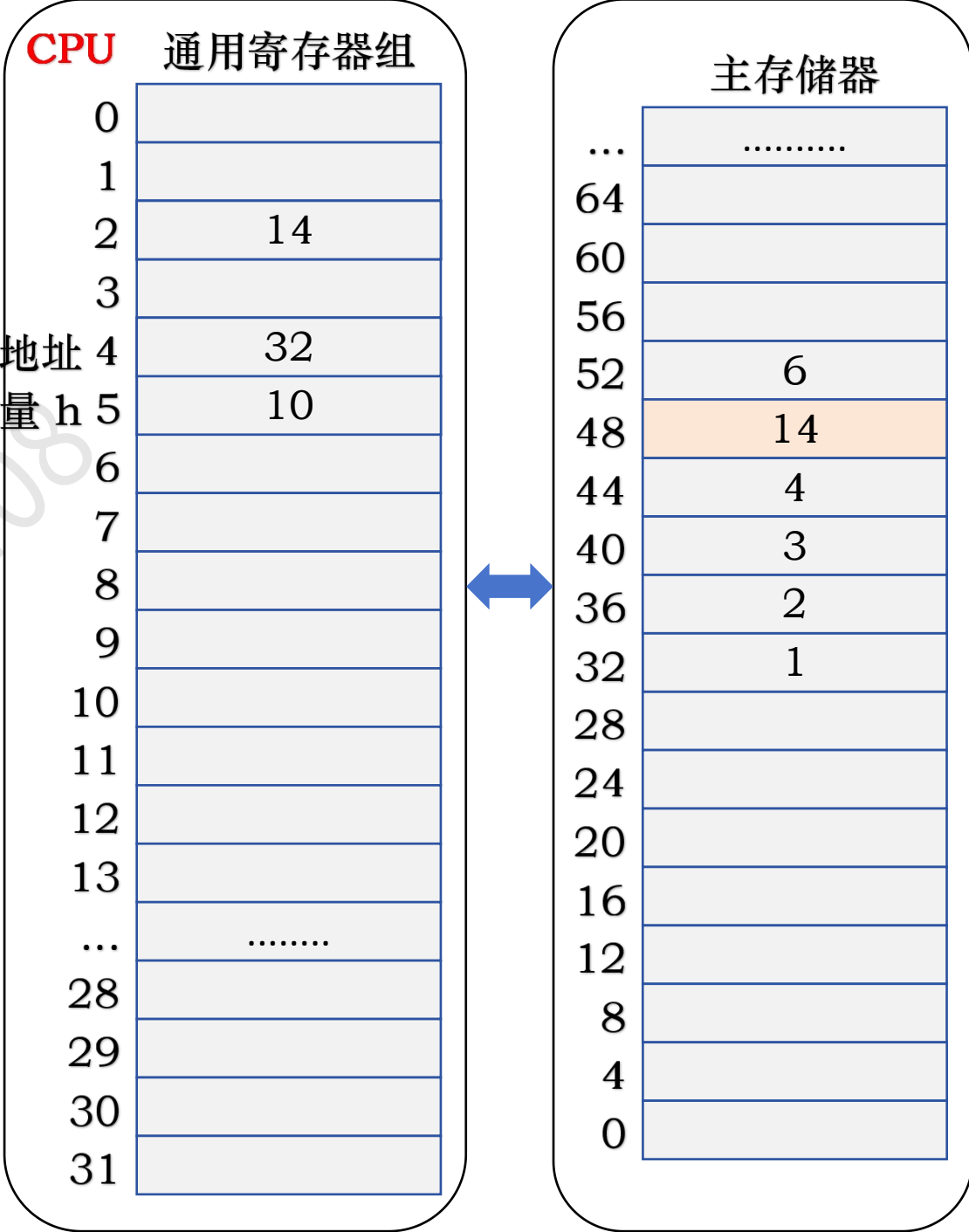
```
void add_ele(int A[], int h) {  
    A[4] = h + A[3];  
}
```

加载 A[3] 到 2 号寄存器: lw \$2, 12(\$4)

将 h 和 A[3] 相加: addu \$2, \$2, \$5

将加法结果存到 A[4] 中: sw \$2, 16(\$4)

数组 A 的基地址
变量 h



MIPS 指令系统：寻址方式

`addu $2, $2, $5`

寄存器寻址

`lw $2, 12($4)`

基址寻址

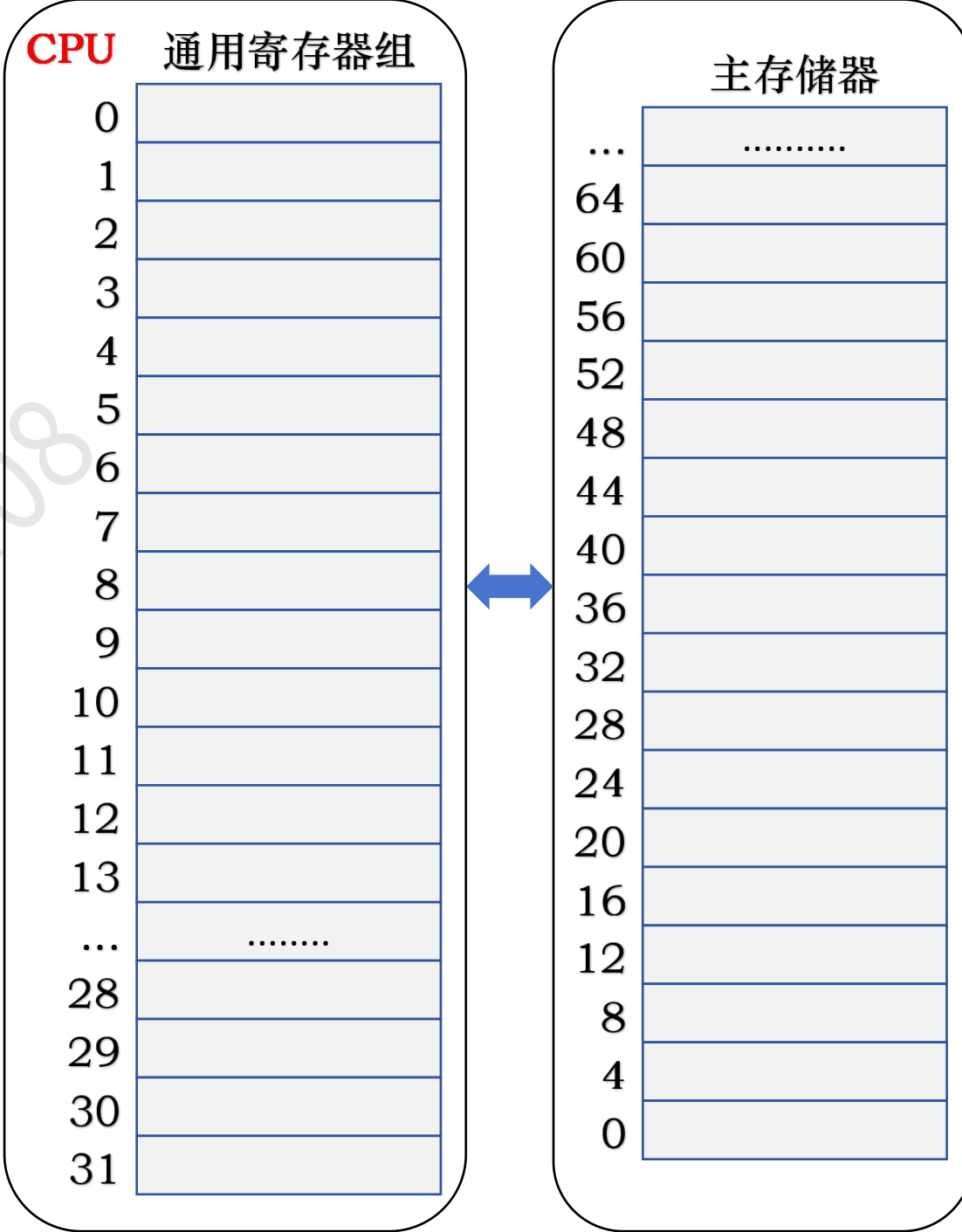
`addiu $2, $sp, 24`

立即寻址

立即数 (immediate)

```
int add_3(int a) {  
    int g = a + 3;  
    return g;  
}
```

➡ `addiu $2,$4,3`



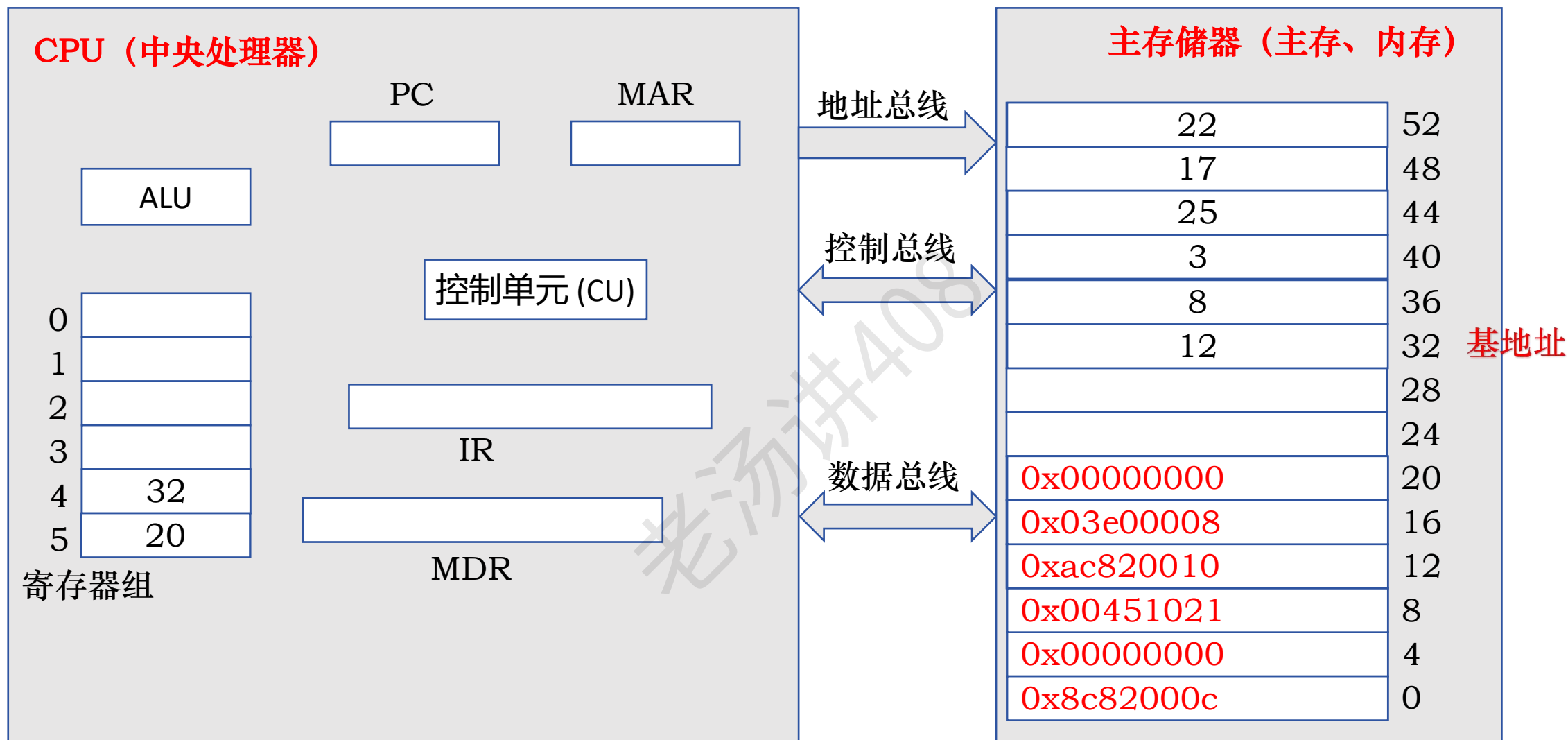
指令整体执行流程

```
void add_ele(int A[], int h) {  
    A[4] = h + A[3];  
}
```

add_ele:

lw	\$2,12(\$4)
nop	
addu	\$2,\$2,\$5
sw	\$2,16(\$4)
jr	\$31
nop	

指令整体执行流程



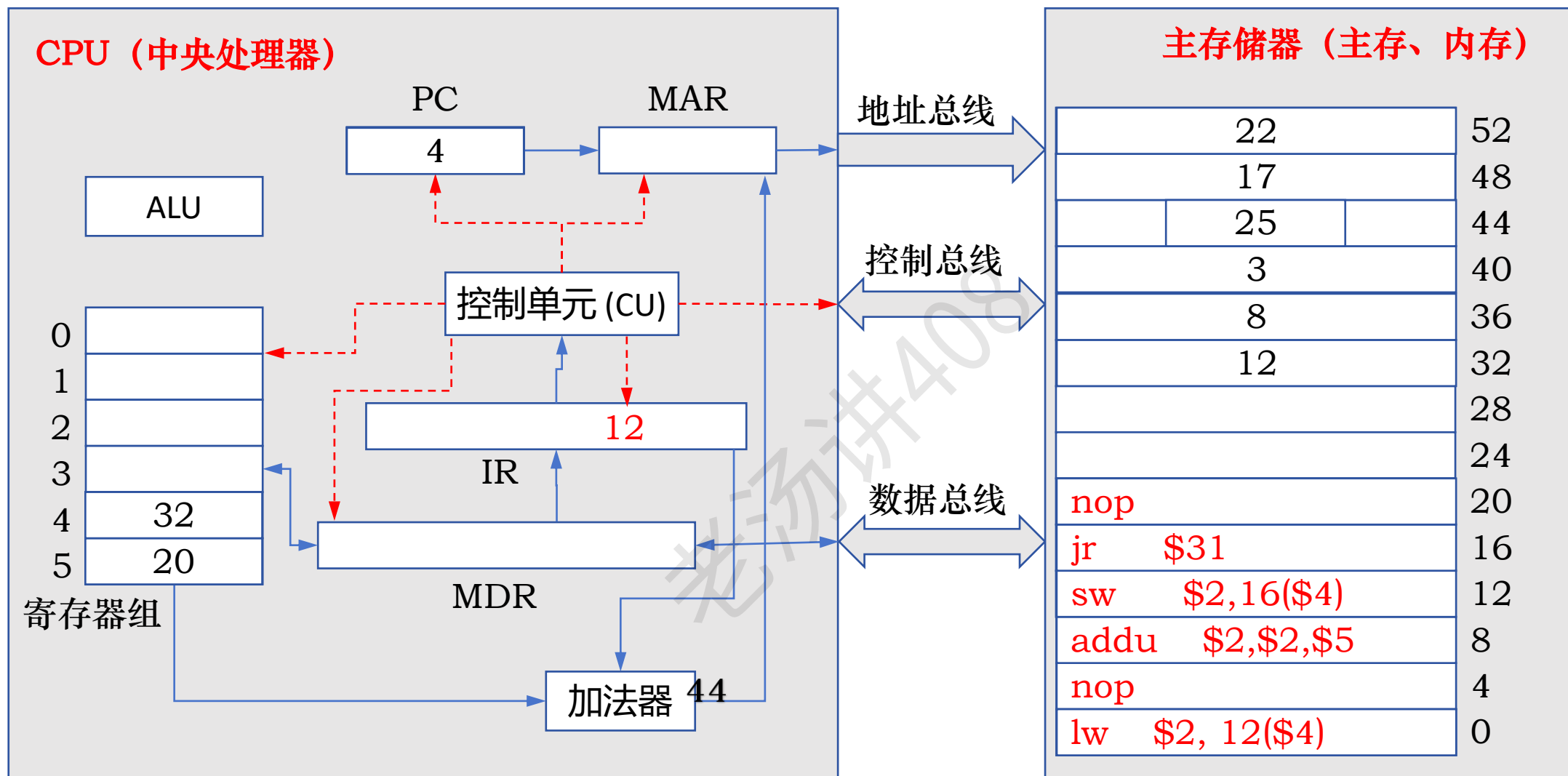
内存地址寄存器 (Memory Address Register, MAR)

内存数据寄存器 (Memory Data Register, MDR)

指令寄存器 (Instruction Register, IR)

```
void add_ele(int A[], int h) {  
    A[4] = h + A[3];  
}
```

指令整体执行流程



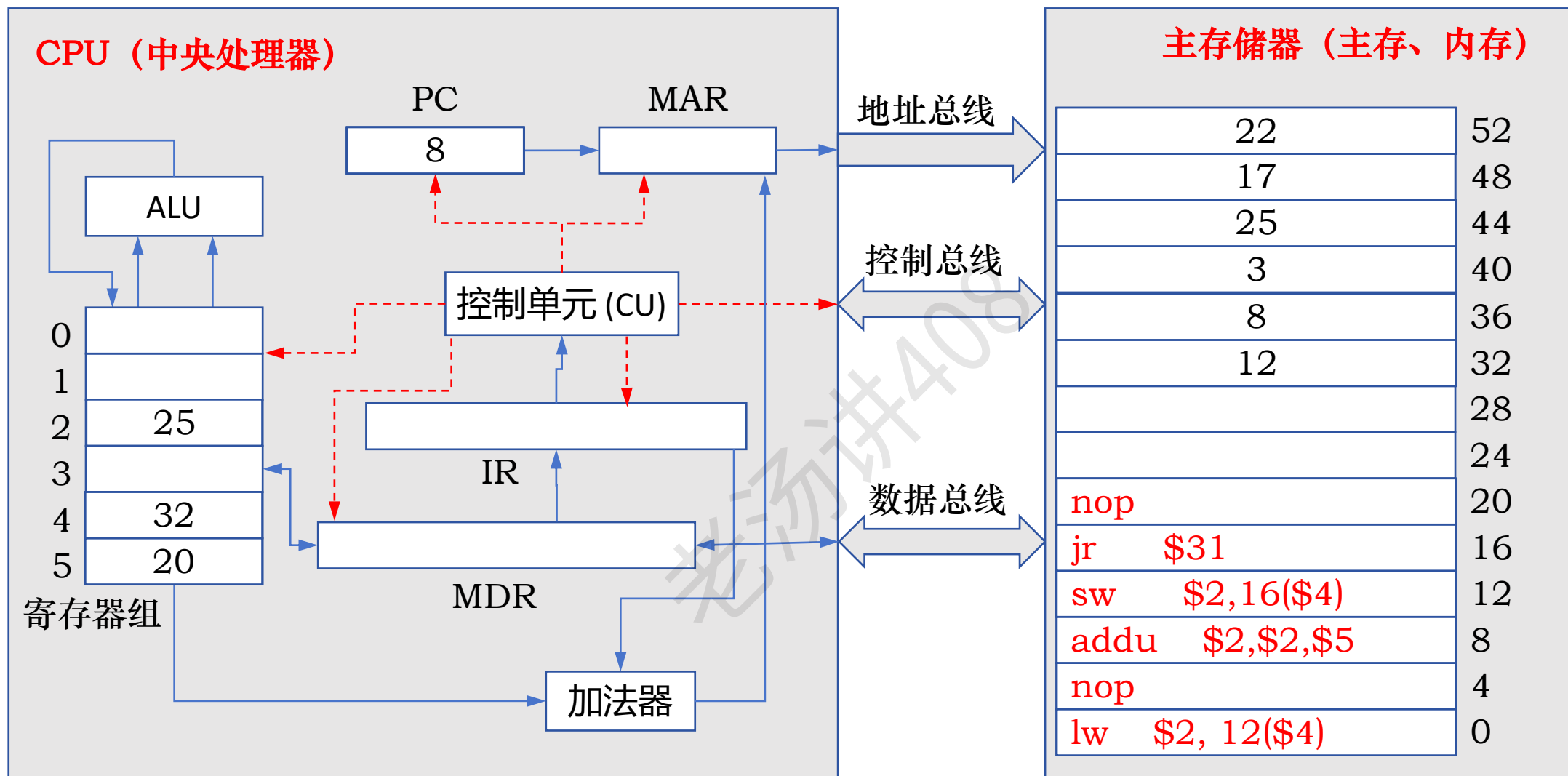
内存地址寄存器 (Memory Address Register, MAR)

内存数据寄存器 (Memory Data Register, MDR)

指令寄存器 (Instruction Register, IR)

```
void add_ele(int A[], int h) {  
    A[4] = h + A[3];  
}
```

指令整体执行流程



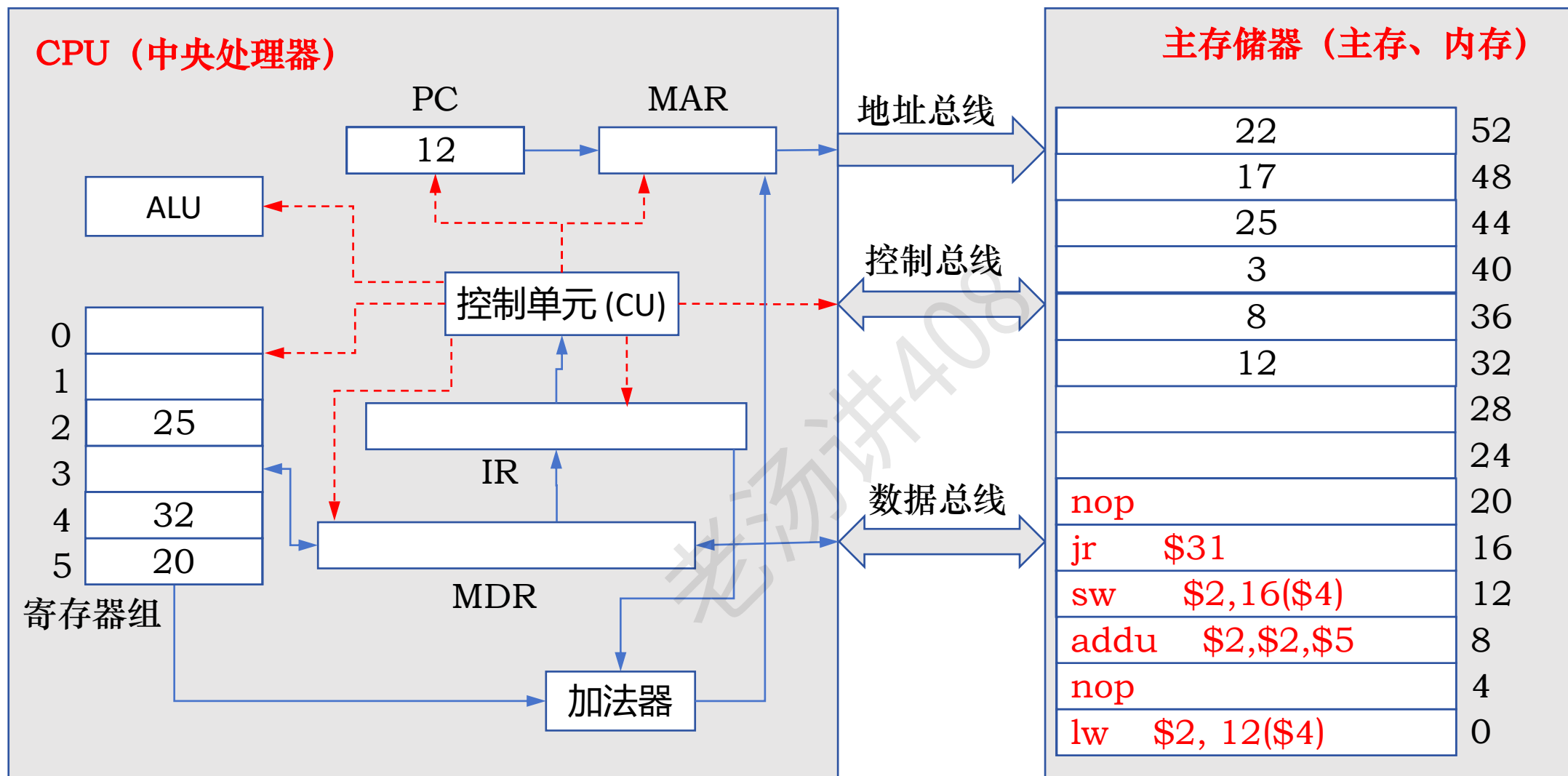
内存地址寄存器 (Memory Address Register, MAR)

内存数据寄存器 (Memory Data Register, MDR)

指令寄存器 (Instruction Register, IR)

```
void add_ele(int A[], int h) {  
    A[4] = h + A[3];  
}
```


指令整体执行流程



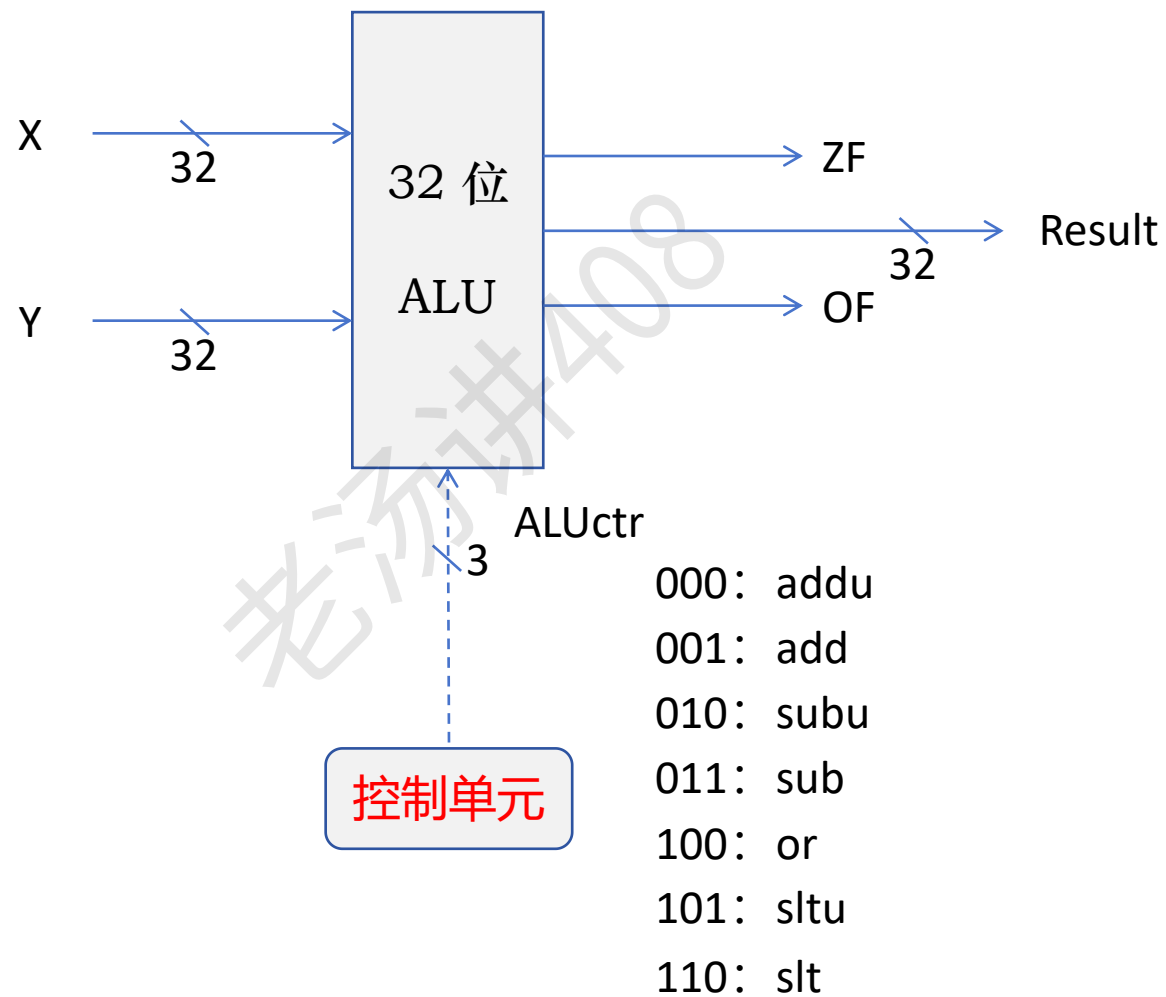
内存地址寄存器 (Memory Address Register, MAR)

内存数据寄存器 (Memory Data Register, MDR)

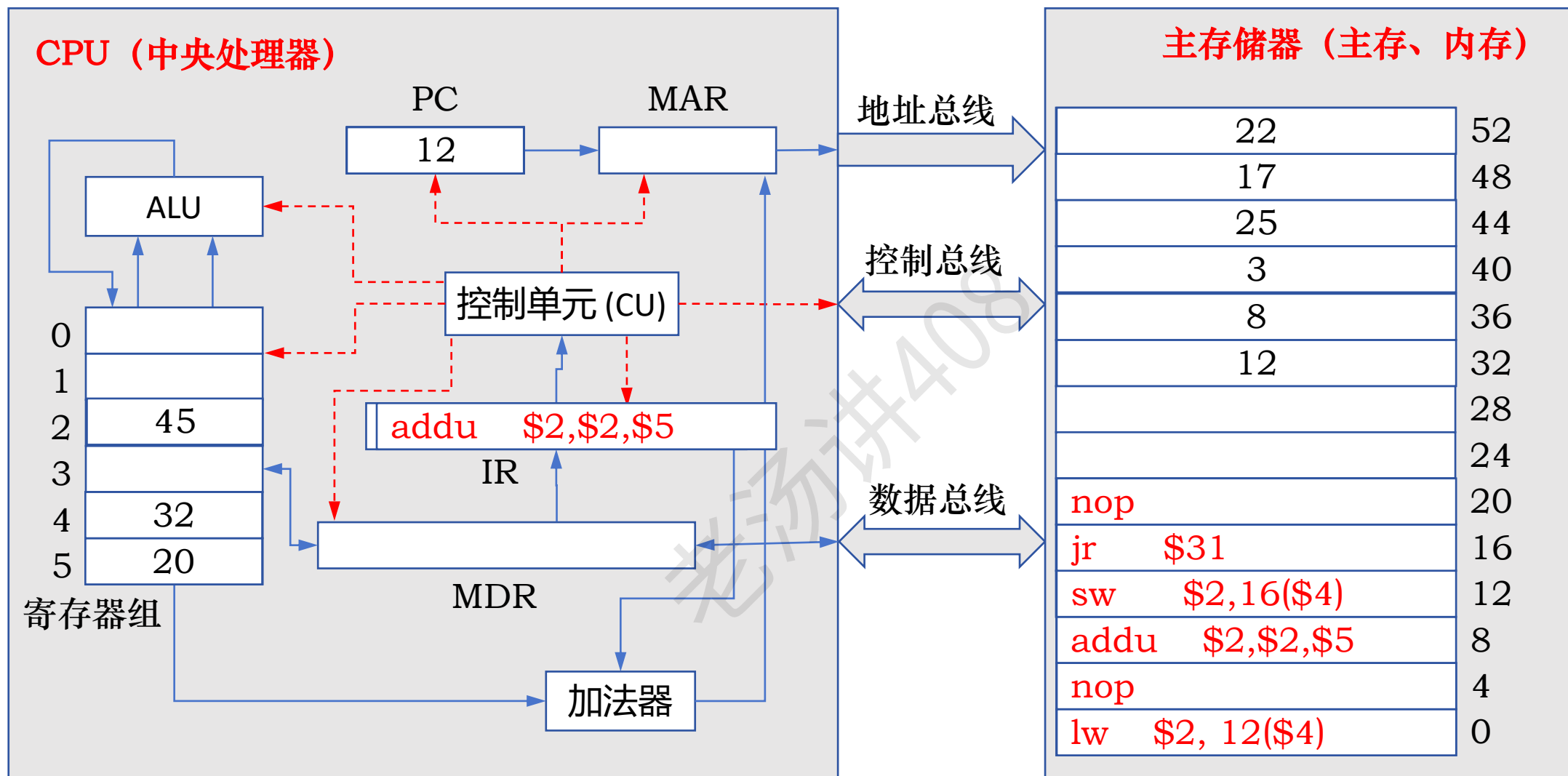
指令寄存器 (Instruction Register, IR)

```
void add_ele(int A[], int h) {  
    A[4] = h + A[3];  
}
```

算术逻辑部件 (Arithmetic Logic Unit, ALU)



指令整体执行流程



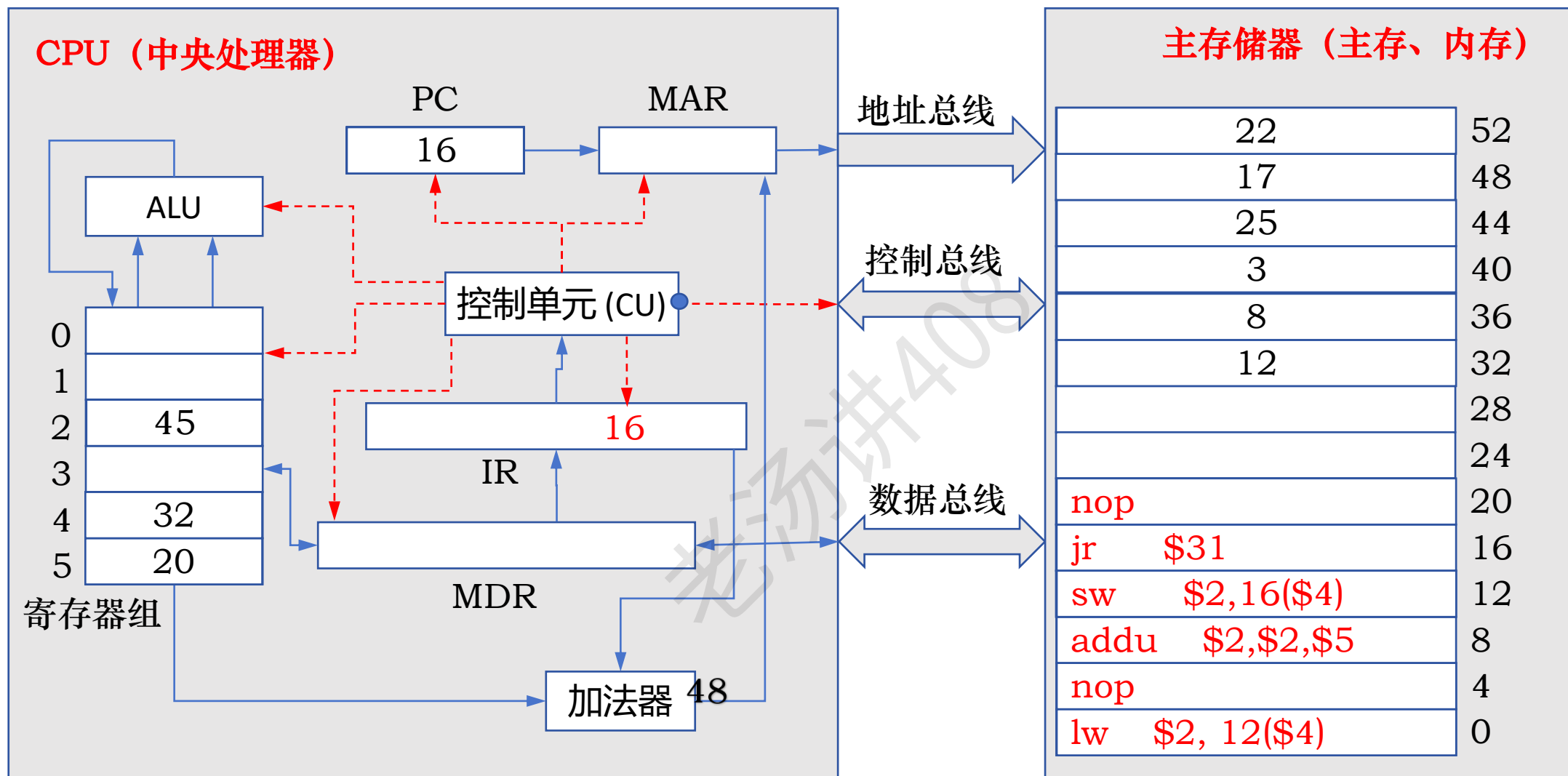
内存地址寄存器 (Memory Address Register, MAR)

内存数据寄存器 (Memory Data Register, MDR)

指令寄存器 (Instruction Register, IR)

```
void add_ele(int A[], int h) {  
    A[4] = h + A[3];  
}
```

指令整体执行流程



内存地址寄存器 (Memory Address Register, MAR)

内存数据寄存器 (Memory Data Register, MDR)

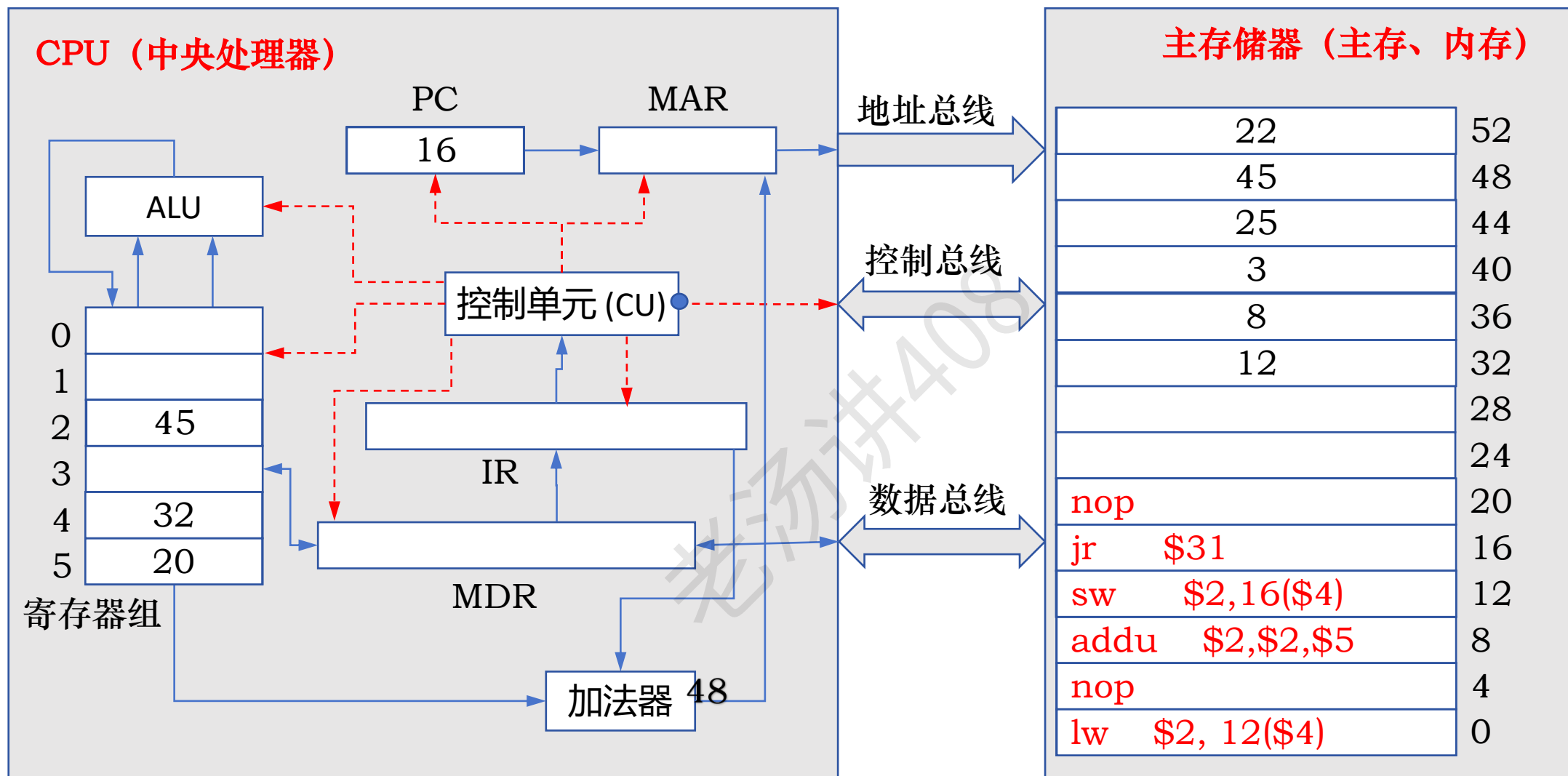
指令寄存器 (Instruction Register, IR)

```
void add_ele(int A[], int h) {  
    A[4] = h + A[3];  
}
```

寄存器传送语言

程序计数器 PC 直接用 PC 表示其内容

用 $M[PC]$ 表示 PC 所指内存中的指令



用 $R[r]$ 表示寄存器堆中 r 的内容 $R[2]$

用 $M[addr]$ 表示读取存储单元 $addr$ 的内容

$M[R[4] + 12]$

```
void add_ele(int A[], int h) {  
    A[4] = h + A[3];  
}
```

寄存器传送语言

程序计数器 PC 直接用 PC 表示其内容

用 $M[PC]$ 表示 PC 所指内存中的指令

```
void add_ele(int A[], int h) {  
    A[4] = h + A[3];  
}
```



add_ele:

lw \$2, 12(\$4)

$R[\$2] \leftarrow M[R[\$4] + 12]$

nop

addu \$2, \$2, \$5

$R[\$2] \leftarrow R[\$2] + R[\$5]$

sw \$2, 16(\$4)

$M[R[\$4] + 16] \leftarrow R[\$2]$

jr

\$31

nop

用 $R[r]$ 表示寄存器堆中 r 的内容 $R[\$2]$

用 $M[addr]$ 表示读取存储单元 $addr$ 的内容

$M[R[\$4] + 16]$

MIPS 指令系统：指令格式

汇编指令：add \$2, \$4, \$6  机器指令 (指令的二进制编码)

问题：一条指令用多少二进制位表示？ 指令字长多大？

原则：简单源于规整

第一种：所有的指令的指令字长都相等（定长指令） RISC（精简指令集）

第二种：不同类型的指令的指令字长不同（变长指令） CISC（复杂指令集）

MIPS 指令系统：指令格式

遵循简单源于规整的原则，所有 MIPS 指令都是 32 位二进制

MIPS 指令字长 = 32 位

汇编指令：add \$2, \$4, \$6 ➡ 机器指令 (指令的二进制编码)



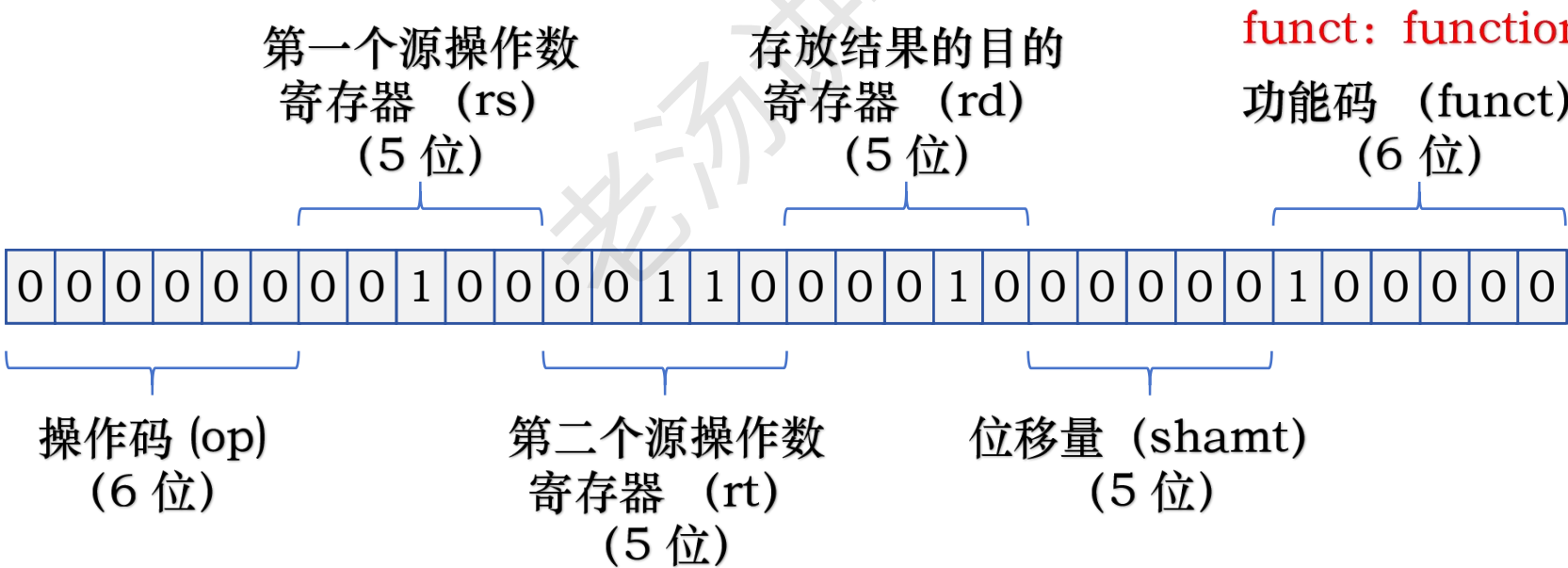
MIPS 指令系统：指令格式

遵循简单源于规整的原则，所有 MIPS 指令都是 32 位二进制

MIPS 指令字长 = 32 位

汇编指令：add \$2, \$4, \$6 ➡ 机器指令 (指令的二进制编码)

rs: register source 的缩写 rd: register destination 的缩写



op: operation code 的缩写 rt: register target 的缩写 shamt: shift amount 的缩写

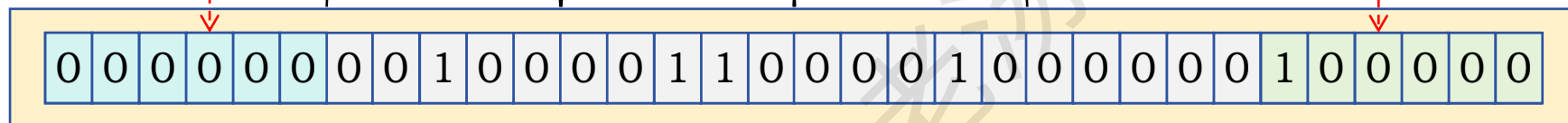
CPU

控制单元

译码器

ALU

通用寄存器组



指令寄存器 (IR)

0
1
2
3
4
5
6
7
8
9
10
11
12
13
...
28
29
30
31

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
...	
28	
29	
30	
31	

MIPS 指令系统：指令格式

汇编指令：add \$2, \$4, \$6



(指令的二进制编码)

0000 0000 1000 0110 0001 0000 0010 0000

指令的机器码



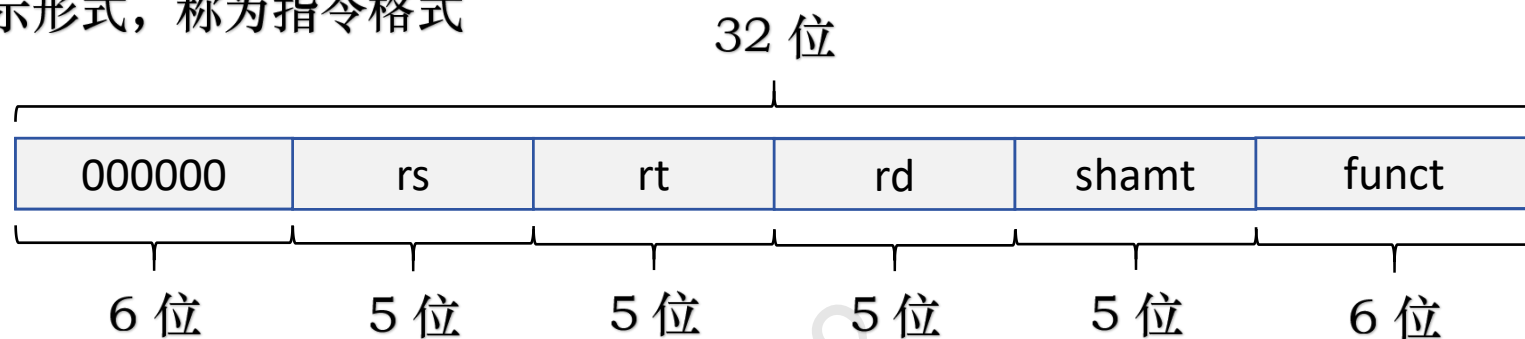
十六进制

0x00861020

MIPS 指令系统：指令格式

二进制数字段组成的指令表示形式，称为指令格式

R-型指令格式



R 是 Register（寄存器）的缩写

op: 指令的基本操作，通常称为操作码 (opcode)

rs (**r**egister **s**ource): 第一个源操作数寄存器编号

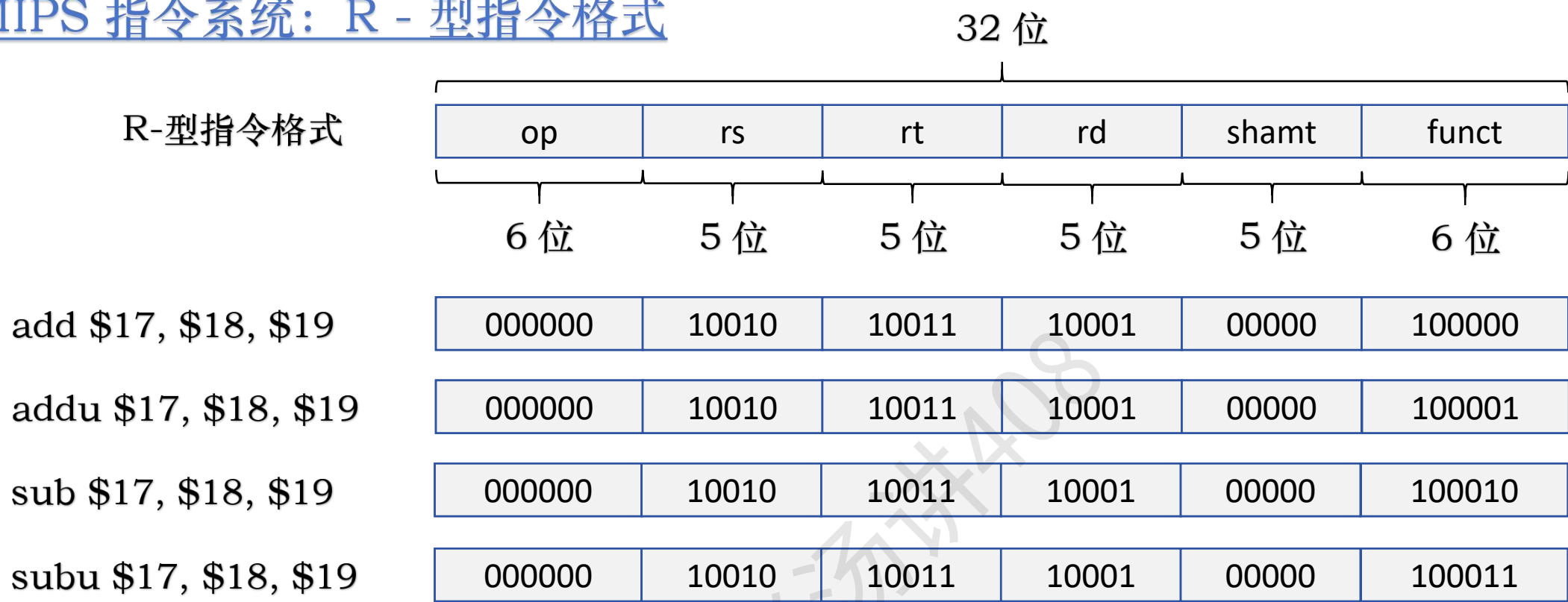
rt (**r**egister **t**arget): 第二个源操作数寄存器编号

rd (**r**egister **d**estination): 用于存放操作结果的目的寄存器编号

shamt (**s**hift **a**mount): 位移量

funct: 功能，一般称为功能码 (**f**unction code)

MIPS 指令系统：R - 型指令格式



操作码 op 都是 0，而具体的操作类型，则由功能码决定

MIPS 指令系统：R - 型指令格式（逻辑运算指令）

逻辑或

```
int logic_cal(int a, int b) {  
    int c = a | b;  
    return c;  
}
```

or \$2,\$4,\$5

$R[\$2] \leftarrow R[\$4] \mid R[\$5]$

逻辑与

```
int logic_cal(int a, int b) {  
    int c = a & b;  
    return c;  
}
```

and \$2,\$4,\$5

$R[\$2] \leftarrow R[\$4] \& R[\$5]$

逻辑异或

```
int logic_cal(int a, int b) {  
    int c = a ^ b;  
    return c;  
}
```

xor \$2,\$4,\$5

$R[\$2] \leftarrow R[\$4] \wedge R[\$5]$

取反操作

```
int logic_cal(int a) {  
    int c = ~a;  
    return c;  
}
```

nor \$2,\$0,\$4

逻辑或非

$R[\$2] \leftarrow \sim (R[\$0] \mid R[\$4])$

恒为 0

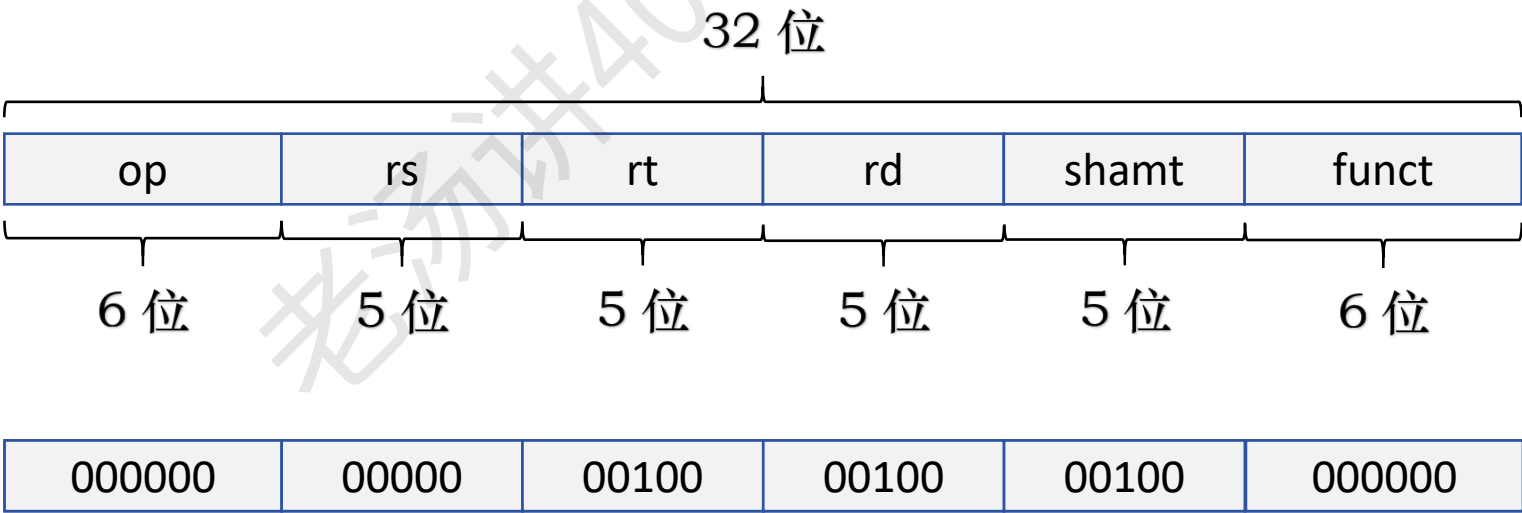
MIPS 指令系统：R - 型指令格式

R-型指令格式	32 位					
	op	rs	rt	rd	shamt	funct
	6 位	5 位	5 位	5 位	5 位	6 位
add \$17, \$18, \$19	000000	10010	10011	10001	00000	100000
addu \$17, \$18, \$19	000000	10010	10011	10001	00000	100001
sub \$17, \$18, \$19	000000	10010	10011	10001	00000	100010
subu \$17, \$18, \$19	000000	10010	10011	10001	00000	100011
and \$2, \$4, \$5	000000	00100	00101	00010	00000	100100
or \$2, \$4, \$5	000000	00100	00101	00010	00000	100101
xor \$2, \$4, \$5	000000	00100	00101	00010	00000	100110
nor \$2, \$4, \$5	000000	00100	00101	00010	00000	100111

MIPS 指令系统：R - 型指令格式（逻辑运算指令）

```
int shift_left4_rightn(int x, int n) {  
左移:  x <<= 4;                                sll    $4,$4,4    sll: shift logical left 的缩写  
      x >>= n;  
      unsignedint y = (unsignedint) x;  
      return y >>= 8;  
}
```

R-型指令格式



sll \$4, \$4, 4

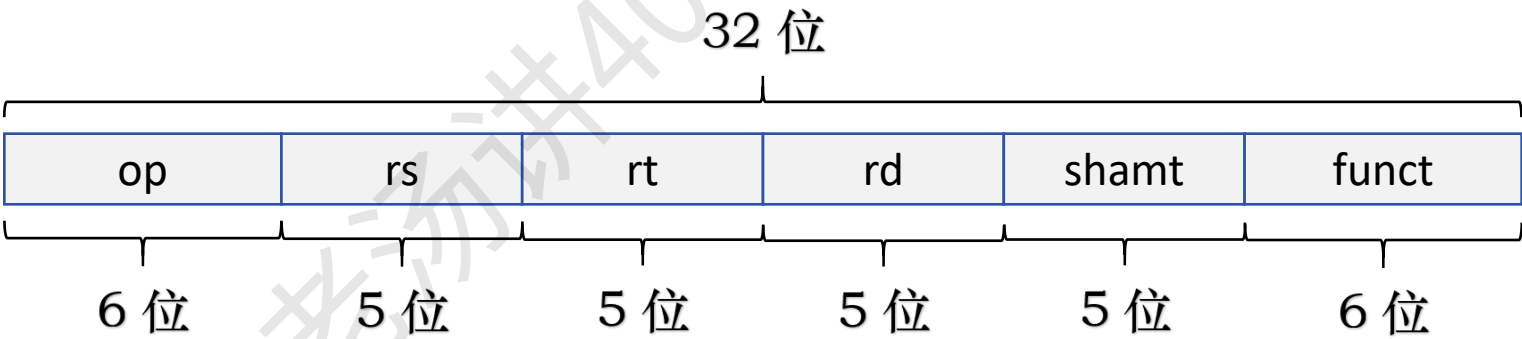
MIPS 指令系统：R - 型指令格式（逻辑运算指令）

```
int shift_left4_rightn(int x, int n) {  
    左移:  x <<= 4;  
    算术右移:  x >>= n;  
    unsignedint y = (unsignedint) x;  
    逻辑右移:  return y >>= 8;  
}
```

```
sll  $4,$4,4  
sra  $4,$4,$5  
srl  $2,$4,8
```

sll: shift logical left 的缩写
sra: shift arithmetic right 的缩写
srl: shift logical right 的缩写

R-型指令格式



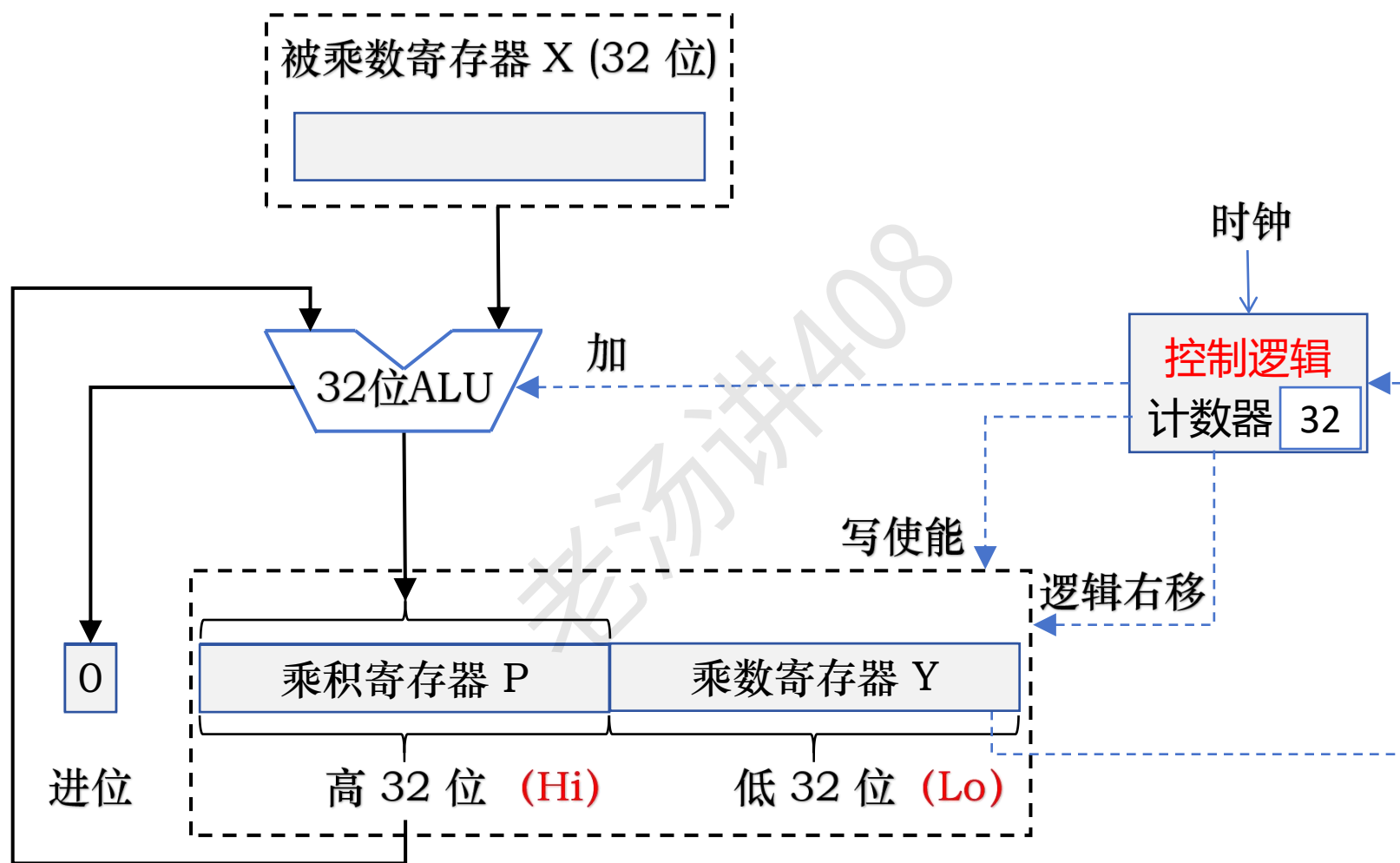
sll \$4, \$4, 4	000000	00000	00100	00100	00100	000000
sra \$4, \$4, \$5	000000	00100	00101	00100	00000	000011
srl \$4, \$4, 8	000000	00000	00100	00100	01000	000010

MIPS 指令系统：R-型指令格式（乘法和除法指令）

两个 32 位的乘商寄存器：Hi 和 Lo

老汤讲408

定点数乘法运算：有符号整数乘法（原码一位乘法）



MIPS 指令系统：R-型指令格式（乘法和除法指令）

两个 32 位的乘商寄存器：Hi 和 Lo 专用寄存器

```
#include <stdio.h>

int mul(int a, int b) {
    int c = a * b;
    return c;
}

int main() {
    int res = mul(9, 3);

    printf("res 的值是: %d\n", res);

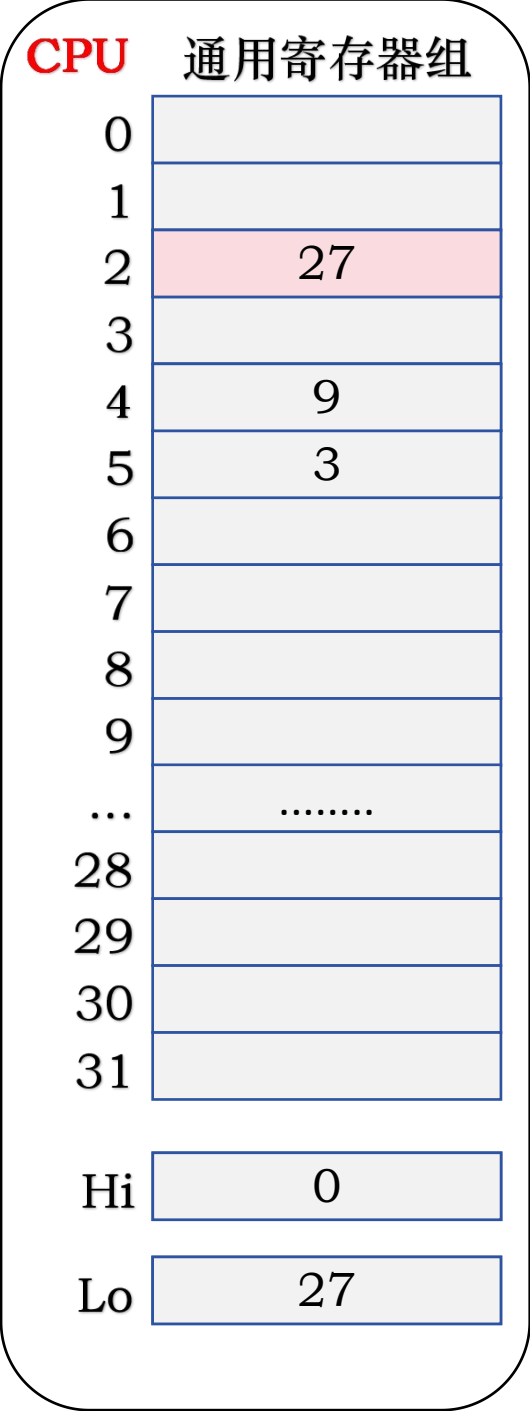
    return 0;
}
```

mult: multiply 的缩写，乘法指令

```
mult    $4,$5
mflo    $2
```

mflo: move from lo 的缩写

```
li      $5,3
li      $4,9
jal     mul
```

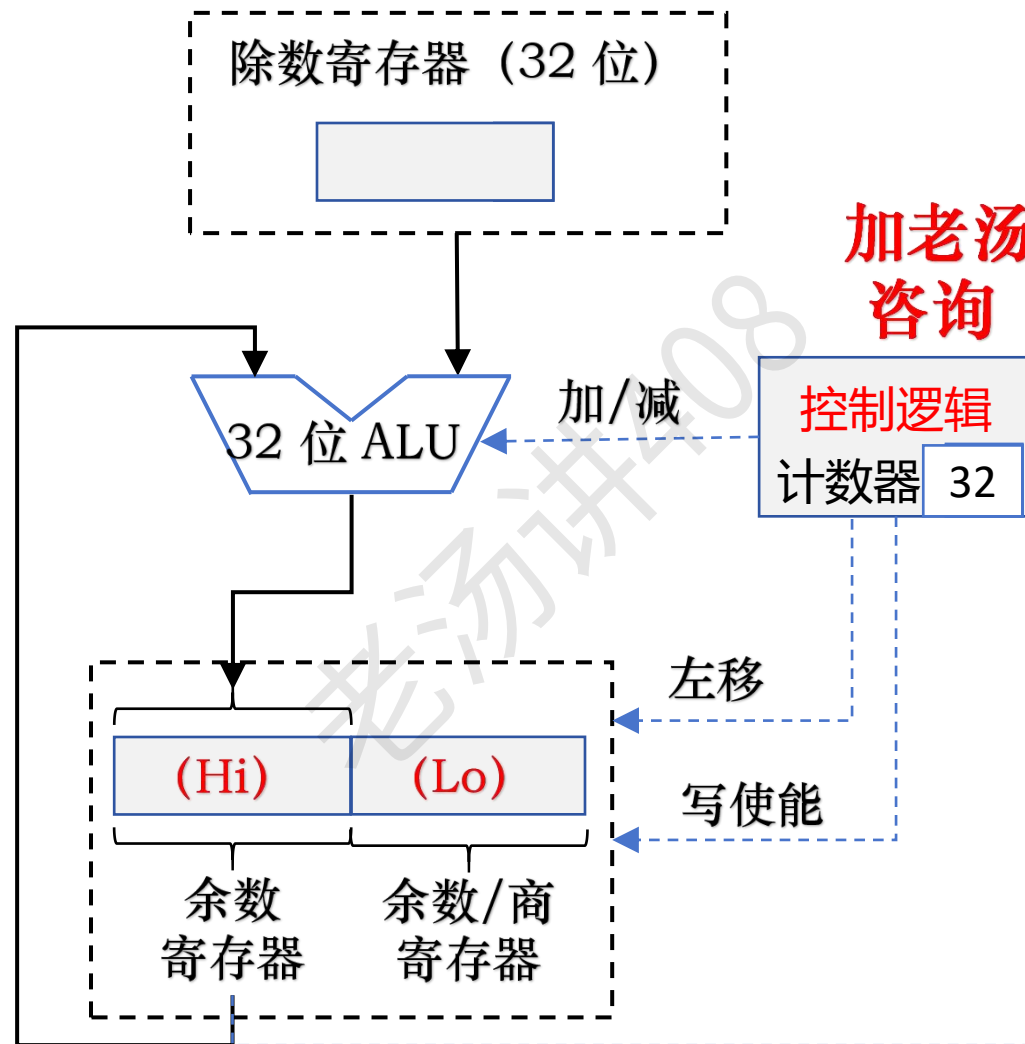


MIPS 指令系统：R - 型指令格式

R-型指令格式	32 位					
	op	rs	rt	rd	shamt	funct
	6 位	5 位	5 位	5 位	5 位	6 位
add \$17, \$18, \$19	000000	10010	10011	10001	00000	100000
addu \$17, \$18, \$19	000000	10010	10011	10001	00000	100001
sub \$17, \$18, \$19	000000	10010	10011	10001	00000	100010
subu \$17, \$18, \$19	000000	10010	10011	10001	00000	100011
and \$2, \$4, \$5	000000	00100	00101	00010	00000	100100
or \$2, \$4, \$5	000000	00100	00101	00010	00000	100101
xor \$2, \$4, \$5	000000	00100	00101	00010	00000	100110
nor \$2, \$4, \$5	000000	00100	00101	00010	00000	100111
mult \$4, \$5	000000	00100	00101	00000	00000	011000
mflo \$2	000000	00000	00000	00010	00000	010010
nop	000000	00000	00000	00000	00000	000000

RISC （精简指令集）缺点：浪费主存空间

定点数除法运算：原码除法运算（电路优化）



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

MIPS 指令系统：R-型指令格式（乘法和除法指令）

两个 32 位的乘商寄存器：Hi 和 Lo 专用寄存器

```
#include <stdio.h>

int div(int a, int b) {
    int c = a / b;
    return c;
}
```

```
int main() {
    int res = div(9, 3);

    printf("res 的值是: %d\n", res);

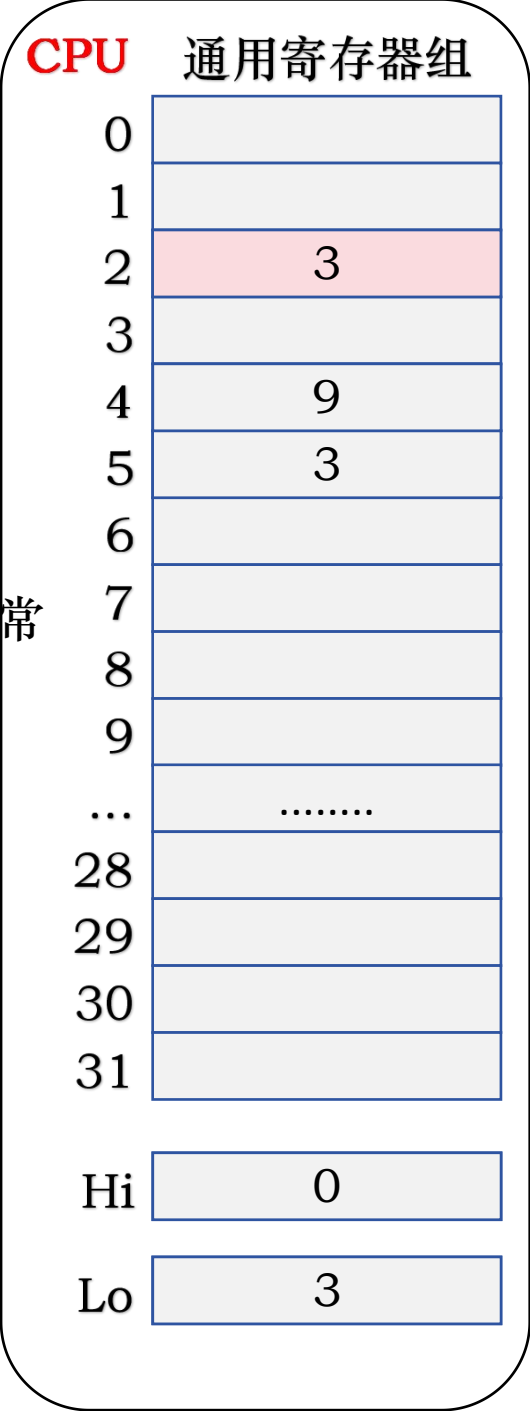
    return 0;
}
```

```
div $4, $5
bne $5, $0, 1f
nop
break 7
1:
mflo $2

li $5, 3
li $4, 9
jal div
```

触发 7 号异常

bne: branch if not equal 的缩写，如果不相等则分支 forward（向前）



MIPS 指令系统：R - 型指令格式

R-型指令格式	32 位					
	op	rs	rt	rd	shamt	funct
	6 位	5 位	5 位	5 位	5 位	6 位
add \$17, \$18, \$19	000000	10010	10011	10001	00000	100000
addu \$17, \$18, \$19	000000	10010	10011	10001	00000	100001
sub \$17, \$18, \$19	000000	10010	10011	10001	00000	100010
subu \$17, \$18, \$19	000000	10010	10011	10001	00000	100011
and \$2, \$4, \$5	000000	00100	00101	00010	00000	100100
or \$2, \$4, \$5	000000	00100	00101	00010	00000	100101
xor \$2, \$4, \$5	000000	00100	00101	00010	00000	100110
nor \$2, \$4, \$5	000000	00100	00101	00010	00000	100111
mult \$4, \$5	000000	00100	00101	00000	00000	011000
mflo \$2	000000	00000	00000	00010	00000	010010
div \$4, \$5	000000	00100	00101	00000	00000	011010

MIPS 指令系统：R-型指令格式

```
#include<stdio.h>
```

```
int cal(int x, int n) {  
    int a = x << 1;  
    int b = a + n;  
    int c = a + b;  
    int d = a * b - c;  
    return d;  
}
```



```
sll    $4,$4,1  
addu   $5,$4,$5  
addu   $3,$4,$5  
mult   $4,$5  
mflo   $2  
subu   $2,$2,$3
```

```
int main() {  
    int res = cal(9, 3);  
    printf("res 的值是: %d\n", res);  
    return 0;  
}
```

op	rs	rt	rd	shamt	funct
000000	00000	00100	00100	00001	000000
000000	00100	00101	00101	00000	100001
000000	00100	00101	00011	00000	100001
000000	00100	00101	00000	00000	011000
000000	00000	00000	00010	00000	010010
000000	00010	00011	00010	00000	100011



0x00042040
0x00852821
0x00851821
0x00850018
0x00001012
0x00431023

MIPS 指令系统：R-型指令格式

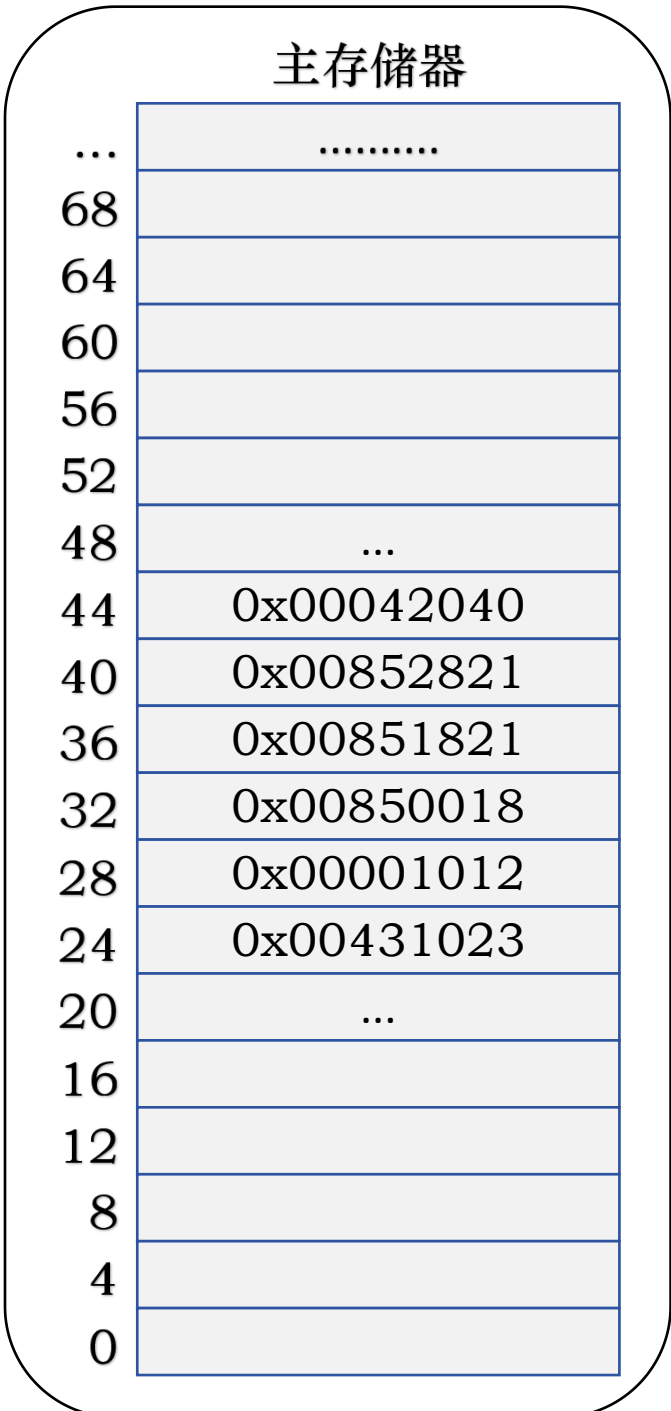
```
#include<stdio.h>

int cal(int x, int n) {
    int a = x << 1;
    int b = a + n;
    int c = a + b;
    int d = a * b - c;
    return d;
}

int main() {
    int res = cal(9, 3);
    printf("res 的值是: %d\n", res);
    return 0;
}
```

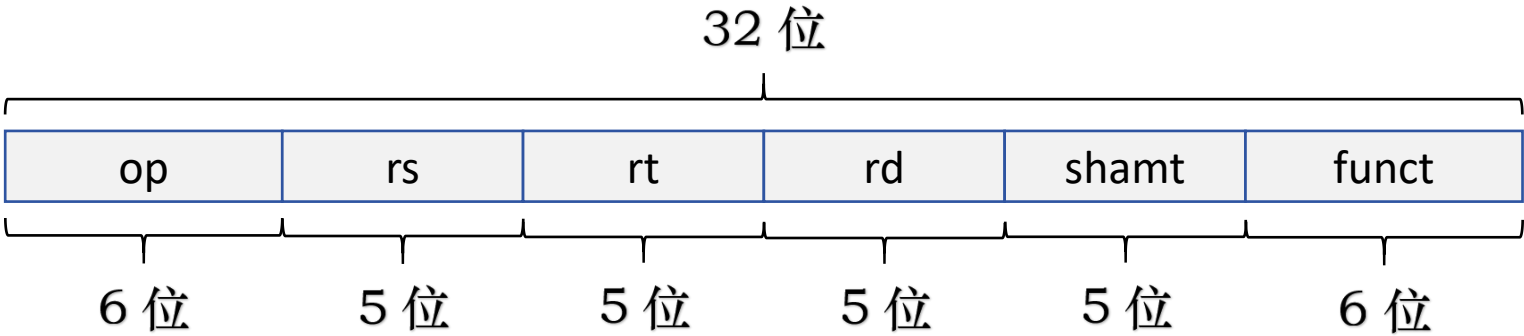
sll	\$4,\$4,1
addu	\$5,\$4,\$5
addu	\$3,\$4,\$5
mult	\$4,\$5
mflo	\$2
subu	\$2,\$2,\$3

0x00042040
0x00852821
0x00851821
0x00850018
0x00001012
0x00431023



MIPS 指令系统：指令格式

R-型指令格式

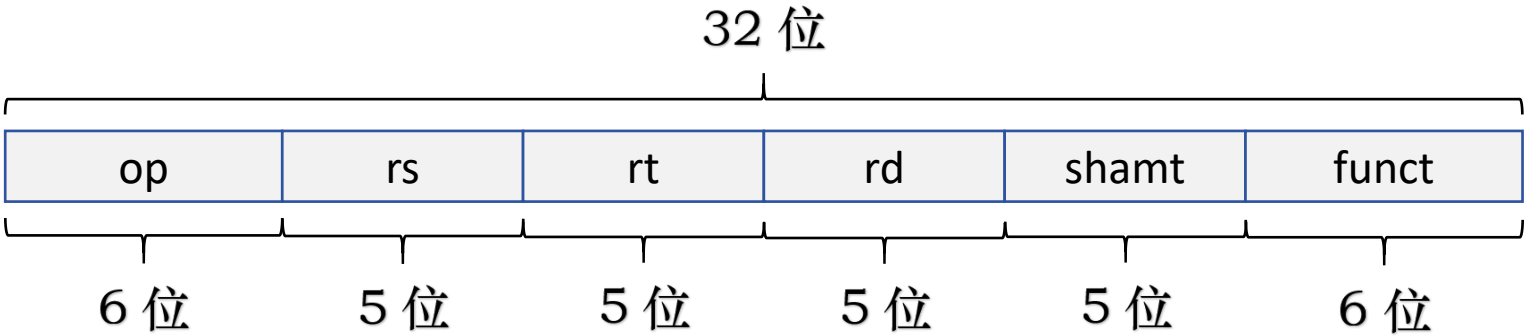


算术运算指令

指令助记符	指令功能	操作描述	功能码 (funct)
add	有符号整数加法	$\$rd = \$rs + \$rt$	100000
addu	无符号整数加法	$\$rd = \$rs + \$rt$	100001
sub	有符号整数减法	$\$rd = \$rs - \$rt$	100010
subu	无符号整数减法	$\$rd = \$rs - \$rt$	100011

MIPS 指令系统：指令格式

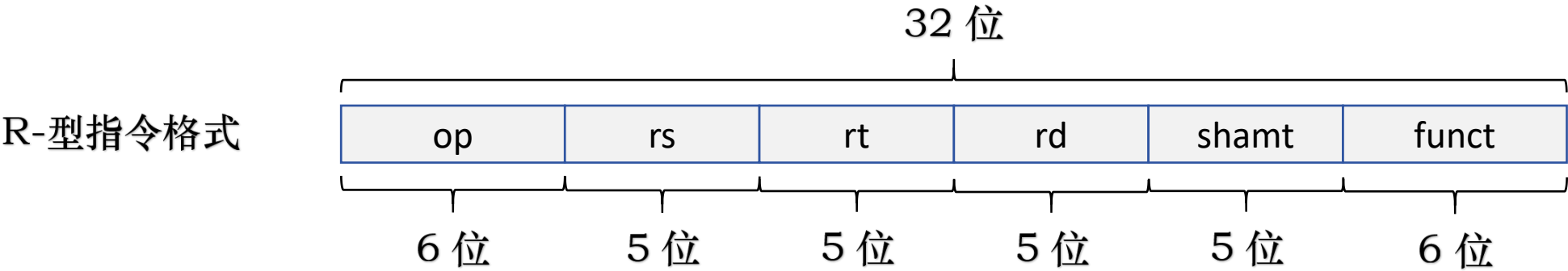
R-型指令格式



逻辑运算指令

指令助记符	指令功能	操作描述	功能码 (funct)
and	按位与	$\$rd = \$rs \& \$rt$	100100
or	按位或	$\$rd = \$rs \mid \$rt$	100101
xor	按位异或	$\$rd = \$rs \wedge \$rt$	100110
nor	按位或非	$\$rd = \sim(\$rs \mid \$rt)$	100111

MIPS 指令系统：指令格式

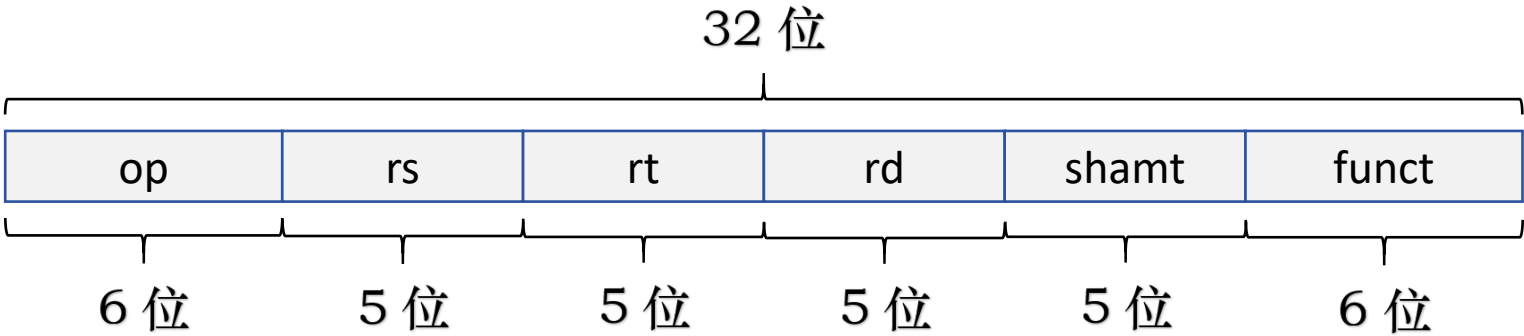


移位运算指令

指令助记符	指令功能	操作描述	功能码 (funct)
sll	逻辑左移	$\$rd = \$rt \ll \text{shamt}$	000000
srl	逻辑右移	$\$rd = \$rt \gg \text{shamt}$	000010
sra	算术右移	$\$rd = \$rt \gg \text{shamt}$ (保留符号位)	000011
sllv	逻辑左移 (由寄存器指定移位置)	$\$rd = \$rt \ll (\$rs)$	000100
srlv	逻辑右移 (由寄存器指定移位置)	$\$rd = \$rt \gg (\$rs)$	000110
srav	算术右移 (由寄存器指定移位置)	$\$rd = \$rt \gg (\$rs)$ (保留符号位)	000111

MIPS 指令系统：指令格式

R-型指令格式



乘法与除法指令

指令助记符	指令功能	操作描述	功能码 (funct)
mult	有符号整数乘法	HI:LO = \$rs * \$rt	011000
multu	无符号整数乘法	HI:LO = \$rs * \$rt	011001
div	有符号整数除法	LO = \$rs / \$rt ; HI = \$rs % \$rt	011010
divu	无符号整数除法	LO = \$rs / \$rt ; HI = \$rs % \$rt	011011
mfhi	从 HI 寄存器传送数据	\$rd = HI	010000
mflo	从 LO 寄存器传送数据	\$rd = LO	010010

MIPS 指令系统：指令格式

```
void add_ele(int A[], int h) {  
    A[300] = h + A[300];  
}
```



```
lw    $2, 1200($4) ←  
nop  
addu  $2, $2, $5  
sw    $2, 1200($4)
```

$300 \times 4 \text{ 字节} = 1200$

数组 A 的基地址
变量 h

CPU	通用寄存器组
0	
1	
2	25
3	
4	0x0008
5	10
6	
7	
8	
9	
10	
11	
12	
13	
...	
28	
29	
30	
31	



	主存储器
...	
0x04BC	10
0x04B8	15
0x04B4	17
0x04B0	25
....	
0x001C	67
0x0018	56
0x0014	1
0x0010	22
0x000C	14
→ 0x0008	4
0x0004	
0x0000	

MIPS 指令系统：指令格式

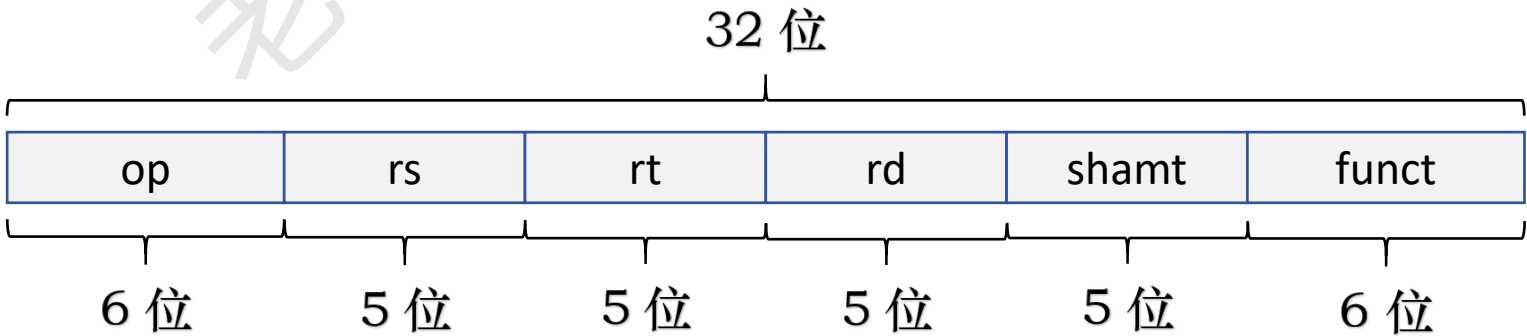
加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

```
void add_ele(int A[], int h) {  
    A[300] = h + A[300];  
}
```

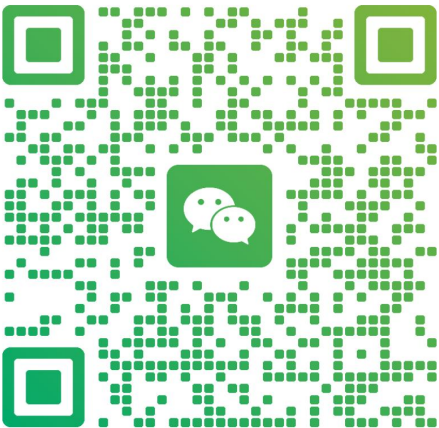


```
lw      $2, 1200($4) ←  
nop  
addu    $2, $2, $5  
sw      $2, 1200($4) ←
```

R-型指令格式



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

MIPS 指令系统：I-型指令格式

Immediate (立即数)

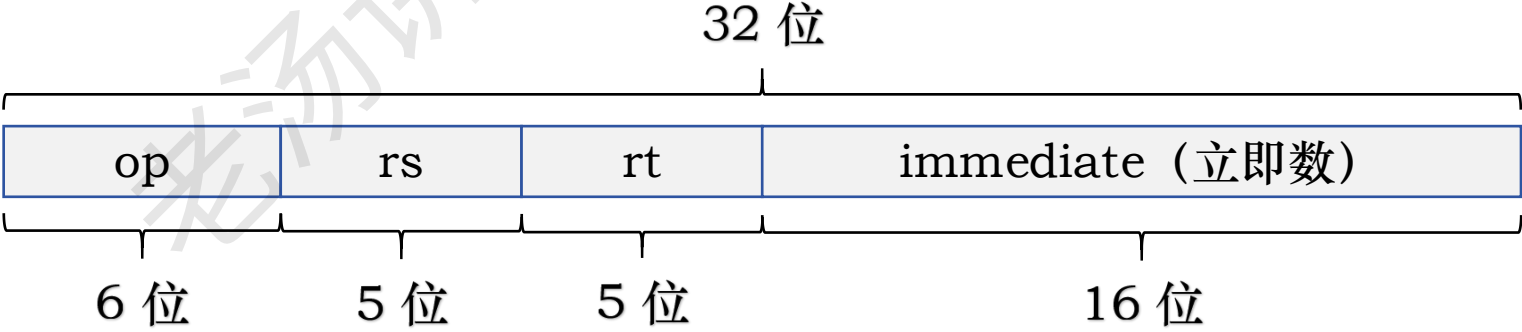
```
void add_ele(int A[], int h) {  
    A[300] = h + A[300];  
}
```



```
lw    $2, 1200($4) ←  
nop  
addu  $2, $2, $5  
sw    $2, 1200($4) ←
```

I-型指令格式

16 位补码的取值范围: $-2^{15} \sim 2^{15} - 1$ (-32768 ~ 32767)



lw \$2, 1200(\$4)

100011	00100	00010	0000 0100 1011 0000
--------	-------	-------	---------------------

sw \$2, 1200(\$4)

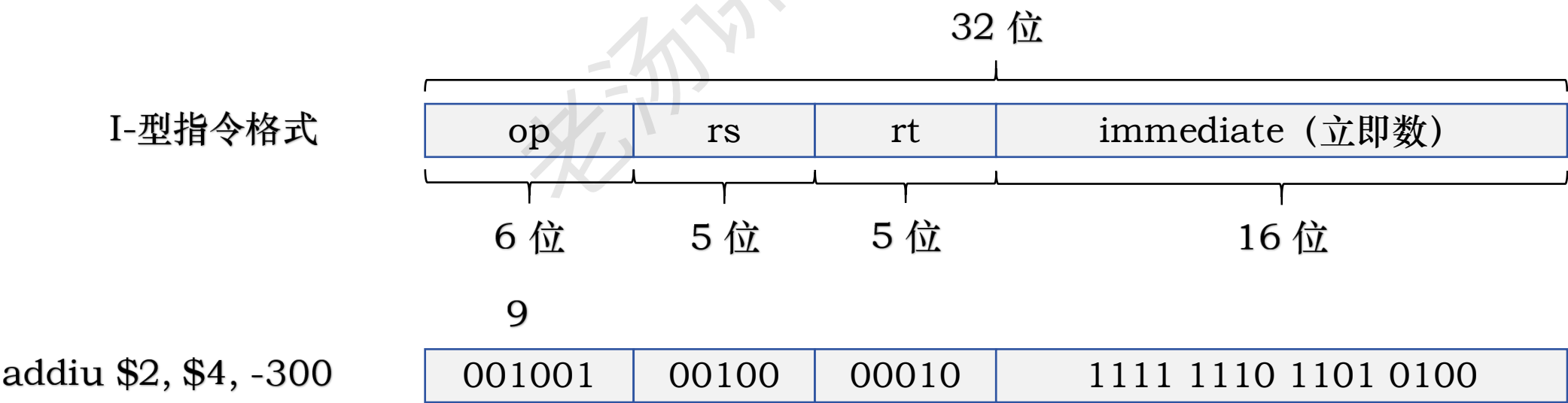
101011	00100	00010	0000 0100 1011 0000
--------	-------	-------	---------------------

MIPS 指令系统：I-型指令格式

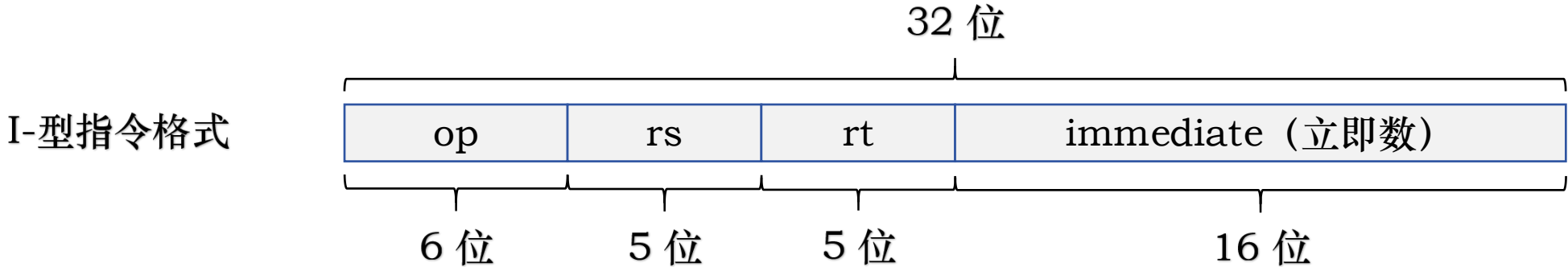
```
int add(int a) {  
    [ a = a - 300; ]  
    return a;  
}
```

➔

```
addiu $2,$4,-300
```



MIPS 指令系统：I-型指令格式



名称	指令功能	汇编格式	操作描述	操作码
addi	有符号立即数加法	addi \$rt, \$rs, imme	$R[\$rt] \leftarrow R[\$rs] + \text{符号扩展(imme)}$	001000
addiu	无符号立即数加法	addiu \$rt, \$rs, imme	$R[\$rt] \leftarrow R[\$rs] + \text{符号扩展(imme)}$	001001
andi	按位与立即数操作	andi \$rt, \$rs, imme	$R[\$rt] \leftarrow R[\$rs] \& \text{零扩展(imme)}$	001100
ori	按位或立即数操作	ori \$rt, \$rs, imme	$R[\$rt] \leftarrow R[\$rs] \text{零扩展(imme)}$	001101
xori	按位异或立即数操作	xori \$rt, \$rs, imme	$R[\$rt] \leftarrow R[\$rs] \wedge \text{零扩展(imme)}$	001110
lw	加载字 (取出一个字)	lw \$rt, offset(\$rs)	$R[\$rt] \leftarrow M[R[\$rs] + \text{符号扩展(imme)}]$	100011
sw	存储字	sw \$rt, offset(\$rs)	$M[R[\$rs] + \text{符号扩展(imme)}] \leftarrow R[\$rt]$	101011

MIPS 指令系统：实现 if 语句（分支指令 beq 和 bne）

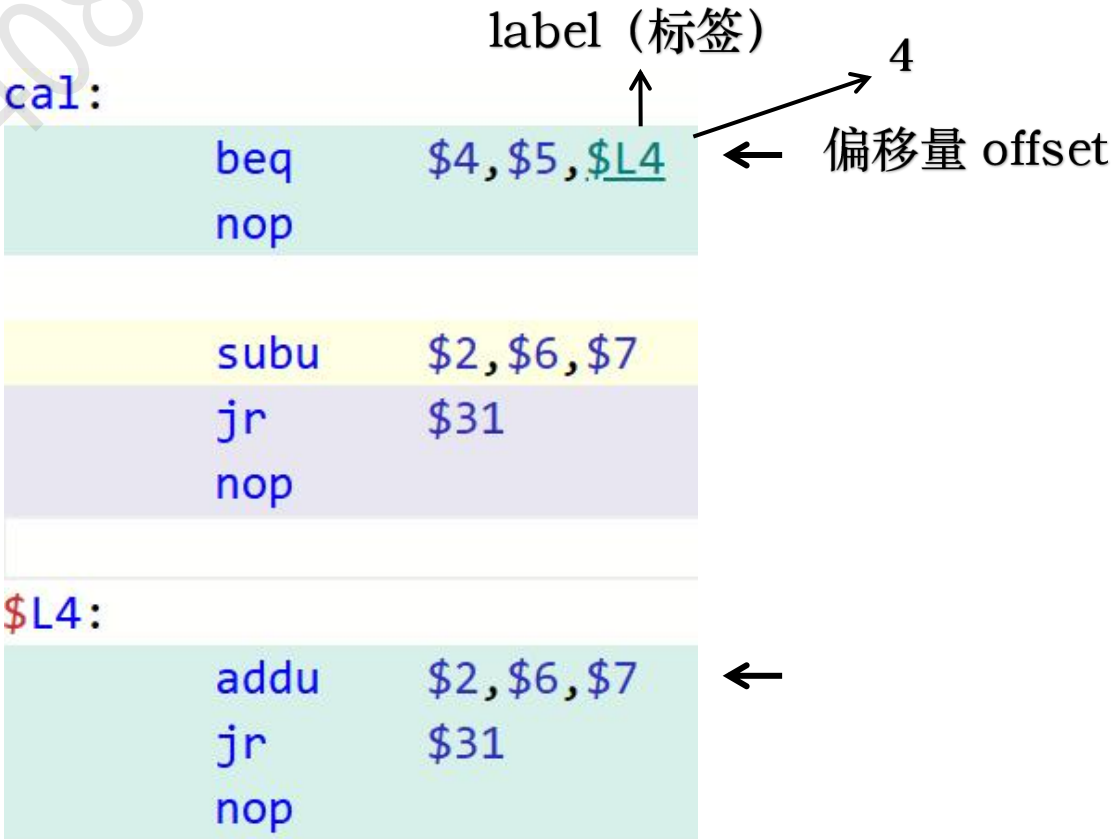
beq \$rs, \$rt, label

beq: branch if equal 的缩写，如果相等则分支（跳转）

bne \$rs, \$rt, label

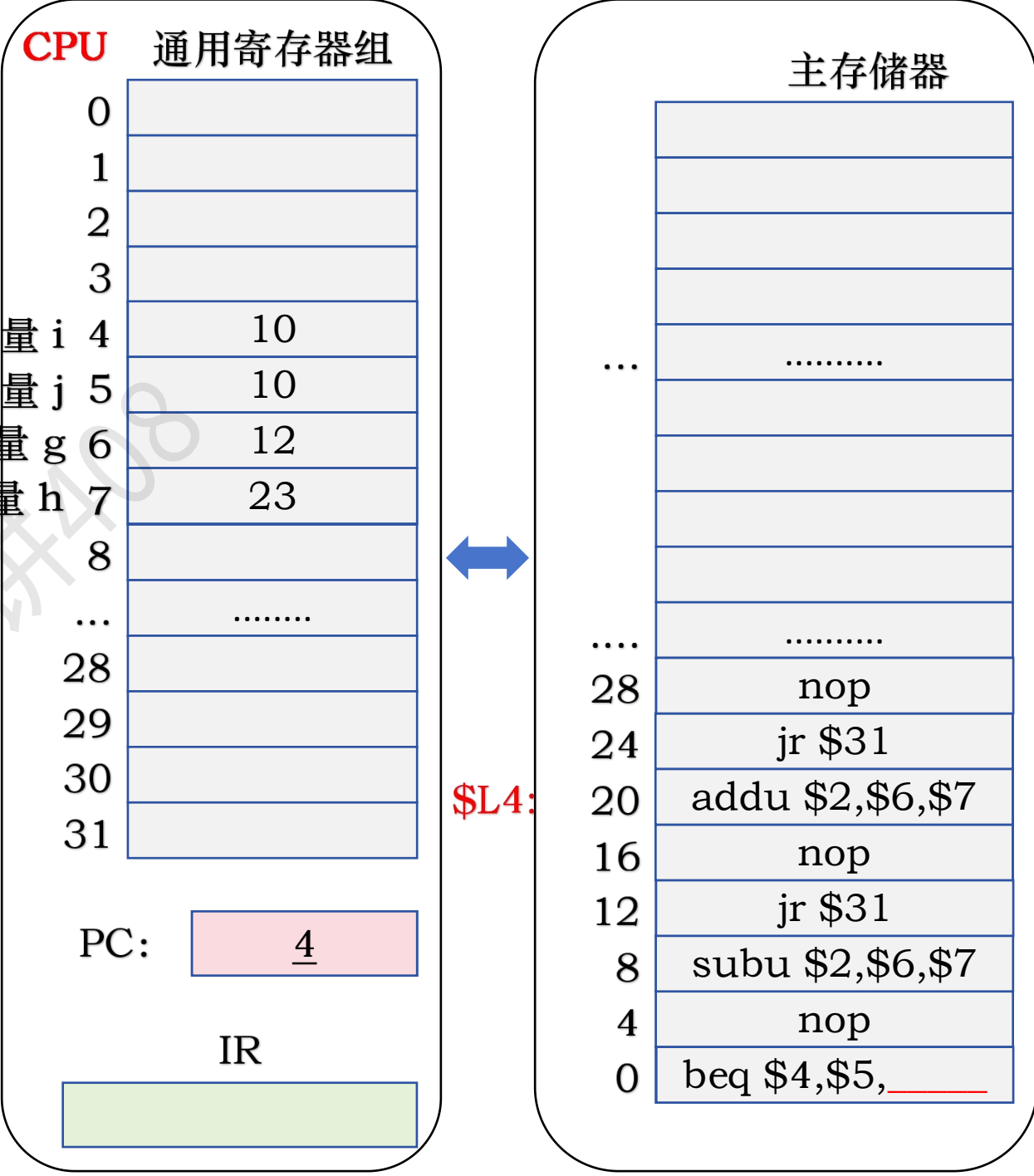
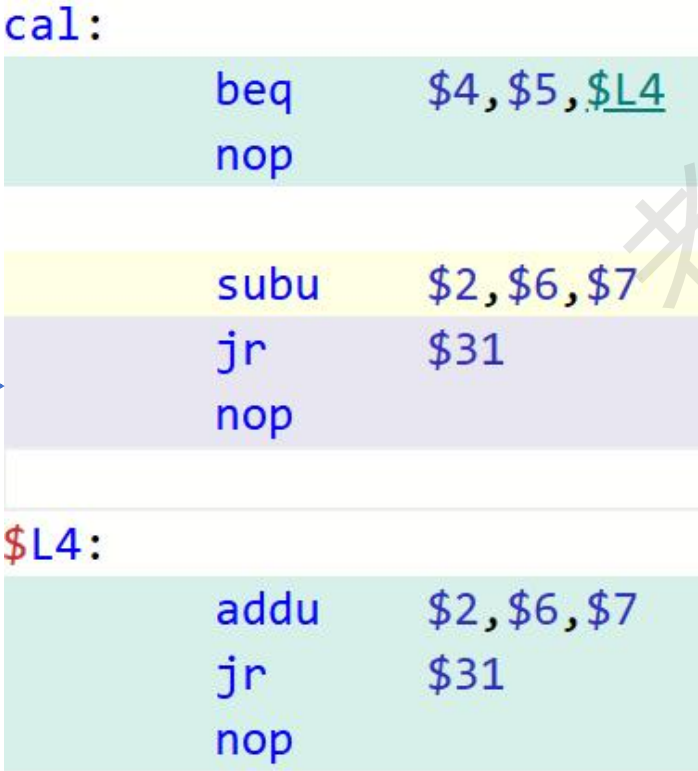
bne: branch if not equal 的缩写，如果不相等则分支（跳转）

```
int cal(int i, int j, int g, int h) {  
    int f;  
    if (i == j) f = g + h;  
    else f = g - h;  
    return f;  
}
```



MIPS 指令系统：分支指令 beq 和 bne

```
int cal(int i, int j, int g, int h) {  
    int f;  
    if (i == j) f = g + h;  
    else f = g - h;  
    return f;  
}
```



MIPS 指令系统：分支指令 beq 和 bne

```
int cal(int i, int j, int g, int h) {  
    int f;  
    if (i == j) f = g + h;  
    else f = g - h;  
    return f;  
}
```

beq \$rs, \$rt, offset

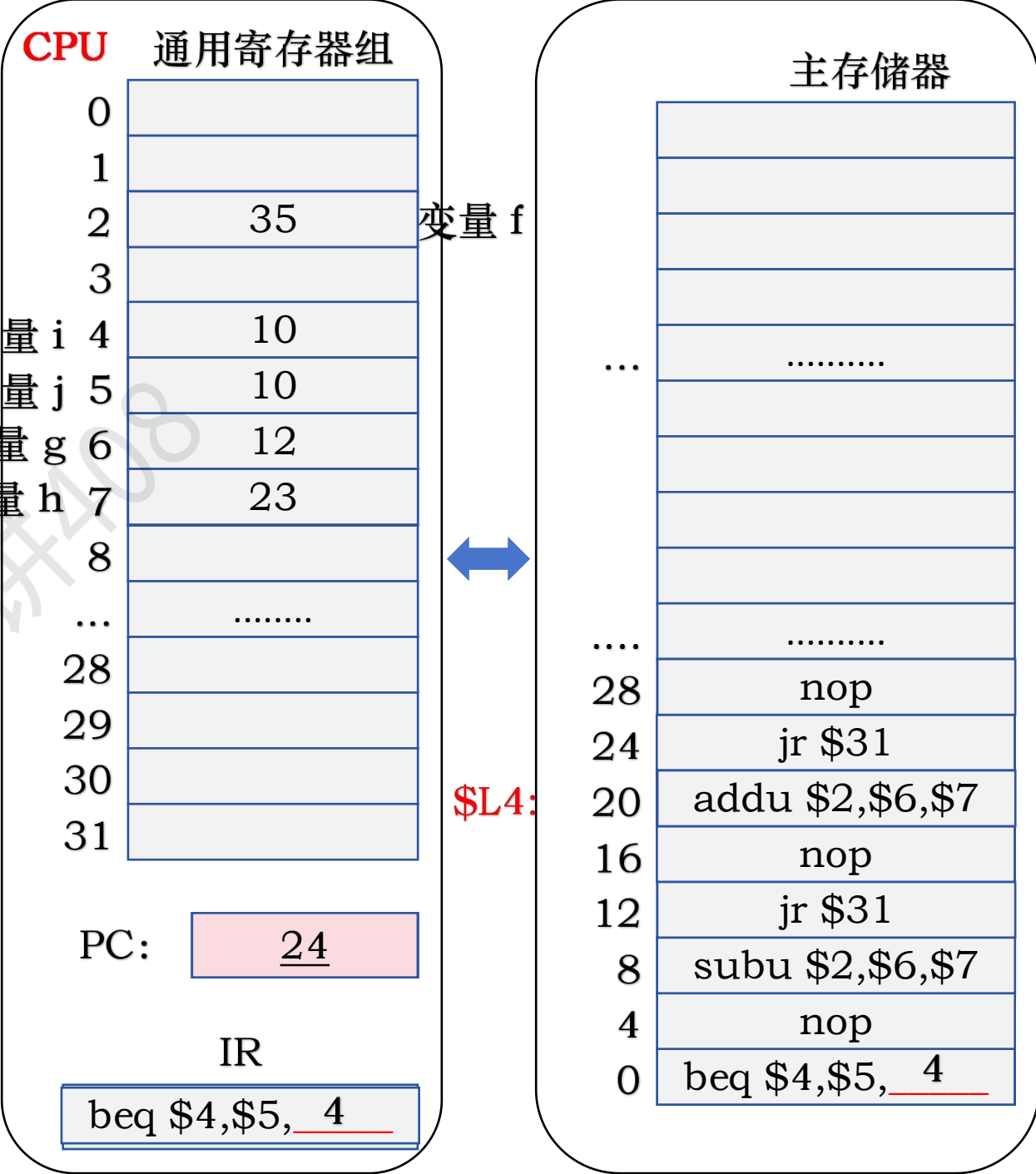
beq 指令所在地址 + 4

若 $R[\$rs] == R[\$rt]$, 则 $PC = PC + (offset \ll 2)$

$20 = 0 + 4 + (offset \ll 2)$

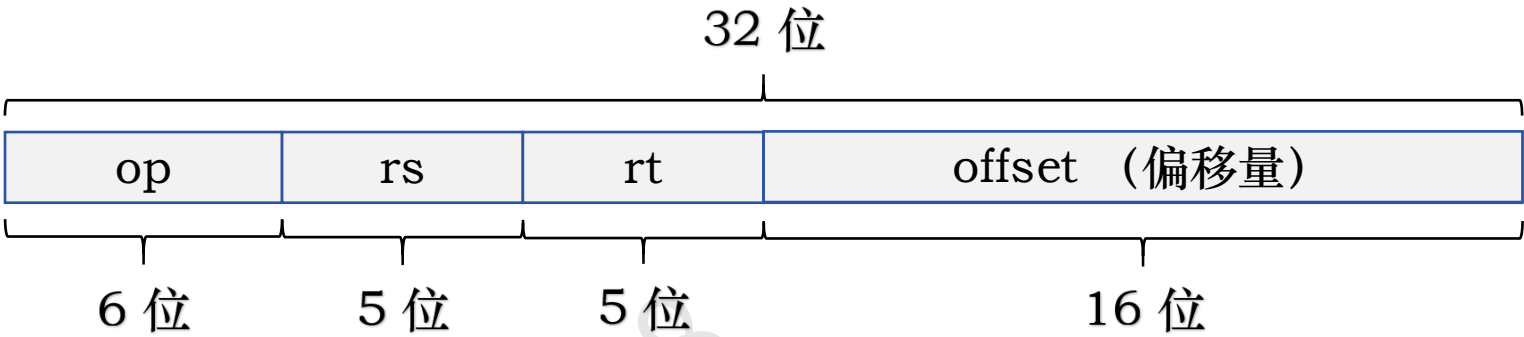
$offset \ll 2 = 16$

$offset = 4$ $PC = PC + 4 + (offset \ll 2)$



MIPS 指令系统：I-型指令格式（分支指令 beq 和 bne）

I-型指令格式



beq \$4, \$5, 4



beq 指令的下一条指令的内存地址

若 $\$rs == \rt , 则 $PC = PC + 4 + (offset \ll 2)$

bne \$4, \$5, 4



bne 指令的下一条指令的内存地址

若 $\$rs != \rt , 则 $PC = PC + 4 + (offset \ll 2)$

MIPS 指令系统：I-型指令格式（分支指令 beq）

\$4\$5\$6\$7

↑↑↑↑

```
int cal(int i, int j, int g, int h) {  
    int f;  
  
    → if (i == j) f = g + h;  
    else f = g - h;  
  
    → if (f >= i) f = f - 10;  
  
    return f;  
}
```

b: branch 的缩写，无条件分支指令

slt: set on less than 的缩写，小于则置位

```
slt $4,$2,$4    # if (R[$2] < R[$4]) R[$4]=1  
                else R[$4]=0
```



cal:

beq \$4,\$5,\$L5
nop

←

subu \$2,\$6,\$7

←

\$L3:

slt \$4,\$2,\$4
bne \$4,\$0,\$L1
nop

←

addiu \$2,\$2,-10

←

\$L1:

jr \$31
nop

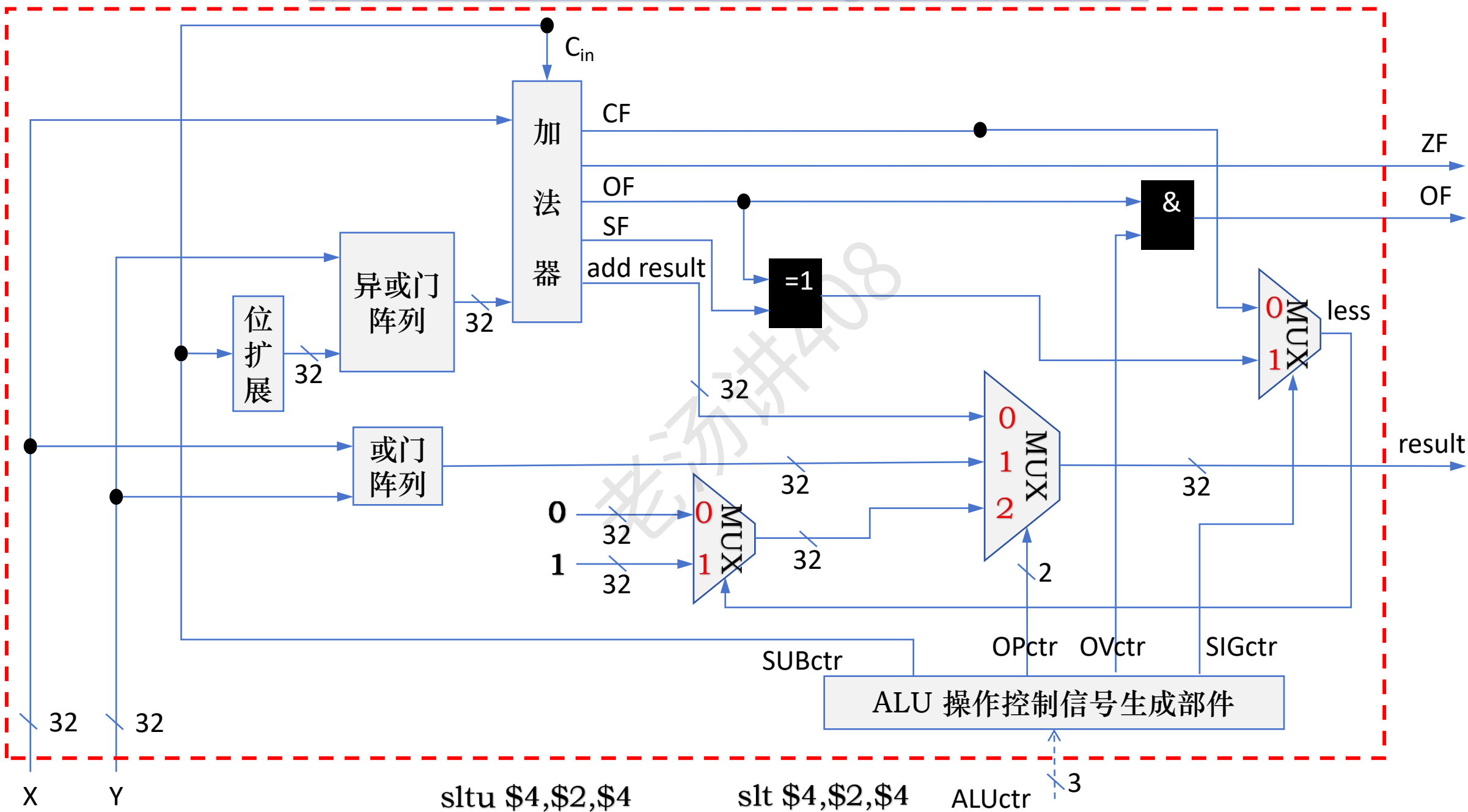
←

\$L5:

addu \$2,\$6,\$7
b \$L3
nop

←

算术逻辑部件 (Arithmetic Logic Unit, ALU)



MIPS 指令系统：分支指令 beq 和 bne

```
int cal(int i,  ircal:
int f;

if (i == j) f
else f = g -

if (f >= i) f $L3:

return f;
}
```

beq

\$4,\$5,\$L5

nop

subu

\$2,\$6,\$7

slt

\$4,\$2,\$4

bne

\$4,\$0,\$L1

nop

addiu

\$2,\$2,-10

jr

\$31

nop

\$L5:

addu

\$2,\$6,\$7

b

\$L3

nop

36 = 0 + 4
+ (offset << 2)

CPU 通用寄存器组

0	
1	
2	
3	
4	10
5	10
6	12
7	23
8	
...	
28	
29	
30	
31	

变量 i

变量 j

变量 g

变量 h

PC: 0

IR



\$L5:

\$L1:

\$L3:

主存储器

...	
44	nop
40	b _____
36	addu \$2,\$6,\$7
32	nop
28	jr \$31
24	addu \$2,\$2,-10
20	nop
16	bne \$4,\$0,_____
12	slt \$4,\$2,\$4
8	subu \$2,\$6,\$7
4	nop
0	beq \$4,\$5,_____

MIPS 指令系统：分支指令 beq 和 bne

```
int cal(int i,  ircal:
```

```
int f;
```

```
if (i == j) f
```

```
else f = g -
```

```
if (f >= i) f $L3:
```

```
return f;
```

```
}
```

12 = 40 + 4
+ (offset << 2)

28 = 16 + 4
+ (offset << 2)

36 = 0 + 4
+ (offset << 2)

```
beq    $4,$5,$L5  
nop
```

```
subu   $2,$6,$7
```

```
slt    $4,$2,$4  
bne    $4,$0,$L1  
nop
```

```
addiu  $2,$2,-10
```

```
jr     $31  
nop
```

```
$L5:  
addu   $2,$6,$7  
b      $L3  
nop
```

CPU 通用寄存器组

0	
1	
2	35
3	
4	0
5	10
6	12
7	23
8	
...	
28	
29	
30	
31	

PC: 20

IR

变量 f

变量 i

变量 j

变量 g

变量 h



\$L5:

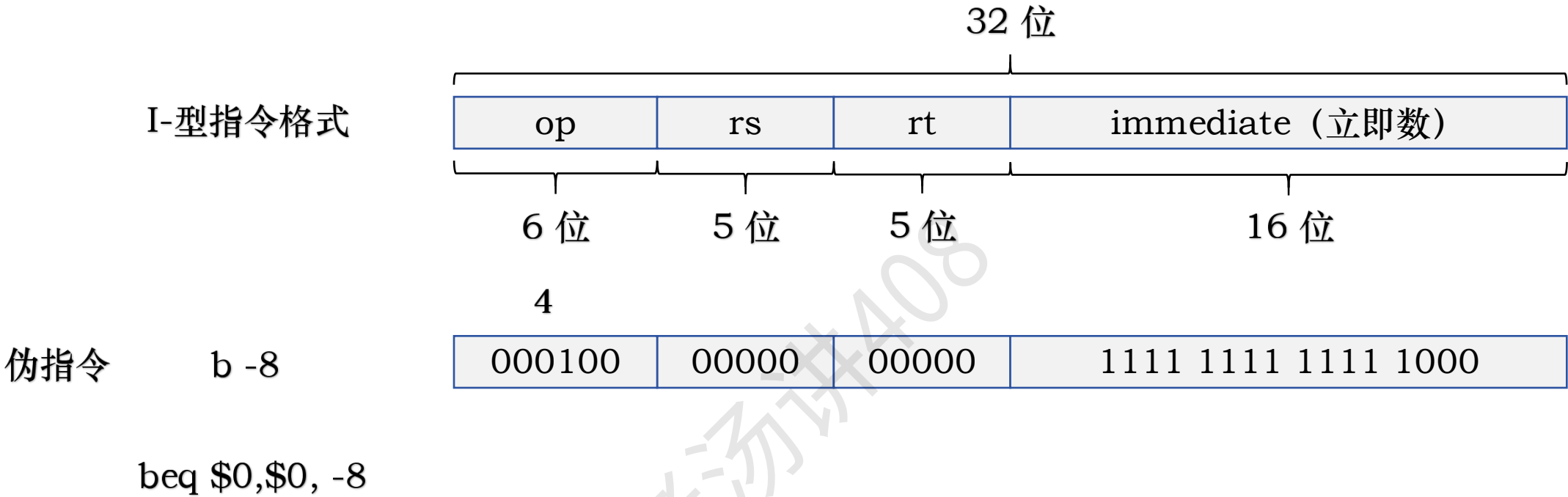
\$L1:

\$L3:

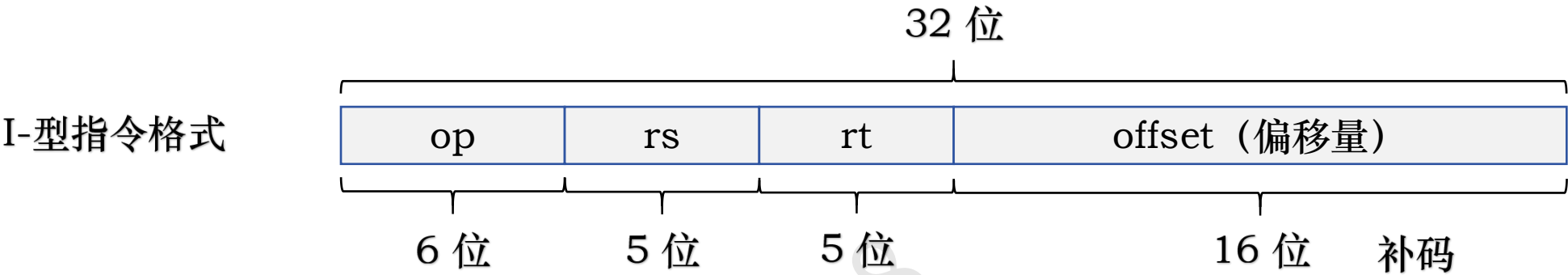
主存储器

...	
44	nop
40	b -8
36	addu \$2,\$6,\$7
32	nop
28	jr \$31
24	addu \$2,\$2,-10
20	nop
16	bne \$4,\$0, 2
12	slt \$4,\$2,\$4
8	subu \$2,\$6,\$7
4	nop
0	beq \$4,\$5, 8

MIPS 指令系统：I-型指令格式（无条件分支指令 b）



MIPS 指令系统：I-型指令格式（分支指令 beq 和 bne）



有条件跳转：

beq \$4, \$5, 4 000100 00100 00101 0000 0000 0000 0100

bne \$4, \$5, 4 000101 00100 00101 0000 0000 0000 0100

无条件跳转：

beq \$0, \$0, 4 000100 0 0 0000 0000 0000 0100

16 位补码的取值范围： $-2^{15} \sim 2^{15} - 1$ (-32768 ~ 32767)

寻址范围： $PC + 4 + (-32768 \ll 2) \sim PC + 4 + (32767 \ll 2)$

相对寻址： 相对当前的 PC 值计算下一条执行指令的内存地址

MIPS 指令系统：循环结构 (do...while)

```

                                $4
                                ↑
long fact_do(long n) {
→ long result = 1;
  do {
    result *= n;
    n = n - 1;
  } while (n > 1);
  return result;
}
```



```

fact_do:
→ li      $2,1
$L2:
    mult   $2,$4
    mflo   $2
    addiu  $4,$4,-1
    slt    $3,$4,2
    beq    $3,$0,$L2
    nop

    jr     $31
```

将 result 的值初始化为 1，放在 \$2

将 result 和 n 相乘，结果放在 Lo
将乘积赋值给 result

将 n 减去 1，结果赋值给 n

如果 n 小于 2，则将 \$3 设置为 1

如果 \$3 的值等于 0，则跳转



```

loop:
  body-statement
  t = test-expr;
  if (t)
    goto loop;
```

MIPS 指令系统：循环结构 (while)

优化级别：-Og

```
long fact_while(long n) {  
    → long result = 1;  
    while (n > 1) {  
        result *= n;  
        n = n - 1;  
    }  
    → return result;  
}
```

\$4
↑

```
goto test;  
loop:  
    body-statement  
test:  
    t = test-expr;  
    if (t)  
        goto loop;
```



```
fact_while:  
    → li      $2,1  
      b       $L2  
      nop  
  
$L3:  
    → mult    $2,$4  
      mflo    $2  
      addiu   $4,$4,-1  
  
$L2:  
    → slt     $3,$4,2  
      beq     $3,$0,$L3  
      nop  
  
      jr      $31
```

将 result 的值初始化为 1，放在 \$2
无条件跳转到 \$L2

将 result 和 n 相乘，结果放在 Lo
将乘积赋值给 result

将 n 减去 1，结果赋值给 n

如果 n 小于 2 的话，则设置 \$3 为 1
如果 \$3 等于 0，即 n >= 2，则跳转

MIPS 指令系统：循环结构 (while) 优化级别：-O1

```
long fact_while(long n) {
    long result = 1;
    → while (n > 1) {
        result *= n;
        n = n - 1;
    }
    return result;
}
```

```
t = test-expr;
if (!t)
    goto done;
loop:
    body-statement
    t = test-expr;
    if (t)
        goto loop;
done:
```

\$4
↑



fact_while:	→ slt	\$2,\$4,2	如果 n < 2 的话，则设置 \$2 为 1
	bne	\$2,\$0,\$L4	如果 \$2 不等于 0，即 n < 2，则跳转
	li	\$3,1	将常量 1 存放 \$3，后面比较会用到
	li	\$2,1	将 result 的值初始化为 1，放在 \$2
	mult	\$2,\$4	将 result 和 n 相乘，结果放在 Lo
\$L6:	mflo	\$2	将乘积赋值给 result
	addiu	\$4,\$4,-1	将 n 减去 1，结果赋值给 n
	mult	\$2,\$4	将 result 和 n 相乘，结果放在 Lo
	bne	\$4,\$3,\$L6	如果 n != 1，则跳转
	jr	\$31	
	nop		
\$L4:	jr	\$31	
	li	\$2,1	

MIPS 指令系统：循环结构 (while)

```
long fact_while(long n) {  
    long result = 1;  
    while (n > 1) {  
        result *= n;  
        n = n - 1;  
    }  
    return result;  
}
```

优化级别: -Og

```
        goto test;  
loop:  
    body-statement  
test:  
    t = test-expr;  
    if (t)  
        goto loop;
```

优化级别: -O1

```
    t = test-expr;  
    if (!t)  
        goto done;  
loop:  
    body-statement  
    t = test-expr;  
    if (t)  
        goto loop;  
done:
```

MIPS 指令系统：循环结构 (while)

```
long fact_while(long n) {  
    long result = 1;  
    while (n > 1) {  
        result *= n;  
        n = n - 1;  
    }  
    return result;  
}
```

\$4
↑

优化级别: -Og

fact_while:		
	li	\$2,1
	b	<u>\$L2</u>
	nop	
\$L3:		
	mult	\$2,\$4
	mflo	\$2
	addiu	\$4,\$4,-1
\$L2:		
n ≥ 2	slt	\$3,\$4,2
	beq	\$3,\$0, <u>\$L3</u>
	nop	
	jr	\$31

优化级别: -O1

fact_while:		
n < 2	slt	\$2,\$4,2
	bne	\$2,\$0,\$L4
	li	\$3,1
	li	\$2,1
	mult	\$2,\$4
\$L6:		
	mflo	\$2
	addiu	\$4,\$4,-1
	mult	\$2,\$4
n != 1	bne	\$4,\$3,\$L6
	jr	\$31
	nop	
\$L4:		
	jr	\$31
	li	\$2,1

MIPS 指令系统：循环结构 (for)

```
long fact_for(long n) {  
    long result = 1;  
    long i;  
    for (i = 2; i <= n; i++)  
        result *= i;  
    return result;  
}
```



```
long fact_while(long n) {  
    long result = 1;  
    while (n > 1) {  
        result *= n;  
        n = n - 1;  
    }  
    return result;  
}
```

```
for (init-expr; test-expr; update-expr)  
    body-statement
```



```
init-expr;  
while (test-expr) {  
    body-statement  
    update-expr;  
}
```

MIPS 指令系统：循环结构 (for)

优化级别：-Og

```
long fact_for(long n) {  
    long result = 1;  
    long i;  
    for (i = 2; i <= n; i++)  
        result *= i;  
    return result;  
}
```



```
long fact_while(long n) {  
    long result = 1;  
    while (n > 1) {  
        result *= n;  
        n = n - 1;  
    }  
    return result;  
}
```

fact_for:

li \$3, 2

li \$2, 1

b \$L2

nop

\$L3:

mult \$2, \$3

mflo \$2

addiu \$3, \$3, 1

\$L2:

slt \$5, \$4, \$3

beq \$5, \$0, \$L3

nop

jr \$31

MIPS 指令系统：循环结构程序 (for)

```
int arr[] = {5, 2, 9, 1, 6}
```

```
int sum_arr(int arr[], int n) {  
    int res = 0;  
    for (int i = 0; i < n; i++) {  
        res += arr[i];  
    }  
    return res;  
}
```

通用寄存器组:

0	0
1	
2	5
3	1
4	0x123456
5	5
6	5
...	

变量 res
变量 i
arr 基地址
变量 n



```
sum_arr:  
    → move    $3,$0  
    move    $2,$0  
    b       $L2  
    nop
```

add \$3, \$0, \$0
add \$2, \$0, \$0

```
$L3:  
    → sll     $6,$3,2  
    addu    $6,$4,$6  
    lw      $6,0($6)  
    nop  
    addu    $2,$2,$6  
    addiu   $3,$3,1
```

基地址 + i × 4
计算 arr[i]
的内存地址
加载 arr[i]
res += arr[i]
i++

```
$L2:  
    → slt     $6,$3,$5  
    bne     $6,$0,$L3  
    nop
```

比较 i < n ?

```
jr      $31  
nop
```


MIPS 指令系统：循环结构程序 (for)

int arr[] = {5, 2, 9, 1, 6}

```
int sum_arr(int arr[], int n) {
    int res = 0;
    for (int i = 0; i < n; i++) {
        res += arr[i];
    }
    return res;
}
```

通用寄存器组:

0	0
1	
2	7
3	2
4	0x123456
5	5
6	2
...	

变量 res
变量 i
arr 基地址
变量 n



sum_arr:

move	\$3,\$0	add \$3, \$0, \$0
move	\$2,\$0	add \$2, \$0, \$0
b	<u>\$L2</u>	
nop		

\$L3:

→ sll	\$6,\$3,2	计算 arr[i] 的内存地址 加载 arr[i]
addu	\$6,\$4,\$6	
lw	\$6,0(\$6)	
nop		
addu	\$2,\$2,\$6	res += arr[i]
→ addiu	\$3,\$3,1	i++

\$L2:

slt	\$6,\$3,\$5	比较 i < n ?
bne	\$6,\$0, <u>\$L3</u>	
nop		

jr	\$31
nop	

MIPS 指令系统：循环结构程序 (for)

int arr[] = {5, 2, 9, 1, 6}

```
int sum_arr(int arr[], int n) {
    int res = 0;
    for (int i = 0; i < n; i++) {
        res += arr[i];
    }
    return res;
}
```

通用寄存器组:

0	0
1	
2	16
3	3
4	0x123456
5	5
6	9
...	

变量 res
变量 i
arr 基地址
变量 n



```
sum_arr:
    move    $3,$0      add $3, $0, $0
    move    $2,$0      add $2, $0, $0
    b       $L2
    nop
```

```
$L3:
    → sll    $6,$3,2    } 计算 arr[i]
    addu    $6,$4,$6    } 的内存地址
    lw      $6,0($6)    } 加载 arr[i]
    nop
    addu    $2,$2,$6    res += arr[i]
    → addiu   $3,$3,1    i++
```

```
$L2:
    slt     $6,$3,$5    比较 i < n ?
    bne     $6,$0,$L3
    nop
```

```
jr    $31
nop
```

MIPS 指令系统：循环结构程序 (for)

int arr[] = {5, 2, 9, 1, 6}

```
int sum_arr(int arr[], int n) {
    int res = 0;
    for (int i = 0; i < n; i++) {
        res += arr[i];
    }
    return res;
}
```

通用寄存器组:

0	0
1	
2	17
3	4
4	0x123456
5	5
6	1
...	

变量 res
变量 i
arr 基地址
变量 n



```
sum_arr:
    move    $3,$0      add $3, $0, $0
    move    $2,$0      add $2, $0, $0
    b       $L2
    nop
```

```
$L3:
    → sll    $6,$3,2    } 计算 arr[i]
    addu    $6,$4,$6    } 的内存地址
    lw      $6,0($6)    } 加载 arr[i]
    nop
    addu    $2,$2,$6    res += arr[i]
    → addiu   $3,$3,1    i++
```

```
$L2:
    slt     $6,$3,$5    比较 i < n ?
    bne     $6,$0,$L3
    nop
```

```
jr $31
nop
```

MIPS 指令系统：循环结构程序 (for)

int arr[] = {5, 2, 9, 1, 6}

```
int sum_arr(int arr[], int n) {
    int res = 0;
    for (int i = 0; i < n; i++) {
        res += arr[i];
    }
    return res;
}
```

通用寄存器组:

0	0
1	
2	23
3	5
4	0x123456
5	5
6	0
...	

变量 res
变量 i
arr 基地址
变量 n



```
sum_arr:
    move    $3,$0      add $3, $0, $0
    move    $2,$0      add $2, $0, $0
    b       $L2
    nop
```

```
$L3:
    → sll    $6,$3,2    } 计算 arr[i]
    addu    $6,$4,$6    } 的内存地址
    lw      $6,0($6)    } 加载 arr[i]
    nop
    addu    $2,$2,$6    res += arr[i]
    → addiu   $3,$3,1    i++
```

```
$L2:
    slt     $6,$3,$5    比较 i < n ?
    bne     $6,$0,$L3
    nop
```

```
jr $31
nop
```

MIPS 指令系统：如何实现过程调用？

```
long sumN_while(int n) {  
    long res = 0;  
    while (n > 1) {  
        res += n;  
        n--;  
    }  
    return res;  
}
```

C: 函数

Java: 方法

```
function sumN_while(n) result(res)  
    implicit none  
    integer, intent(in) :: n  
    integer(kind=8) :: res  
    integer :: i  
  
    res = 0  
    i = n  
    do while (i > 1)  
        res = res + i  
        i = i - 1  
    end do  
  
end function sumN_while
```

Fortran: 子例程

过程

MIPS 指令系统：如何实现过程调用？

jr: jump register 的缩写，寄存器跳转指令

跳转到存放在 31 号寄存器中的内存地址

```
#include<stdio.h>
```

```
int add(int a, int b) {  
    int res = a + b;  
    return res;  
}
```

```
int main() {  
    int res = add(10, 20);  
    printf("res = %d\n", res);  
    return 0;  
}
```

通用寄存器组:

0	0
1	
2	30
3	
4	10
5	20
...	
31	32

PC:

36

\$ra 寄存器 返回地址

add:

```
→ 0    addu    $2,$4,$5  
    4    jr      $31  
    8    nop
```

main:

```
    12    addiu   $sp,$sp,-32  
    16    sw      $31,28($sp)  
→   20    li      $5,20  
    24    li      $4,10  
    28    jal     add  
    32    nop
```

jal: jump and link 的缩写，跳转和链接指令

① 将下一条指令的内存地址保存到 31 号寄存器中

② 跳转到 add 函数第一条指令所在内存地址

MIPS 指令系统：如何实现过程调用？

```
int func3(int a, int b) {  
    return a - b;  
}  
  
int func2(int a, int b) {  
    return func3(a, b);  
}  
  
int func1(int a, int b) {  
    return func2(a, b);  
}  
  
int main() {  
    int res = func1(10, 20);  
    return 0;  
}
```

```
func3:  
→ 0  addu    $2,$4,$5  
   4  jr      $31  
  
func2:  
→ 8  jal     func3  
→12  jr      $31  
  
func1:  
→16  jal     func2  
   20  jr     $31  
  
main:  
   24  li     $5,20  
   28  li     $4,10  
→ 32  jal     func1  
   36  nop  
   40  jr     $31
```

通用寄存器组：

0	0
1	
2	
3	
4	
5	
...	
31	12

\$ra

将返回地址存储到主存中

MIPS 指令系统：如何实现过程调用？

```
int func3(int a, int b) {  
    return a - b;  
}  
  
int func2(int a, int b) {  
    return func3(a, b);  
}  
  
int func1(int a, int b) {  
    return func2(a, b);  
}  
  
int main() {  
    int res = func1(10, 20);  
    return 0;  
}
```

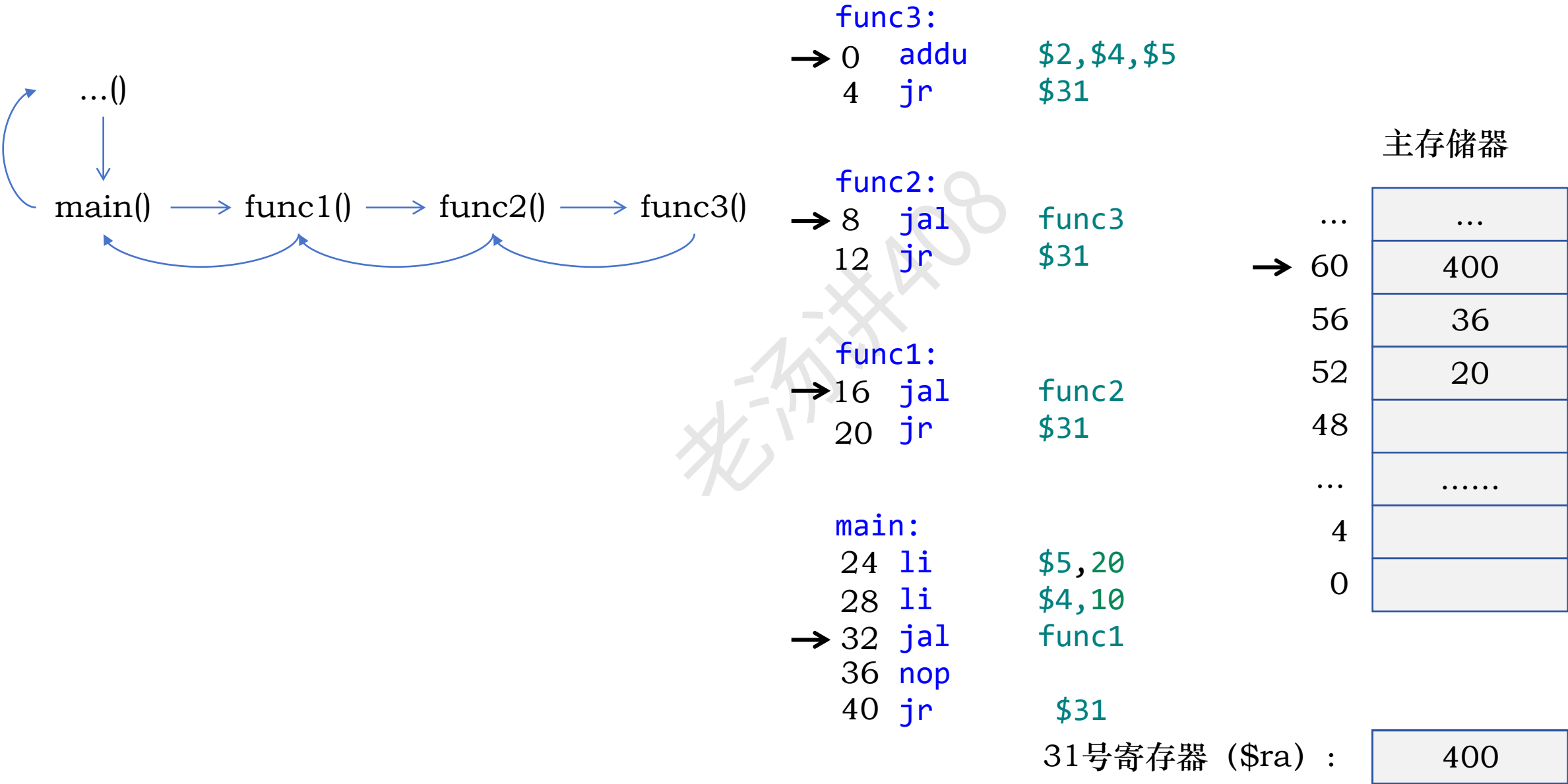
```
func3:  
→ 0 addu    $2,$4,$5  
  4 jr      $31  
  
func2:  
→ 8 jal     func3  
→ 12 jr     $31  
  
func1:  
→ 16 jal    func2  
→ 20 jr     $31  
  
main:  
→ 24 li     $5,20  
  28 li     $4,10  
  32 jal    func1  
→ 36 nop  
  40 jr     $31
```

主存储器

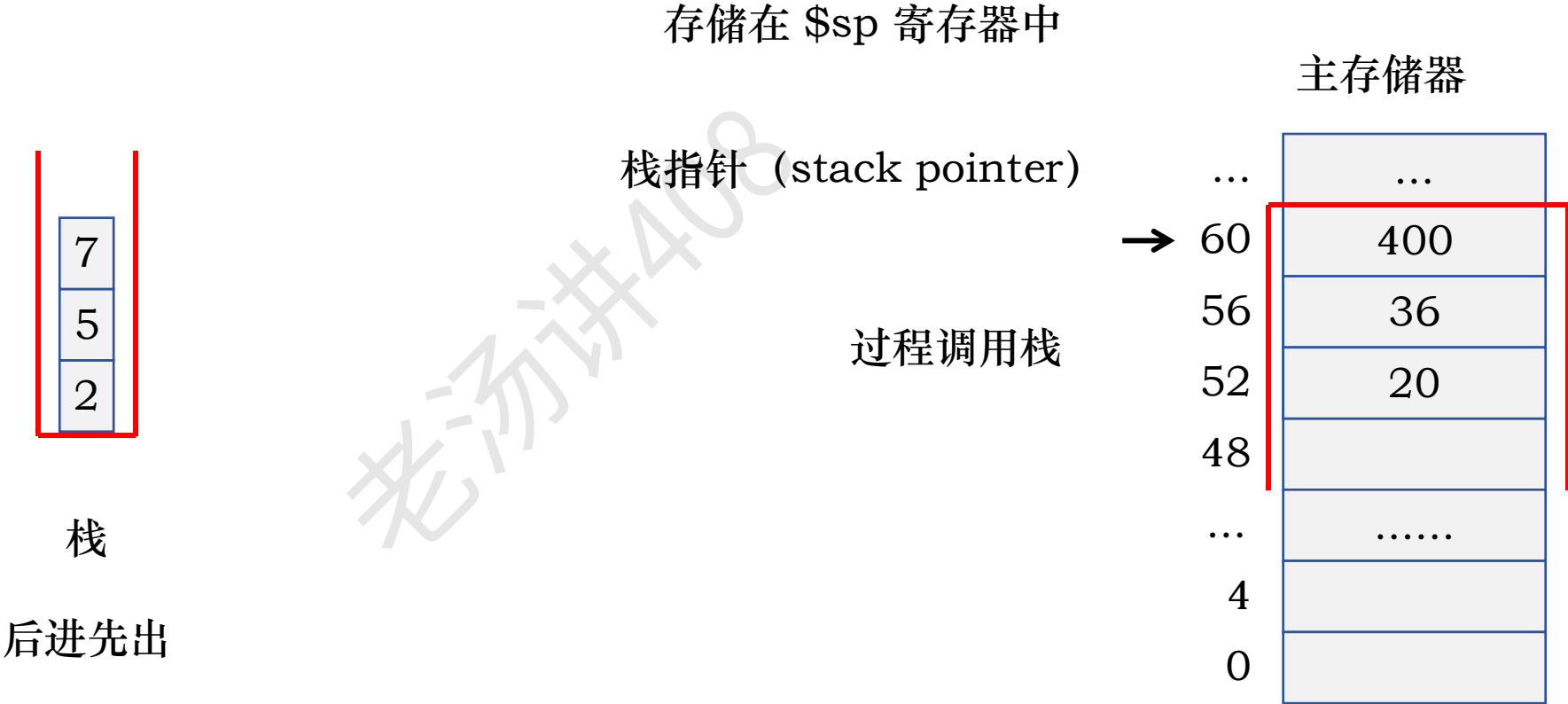
...	...
→ 60	400
56	36
52	20
48	
...	
4	
0	

31号寄存器 (\$ra) : 400

MIPS 指令系统：如何实现过程调用？



MIPS 指令系统：如何实现过程调用？



MIPS 指令系统：如何实现过程调用？

```
int func3(int a, int b) {  
    return a - b;  
}  
  
int func2(int a, int b) {  
    return func3(a, b);  
}  
  
int func1(int a, int b) {  
    return func2(a, b);  
}  
  
int main() {  
    int res = func1(10, 20);  
    printf("res = %d\n", res);  
    return 0;  
}
```

```
func3:  
0    addu    $2,$4,$5  
4    jr      $31
```

```
func2:  
8    jal     func3  
12   jr      $31
```

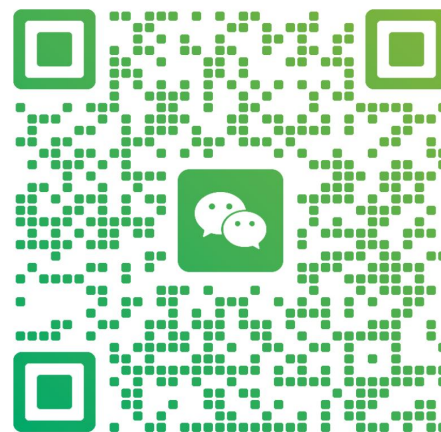
```
func1:  
16   jal     func2  
20   jr      $31
```

```
main:  
24   li      $5,20  
28   li      $4,10  
32   jal     func1  
36   nop  
40   jr      $31
```

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

MIPS 指令系统：过程的机器级表示

```
int func3(int a, int b) {  
    return a - b;  
}  
  
int func2(int a, int b) {  
    return func3(a, b);  
}  
  
int func1(int a, int b) {  
    return func2(a, b);  
}  
  
int main() {  
    int res = func1(10, 20);  
    printf("res = %d\n", res);  
    return 0;  
}
```

func3:

subu	\$2,\$4,\$5
jr	\$31

func2:

addiu	\$sp,\$sp,-32
sw	\$31,28(\$sp)
jal	<u>func3</u>
nop	

lw	\$31,28(\$sp)
addiu	\$sp,\$sp,32
jr	\$31
nop	

func1:

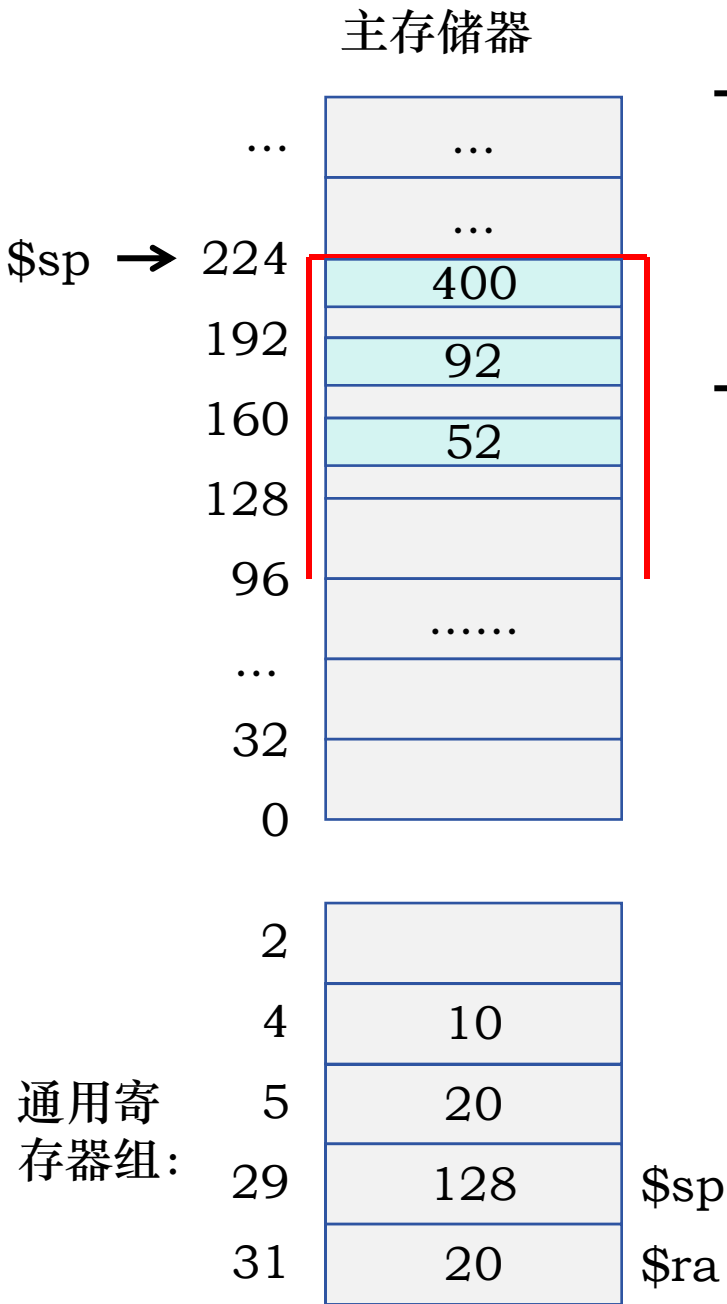
addiu	\$sp,\$sp,-32
sw	\$31,28(\$sp)
jal	<u>func2</u>
nop	

lw	\$31,28(\$sp)
addiu	\$sp,\$sp,32
jr	\$31
nop	

main:

addiu	\$sp,\$sp,-32
sw	\$31,28(\$sp)
li	\$5,20
li	\$4,10
jal	<u>func1</u>
nop	

MIPS 指令系统：过程的机器级表示



func3:

```
→ 0 subu $2,$4,$5
   4 jr $31
```

func2: 分配 32 字节的栈内存

```
→ 8 addiu $sp,$sp,-32
   12 sw $31,28($sp)
   16 jal func3
   20 nop
```

```
24 lw $31,28($sp)
28 addiu $sp,$sp,32
32 jr $31
36 nop
```

func1: 分配 32 字节的栈内存

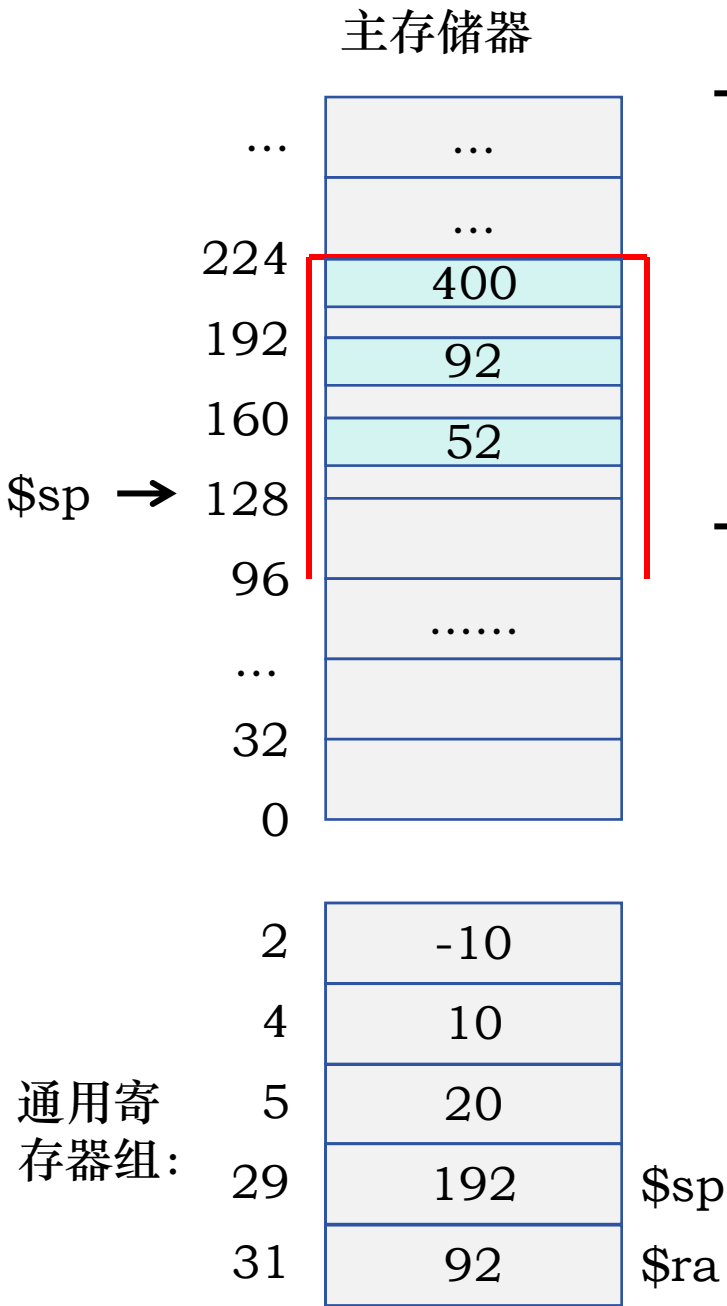
```
→ 40 addiu $sp,$sp,-32
   44 sw $31,28($sp)
   48 jal func2
   52 nop
```

```
56 lw $31,28($sp)
60 addiu $sp,$sp,32
64 jr $31
68 nop
```

main: 分配 32 字节的栈内存

```
→ 72 addiu $sp,$sp,-32
   76 sw $31,28($sp)
   80 li $5,20
   84 li $4,10
   88 jal func1
   92 nop
```


MIPS 指令系统：过程的机器级表示



func3:

```
→ 0    subu    $2,$4,$5
   4    jr      $31
```

func2: 分配 32 字节的栈内存

```
8    addiu    $sp,$sp,-32
12   sw       $31,28($sp)
→ 16   jal     func3
20   nop
```

```
24   lw       $31,28($sp)
28   addiu    $sp,$sp,32
32   jr       $31
36   nop
```

回收 32 字节的栈内存

func1: 分配 32 字节的栈内存

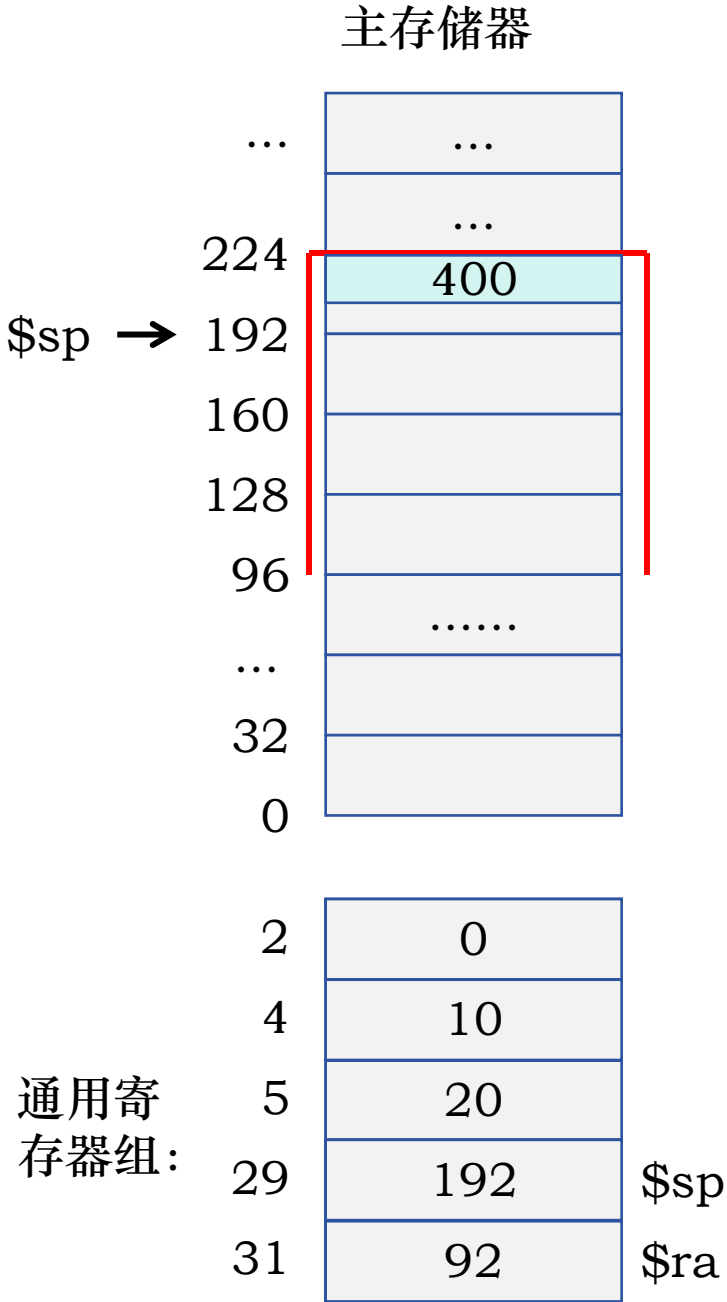
```
40   addiu    $sp,$sp,-32
44   sw       $31,28($sp)
→ 48   jal     func2
52   nop
```

```
56   lw       $31,28($sp)
60   addiu    $sp,$sp,32
64   jr       $31
68   nop
```

main: 分配 32 字节的栈内存

```
72   addiu    $sp,$sp,-32
76   sw       $31,28($sp)
80   li       $5,20
84   li       $4,10
→ 88   jal     func1
92   nop
```

MIPS 指令系统：过程的机器级表示



```
main:
    addiu    $sp,$sp,-32
    sw       $31,28($sp)
    li       $5,20
    li       $4,10
    jal      func1
    nop

    move     $5,$2
    lui      $4,%hi($LC0)
    addiu    $4,$4,%lo($LC0)
    jal      printf
    nop

    move     $2,$0
    lw       $31,28($sp)
    addiu    $sp,$sp,32
    jr       $31
    nop
```

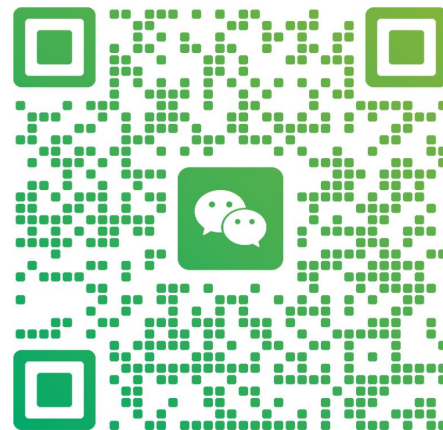

MIPS 指令系统：过程的机器级表示

```
int func3(int a, int b) {  
    return a - b;  
}  
  
int func2(int a, int b) {  
    return func3(a, b);  
}  
  
int func1(int a, int b) {  
    return func2(a, b);  
}  
  
int main() {  
    int res = func1(10, 20);  
    printf("res = %d\n", res);  
    return 0;  
}
```

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

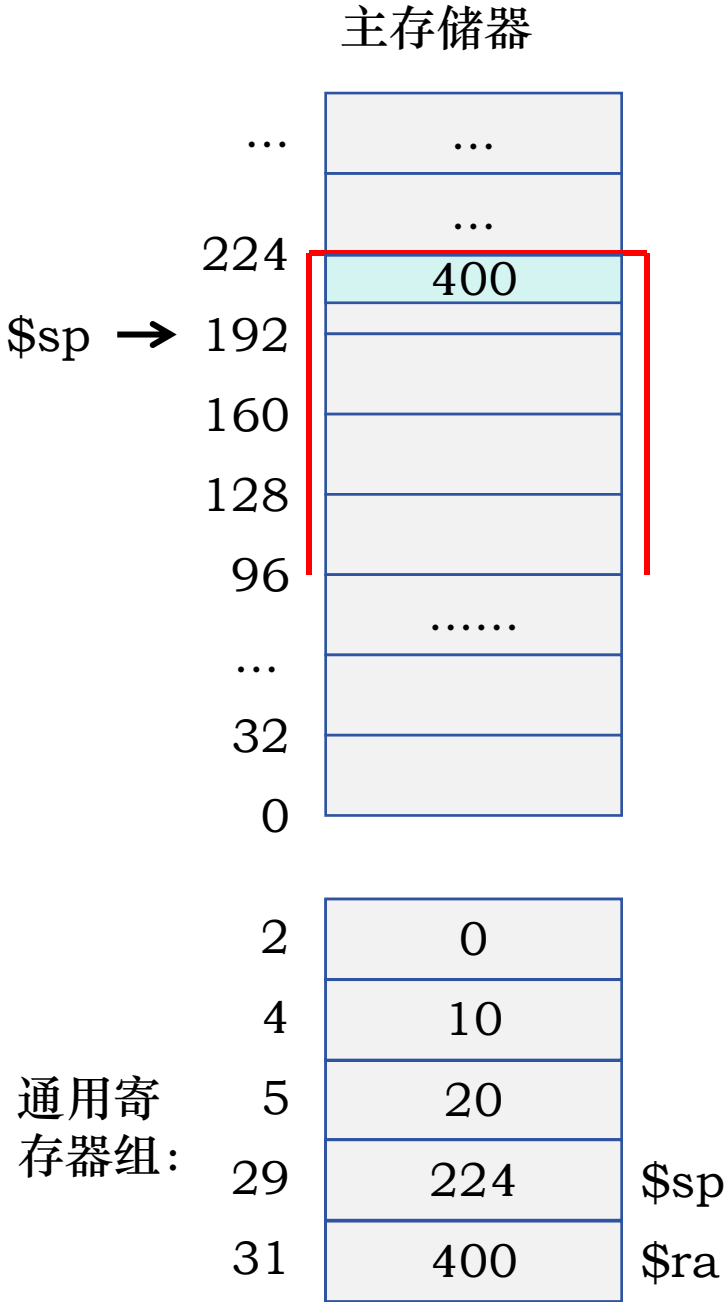


老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

MIPS 指令系统：过程的机器级表示



main:

```
addiu    $sp,$sp,-32
sw        $31,28($sp)
li        $5,20
li        $4,10
jal       func1
nop
```

```
move      $5,$2
lui        $4,%hi($LC0)
addiu      $4,$4,%lo($LC0)
jal        printf
nop
```

```
→ move     $2,$0
lw          $31,28($sp)
addiu       $sp,$sp,32
jr          $31
nop
```

MIPS 指令系统：通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0 ~ v1	2 ~ 3	过程调用返回值
a0 ~ a3	4 ~ 7	过程调用参数
t0 ~ t7	8 ~ 15	临时变量，在被调用过程无需保存
s0 ~ s7	16 ~ 23	在被调用过程需保存
t8 ~ t9	24 ~ 25	临时变量，在被调用过程无需保存
k0 ~ k1	26 ~ 27	为 OS 保留
gp	28	全局指针
sp	29	栈指针
fp	30	帧指针
ra	31	过程调用返回地址

MIPS 指令系统：哪些数据保存到栈内存？

```
int func1(int a, int b, int c, int d) {  
    return a + b + c + d;  
}  
  
int main() {  
    int res = func1(10, 20, 20, 11);  
    return res;  
}
```

func1:

addu	\$4,\$4,\$5
addu	\$4,\$4,\$6
addu	\$2,\$4,\$7
jr	\$31
nop	

main:

addiu	\$sp,\$sp,-32
sw	\$31,28(\$sp)
li	\$7,11
li	\$6,20
li	\$5,20
li	\$4,10
jal	<u>func1</u>
nop	

当参数的个数超过了 4 个，那么就需要通过栈内存来传递参数了

lw	\$31,28(\$sp)
addiu	\$sp,\$sp,32
jr	\$31
nop	

MIPS 指令系统：哪些数据保存到栈内存？

```
int func1(int a, int b, int c, int d, int e, int f) {  
    return a + b + c + d + e + f;  
}  
  
int main() {  
    int res = func1(10, 20, 20, 11, 5, 8);  
    return res;  
}
```

func1:

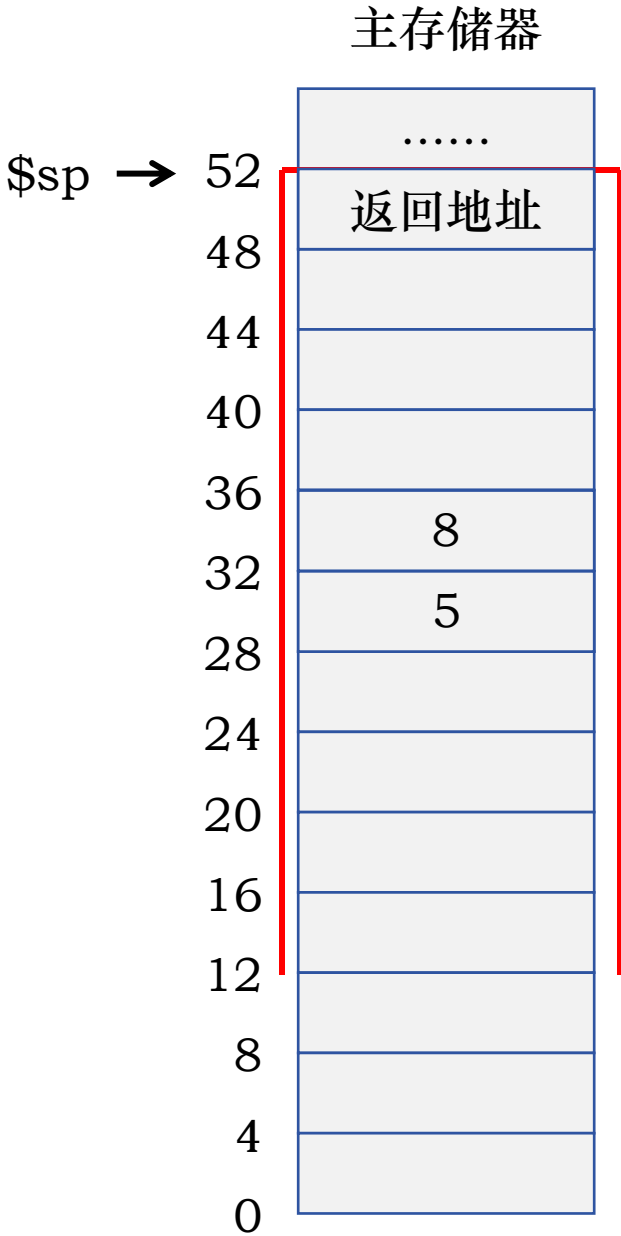
addu	\$4,\$4,\$5
addu	\$4,\$4,\$6
addu	\$4,\$4,\$7
lw	\$2,16(\$sp)
nop	
addu	\$4,\$4,\$2
lw	\$2,20(\$sp)
nop	
addu	\$2,\$4,\$2
jr	\$31
nop	

main:

addiu	\$sp,\$sp,-40
sw	\$31,36(\$sp)
li	\$2,8
sw	\$2,20(\$sp)
li	\$2,5
sw	\$2,16(\$sp)
li	\$7,11
li	\$6,20
li	\$5,20
li	\$4,10
jal	<u>func1</u>
nop	

lw	\$31,36(\$sp)
addiu	\$sp,\$sp,40
jr	\$31
nop	

MIPS 指令系统：哪些数据保存到栈内存？



func1:

```
addu $4,$4,$5
addu $4,$4,$6
addu $4,$4,$7
→ lw $2,16($sp)
nop
addu $4,$4,$2
→ lw $2,20($sp)
nop
addu $2,$4,$2
jr $31
nop
```

main:

```
→ addiu $sp,$sp,-40
sw $31,36($sp)
li $2,8
sw $2,20($sp)
li $2,5
sw $2,16($sp)
li $7,11
li $6,20
li $5,20
li $4,10
jal func1
nop

lw $31,36($sp)
addiu $sp,$sp,40
jr $31
nop
```

MIPS 指令系统：通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0 ~ v1	2 ~ 3	过程调用返回值
a0 ~ a3	4 ~ 7	过程调用参数
t0 ~ t7	8 ~ 15	临时变量，在被调用过程无需保存
s0 ~ s7	16 ~ 23	在被调用过程需保存
t8 ~ t9	24 ~ 25	临时变量，在被调用过程无需保存
k0 ~ k1	26 ~ 27	为 OS 保留
gp	28	全局指针
sp	29	栈指针
fp	30	帧指针
ra	31	过程调用返回地址

MIPS 指令系统：哪些数据保存到栈内存？

```
int func3(int a, int b) {  
    return a - b;  
}
```

```
int func2(int a, int b) {  
    int c = func3(a, b);  
    int d = a - b - c;  
    return d;  
}
```

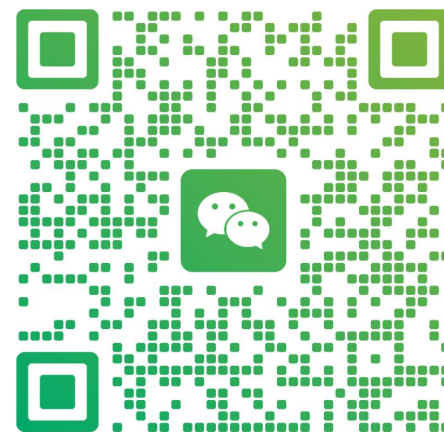
```
int func1(int a, int b) {  
    int m = a + b;  
    int c = func2(m, b);  
    int d = m + c;  
    return d;  
}
```

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

MIPS 指令系统：通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0 ~ v1	2 ~ 3	过程调用返回值
a0 ~ a3	4 ~ 7	过程调用参数
t0 ~ t7	8 ~ 15	临时变量，在被调用过程无需保存
s0 ~ s7	16 ~ 23	在被调用过程需保存
t8 ~ t9	24 ~ 25	临时变量，在被调用过程无需保存
k0 ~ k1	26 ~ 27	为 OS 保留
gp	28	全局指针
sp	29	栈指针
fp	30	帧指针
ra	31	过程调用返回地址

MIPS 指令系统：哪些数据保存到栈内存？

```
int func3(int a, int b) {  
    return a - b;  
}
```

```
int func2(int a, int b) {  
    int c = func3(a, b);  
    int d = a - b - c;  
    return d;  
}
```

```
int func1(int a, int b) {  
    int m = a + b;  
    int c = func2(m, b);  
    int d = m + c;  
    return d;  
}
```

保存 \$16 到栈内存
a + b, 结果放 \$16

恢复 \$16 的值

func (...)

func1:

```
addiu    $sp,$sp,28  
sw        $31,28($sp)  
sw        $16,24($sp)  
addu     $16,$4,$5  
move     $4,$16  
jal      func2  
nop
```

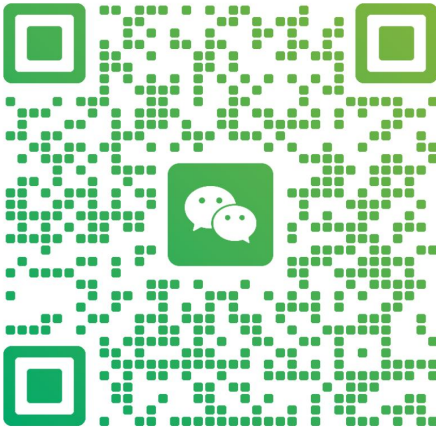
```
addu     $2,$16,$2  
lw        $31,28($sp)  
lw        $16,24($sp)  
addiu    $sp,$sp,32  
jr        $31  
nop
```

在 func 函数中，在 16 号寄存器中保存了某局部变量的数据，而且这个数据是需要等 func1 调用完后才会被用到

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

MIPS 指令系统：哪些数据保存到栈内存？

```
int func3(int a, int b) {  
    return a - b;  
}
```

```
int func2(int a, int b) {  
    int c = func3(a, b);  
    int d = a - b - c;  
    return d;  
}
```

```
int func1(int a, int b) {  
    int m = a + b;  
    int c = func2(m, b);  
    int d = m + c;  
    return d;  
}
```

func (...)

func1:

addiu	\$sp, \$sp, -32
sw	\$31, 28(\$sp)
sw	\$16, 24(\$sp)
addu	\$16, \$4, \$5
move	\$4, \$16
jal	func2
nop	

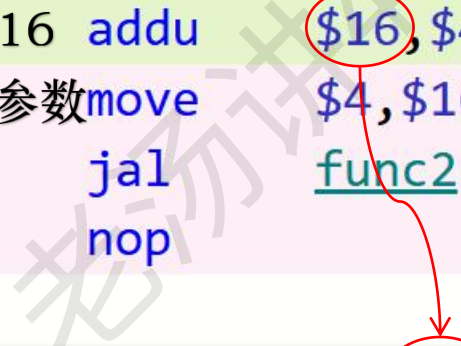
addu	\$2, \$16, \$2
lw	\$31, 28(\$sp)
lw	\$16, 24(\$sp)
addiu	\$sp, \$sp, 32
jr	\$31
nop	

func2:

addiu	\$sp, \$sp, -40
sw	\$31, 36(\$sp)
sw	\$17, 32(\$sp)
sw	\$16, 28(\$sp)
move	\$16, \$4
move	\$17, \$5
jal	func3
nop	

subu	\$16, \$16, \$17
subu	\$2, \$16, \$2
lw	\$31, 36(\$sp)
lw	\$17, 32(\$sp)
lw	\$16, 28(\$sp)
addiu	\$sp, \$sp, 40
jr	\$31
nop	

a + b, 结果放 \$16
\$16 中的 m 作为参数

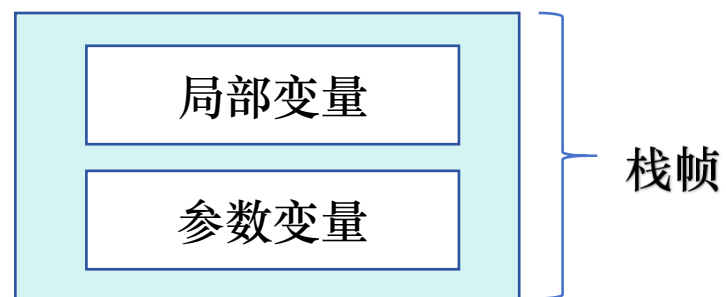


MIPS 指令系统：通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0 ~ v1	2 ~ 3	过程调用返回值
a0 ~ a3	4 ~ 7	过程调用参数
t0 ~ t7	8 ~ 15	临时变量，在被调用过程无需保存
s0 ~ s7	16 ~ 23	在被调用过程需保存
t8 ~ t9	24 ~ 25	临时变量，在被调用过程无需保存
k0 ~ k1	26 ~ 27	为 OS 保留
gp	28	全局指针
sp	29	栈指针
fp	30	帧指针
ra	31	过程调用返回地址

MIPS 指令系统：过程的机器级表示

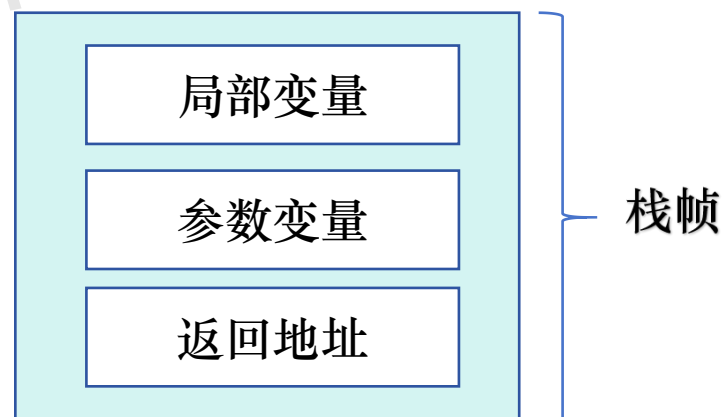
```
int func3(int a, int b) {  
    return a - b;  
}
```



```
int func2(int a, int b) {  
    int c = func3(a, b);  
    int d = a - b - c;  
    return d;  
}
```



```
int func1(int a, int b) {  
    int m = a + b;  
    int c = func2(m, b);  
    int d = m + c;  
    return d;  
}
```



MIPS 指令系统：过程的机器级表示

编译时，将 -Og 这个优化选项去掉

```
main:
int func2(int a, int b) {
    return a - b;
}

int func1(int a, int b) {
    int c = a + b;
    return func2(a, c);
}

int main() {
    int res = func1(10, 20);
    return res;
}
```

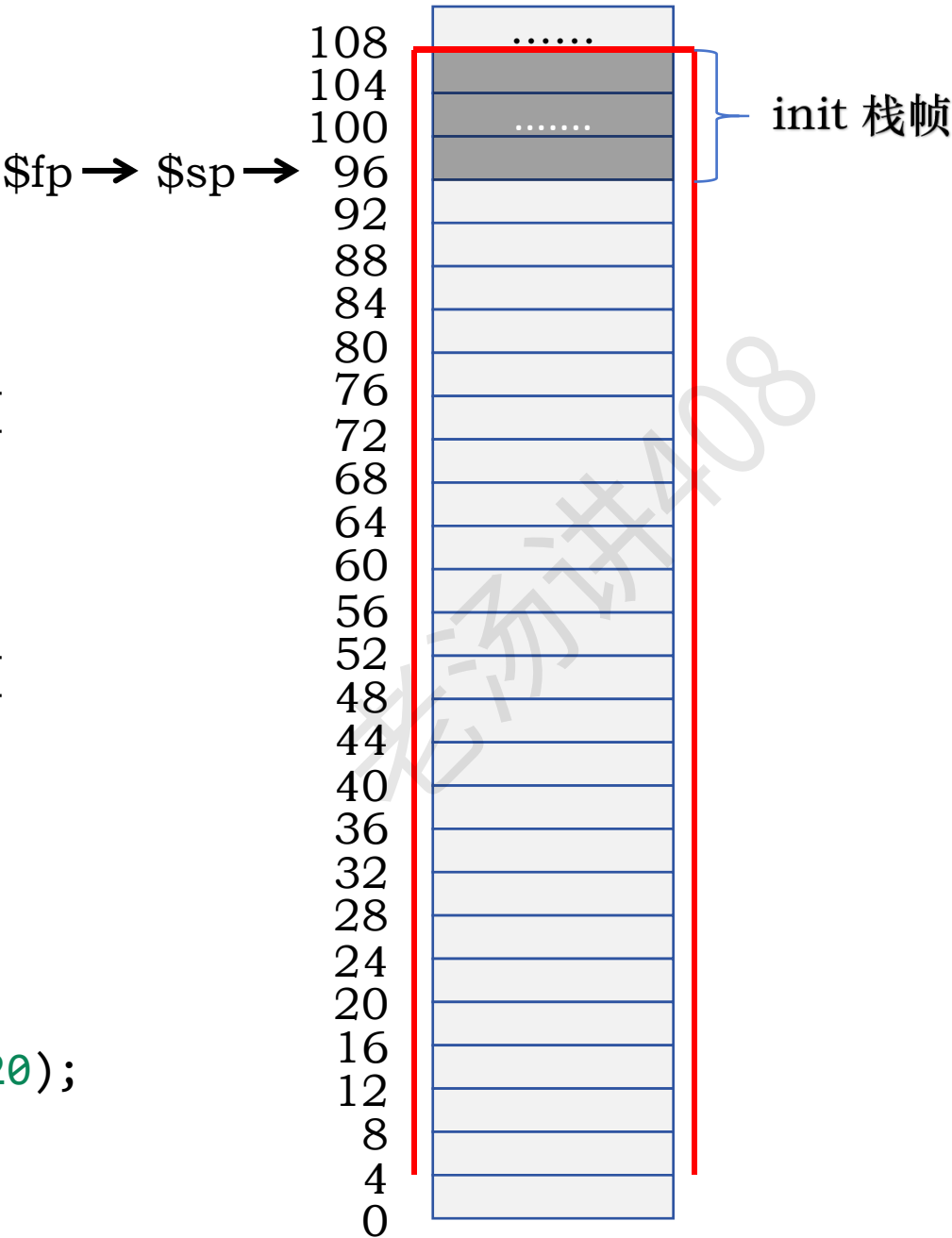
```
addiu $sp,$sp,-40
sw
sw
move
li
li
jal
nop
sw
lw
move
lw
lw
addiu
jr
nop
```

```
func2:
addiu $sp,$sp,-8
sw
move
sw
sw
lw
lw
nop
subu
move
lw
addiu
jr
nop
```

```
func1:
addiu $sp,$sp,-40
sw
sw
$fp,$sp
$4,40($fp)
$5,44($fp)
$3,40($fp)
$2,44($fp)
$2,$3,$2
$2,24($fp)
$5,24($fp)
$4,40($fp)
func2
$sp,$fp
$fp,4($sp)
$sp,$sp,8
$31
addiu $sp,$sp,40
jr
nop
```


MIPS 指令系统：过程的机器级表示

主存储器



```
int func2(int a, int b) {  
    return a - b;  
}  
  
int func1(int a, int b) {  
    int c = a + b;  
    return func2(a, c);  
}  
  
int main() {  
    int res = func1(10, 20);  
    return res;  
}
```

main:

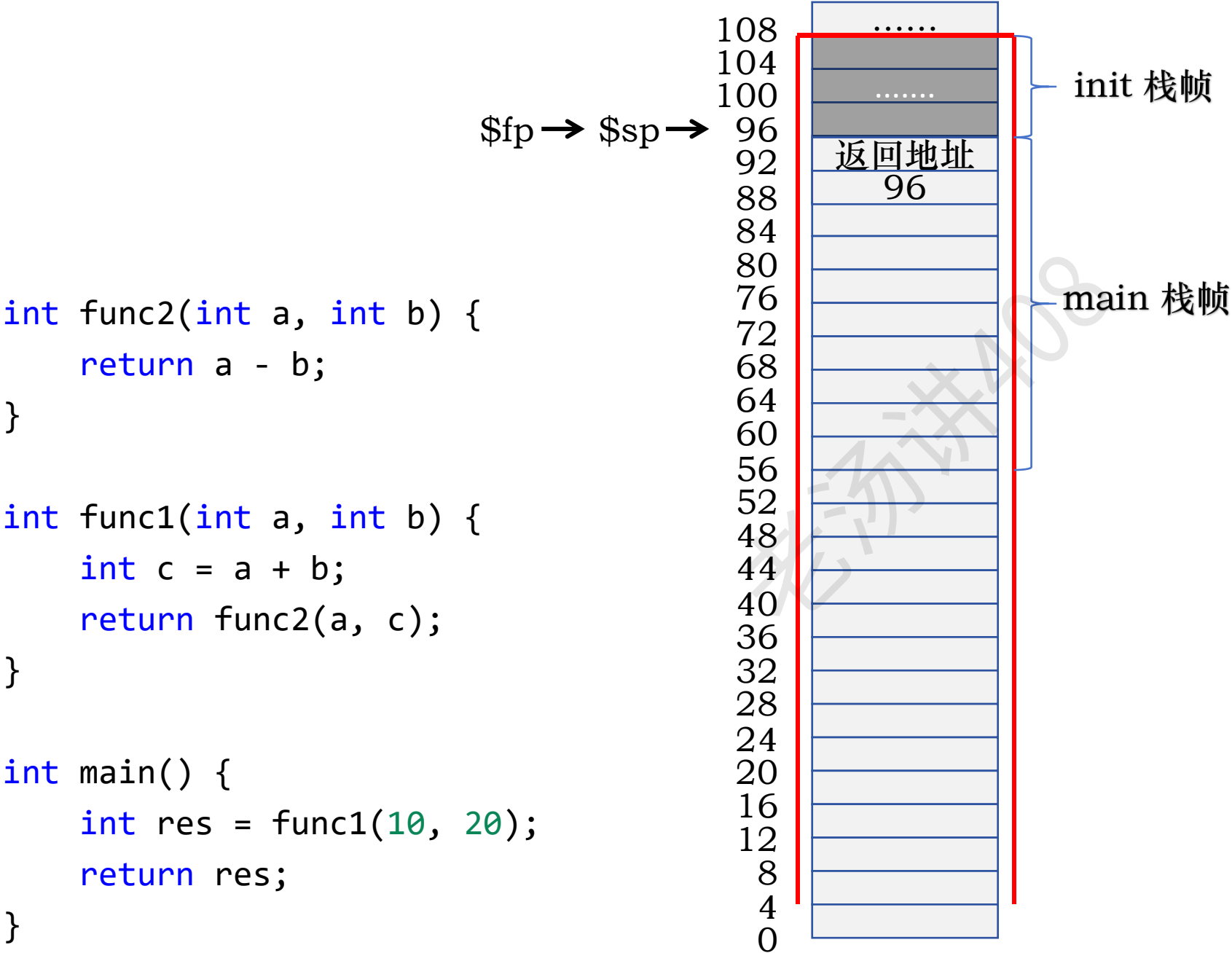
```
addiu    $sp,$sp,-40  
sw       $31,36($sp)  
sw       $fp,32($sp)  
move     $fp,$sp  
  
li       $5,20  
li       $4,10  
jal      func1  
nop  
  
sw       $2,24($fp)  
lw       $2,24($fp)  
  
move     $sp,$fp  
lw       $31,36($sp)  
lw       $fp,32($sp)  
addiu    $sp,$sp,40  
jr       $31  
nop
```

MIPS 指令系统：通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0 ~ v1	2 ~ 3	过程调用返回值
a0 ~ a3	4 ~ 7	过程调用参数
t0 ~ t7	8 ~ 15	临时变量，在被调用过程无需保存
s0 ~ s7	16 ~ 23	在被调用过程需保存
t8 ~ t9	24 ~ 25	临时变量，在被调用过程无需保存
k0 ~ k1	26 ~ 27	为 OS 保留
gp	28	全局指针
sp	29	栈指针 (stack pointer)
fp	30	帧指针 (frame pointer)
ra	31	过程调用返回地址

MIPS 指令系统：过程的机器级表示

主存储器



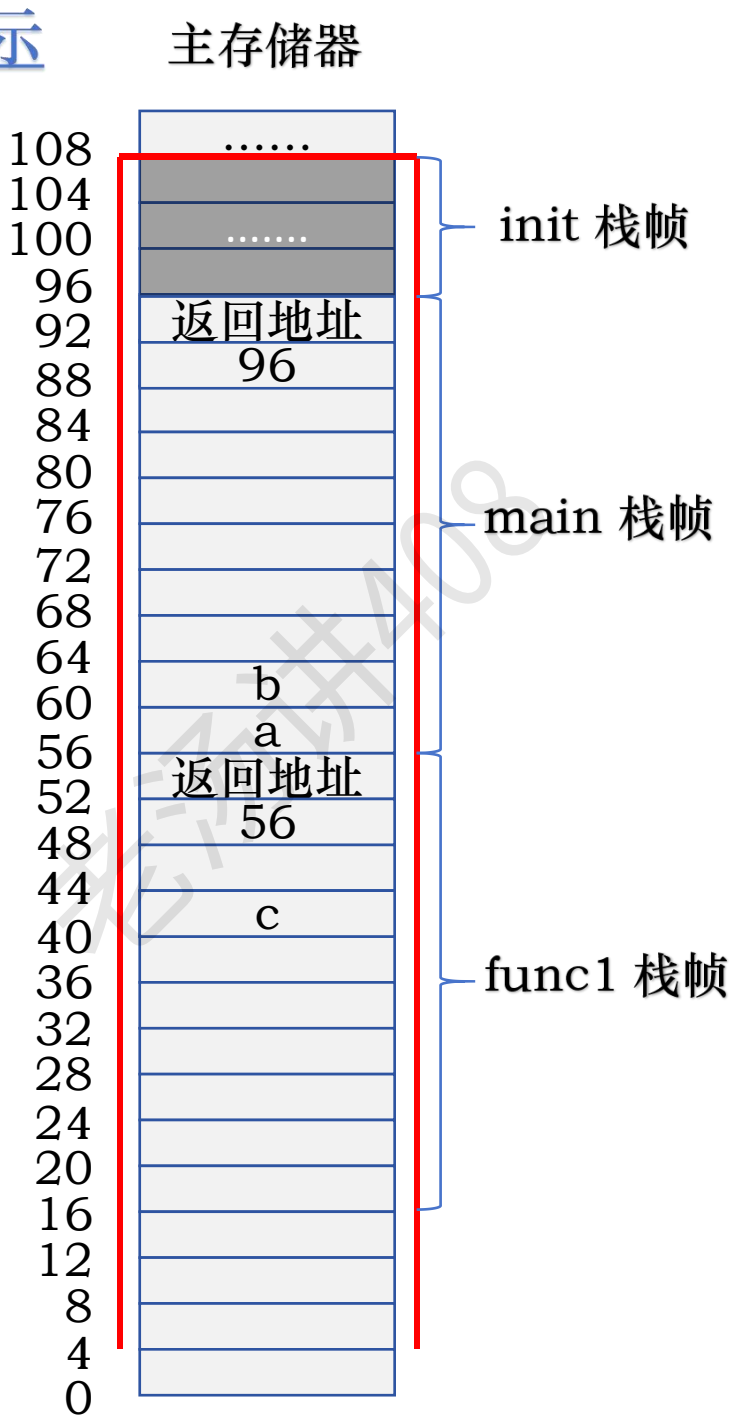
main:

```
→ addiu    $sp, $sp, -40  
   sw      $31, 36($sp)  
   sw      $fp, 32($sp)  
   move    $fp, $sp  
  
   li      $5, 20  
   li      $4, 10  
   jal     func1  
   nop
```

```
   sw      $2, 24($fp)  
   lw      $2, 24($fp)  
  
   move    $sp, $fp  
   lw      $31, 36($sp)  
   lw      $fp, 32($sp)  
   addiu   $sp, $sp, 40  
   jr      $31  
   nop
```

MIPS 指令系统：过程的机器级表示

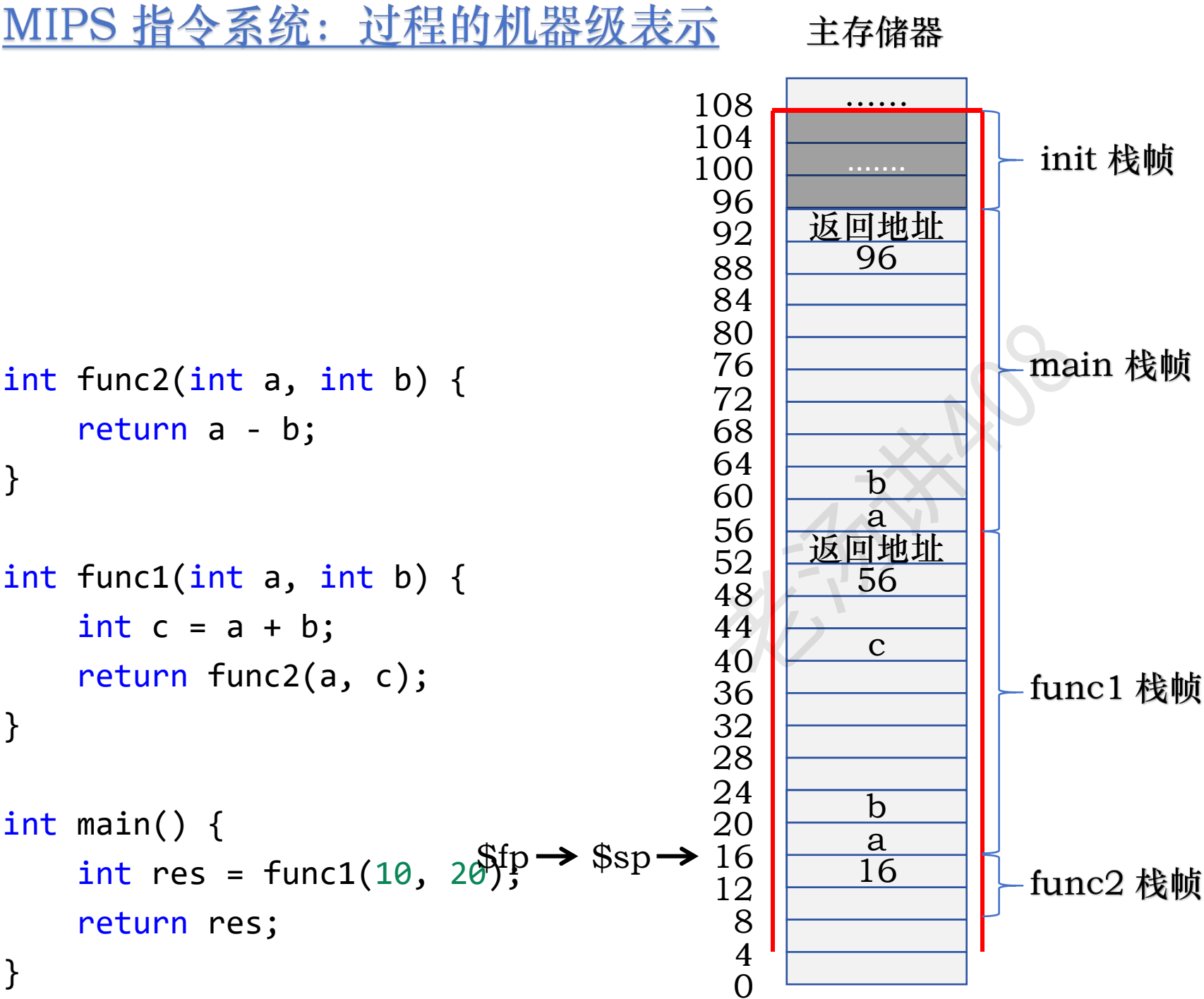
```
int func2(int a, int b) {  
    return a - b;  
}  
  
int func1(int a, int b) {  
    int c = a + b;  
    return func2(a, c);  
}  
  
int main() {  
    int res = func1(10, 20);  
    return res;  
}
```



func1:

```
→ addiu    $sp, $sp, -40  
   sw      $31, 36($sp)  
   sw      $fp, 32($sp)  
   move    $fp, $sp  
   sw      $4, 40($fp)  
   sw      $5, 44($fp)  
  
   lw      $3, 40($fp)  
   lw      $2, 44($fp)  
   nop  
   addu    $2, $3, $2  
   sw      $2, 24($fp)  
  
   lw      $5, 24($fp)  
   lw      $4, 40($fp)  
   jal     func2  
   nop  
  
   move    $sp, $fp  
   lw      $31, 36($sp)  
   lw      $fp, 32($sp)  
   addiu   $sp, $sp, 40  
   jr      $31  
   nop
```

MIPS 指令系统：过程的机器级表示



```
func2:  
    → addiu    $sp,$sp,-8  
    sw        $fp,4($sp)  
    move      $fp,$sp  
    sw        $4,8($fp)  
    sw        $5,12($fp)  
  
    lw        $3,8($fp)  
    lw        $2,12($fp)  
    nop  
    subu      $2,$3,$2  
  
    move      $sp,$fp  
    lw        $fp,4($sp)  
    addiu     $sp,$sp,8  
    jr        $31  
    nop
```


MIPS 指令系统：过程的机器级表示

fp 的作用：

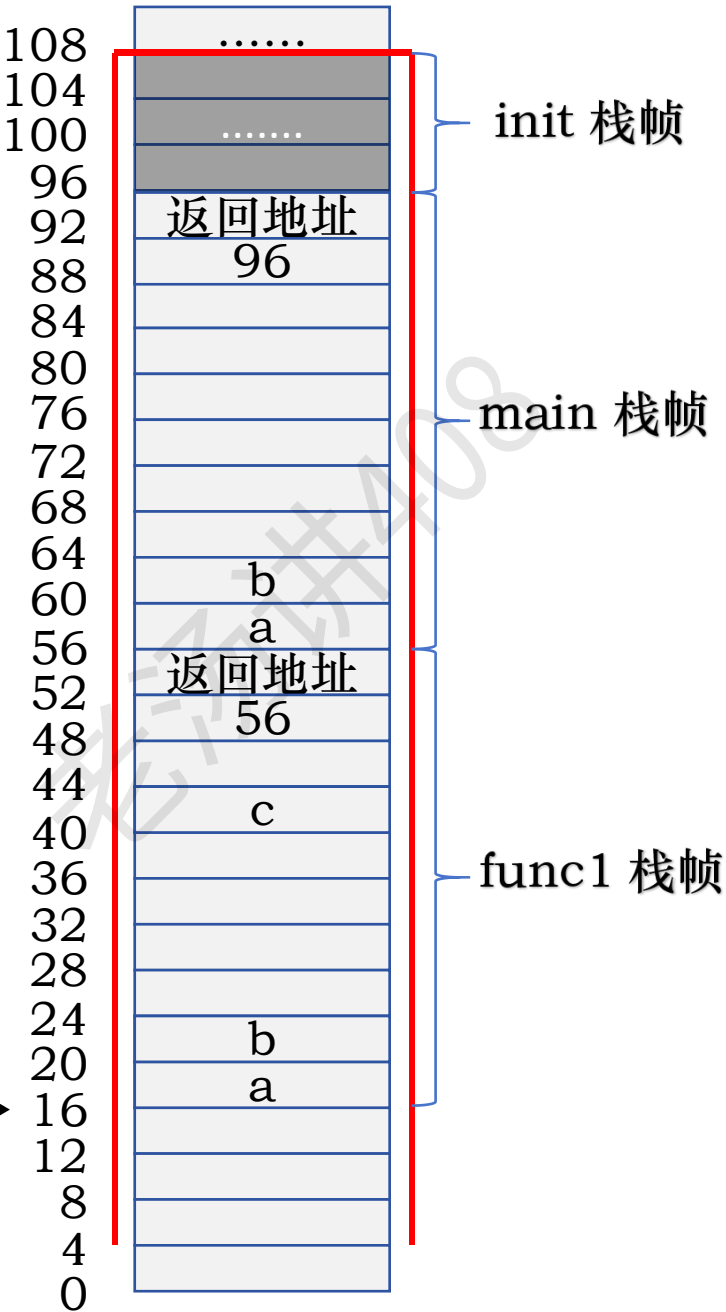
- ① 用于函数调用返回时，回到上一个栈帧
- ② 用于计算当前栈帧的相对内存地址

```
int func2(int a, int b) {  
    return a - b;  
}
```

```
int func1(int a, int b) {  
    int c = a + b;  
    return func2(a, c);  
}
```

```
int main() {  
    int res = func1(10, 20);  
    return res;  
}
```

主存储器



func1:

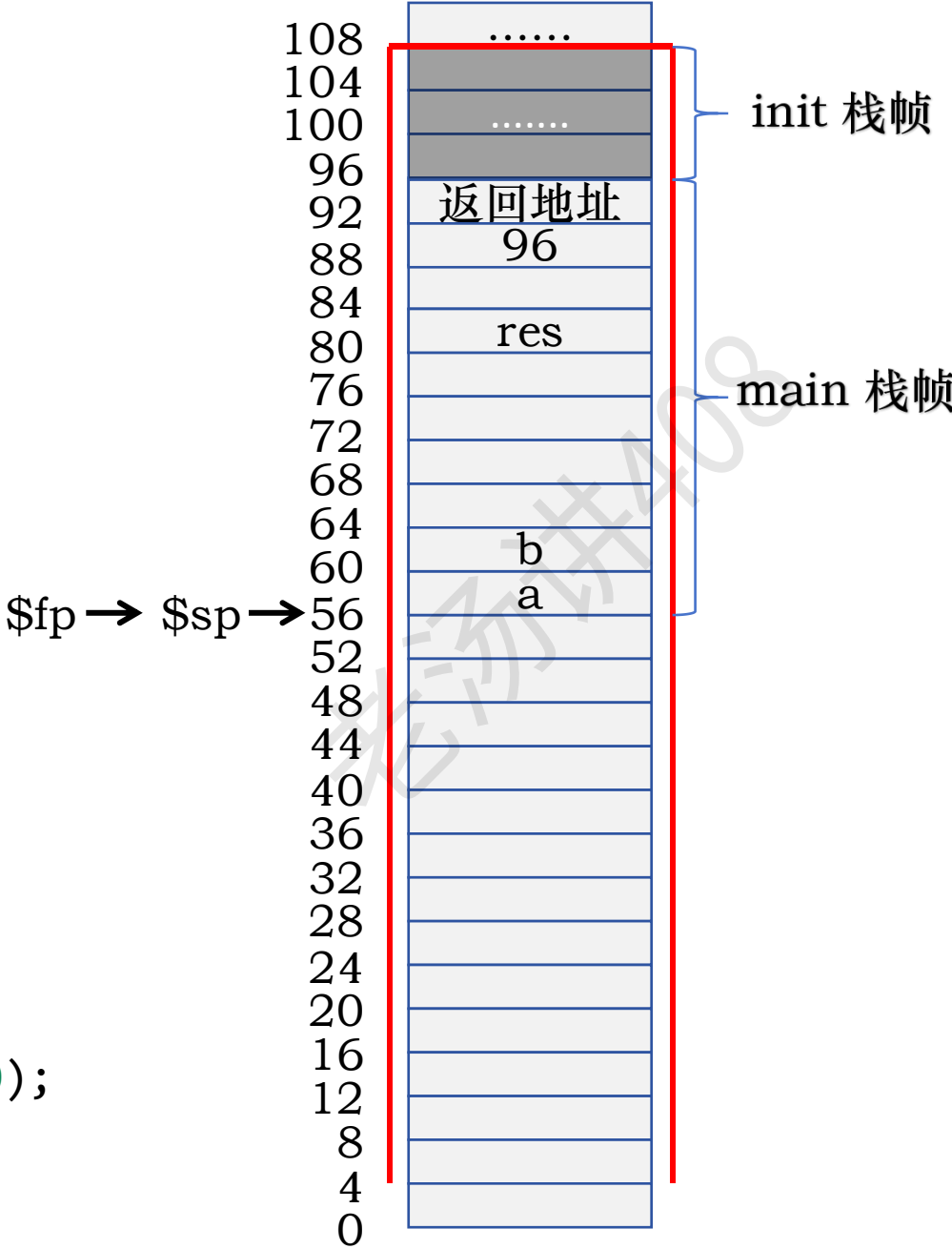
```
addiu    $sp,$sp,-40  
sw       $31,36($sp)  
sw       $fp,32($sp)  
move     $fp,$sp  
sw       $4,40($fp)  
sw       $5,44($fp)  
lw       $3,40($fp)  
lw       $2,44($fp)  
nop  
addu     $2,$3,$2  
sw       $2,24($fp)  
lw       $5,24($fp)  
lw       $4,40($fp)  
jal      func2  
→nop
```

```
move     $sp,$fp  
lw       $31,36($sp)  
lw       $fp,32($sp)  
addiu    $sp,$sp,40  
jr       $31  
nop
```

MIPS 指令系统：过程的机器级表示

主存储器

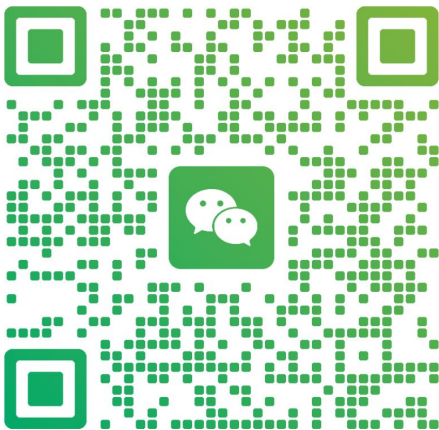
```
int func2(int a, int b) {  
    return a - b;  
}  
  
int func1(int a, int b) {  
    int c = a + b;  
    return func2(a, c);  
}  
  
int main() {  
    int res = func1(10, 20);  
    return res;  
}
```



```
main:  
    addiu    $sp,$sp,-40  
    sw       $31,36($sp)  
    sw       $fp,32($sp)  
    move     $fp,$sp  
    li       $5,20  
    li       $4,10  
    jal      func1  
    → nop  
  
    sw       $2,24($fp)  
    lw       $2,24($fp)  
    move     $sp,$fp  
    lw       $31,36($sp)  
    lw       $fp,32($sp)  
    addiu    $sp,$sp,40  
    jr       $31  
    nop
```


MIPS 指令系统：J-型指令格式（无条件跳转指令）

加老汤微信，帮你 408 冲 120+



扫一扫上面的二维码图案，加我为朋友。

main:

```
addiu    $sp,$sp,-40
sw        $31,36($sp)
sw        $fp,32($sp)
move     $fp,$sp
```

```
li        $5,20
li        $4,10
jal       func1
nop
```

```
sw        $2,24($fp)
lw        $2,24($fp)
```

```
move     $sp,$fp
lw        $31,36($sp)
lw        $fp,32($sp)
addiu    $sp,$sp,40
jr        $31
nop
```

MIPS 指令系统：J-型指令格式（无条件跳转指令）

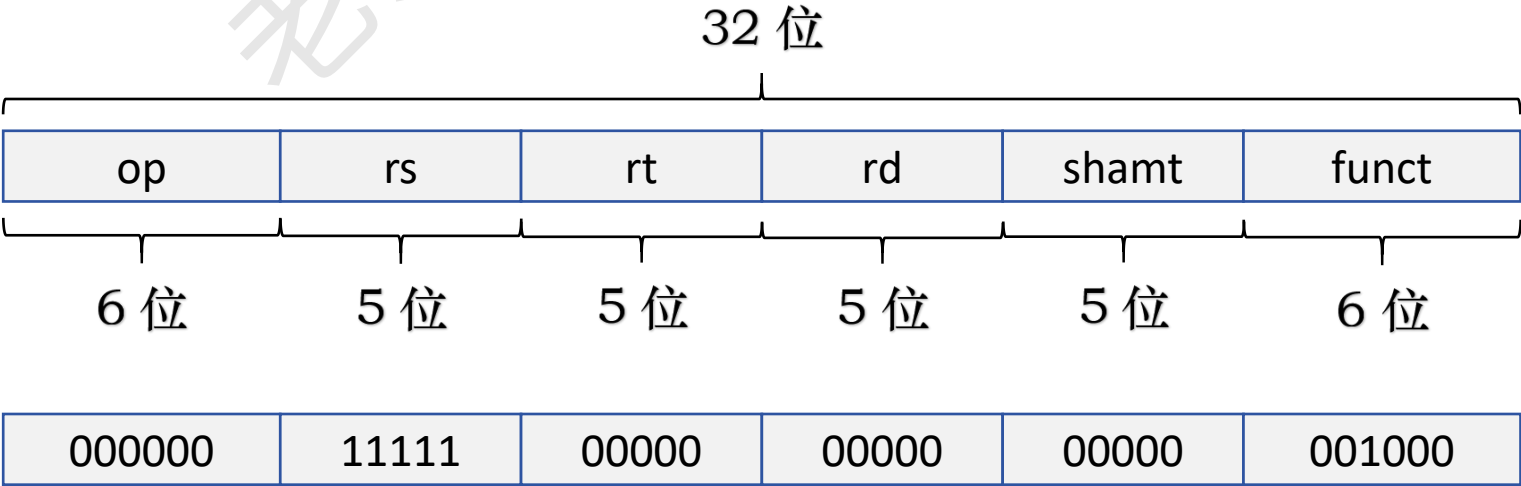
无条件跳转	跳转	j 2500	go to 10000	跳转到目标地址
	跳转到寄存器所指位置	jr \$ra	go to \$ra	用于 switch 和过程调用
	跳转并链接	jal 2500	\$ra=PC+4; go to 10000	用于过程调用

j: jump 的缩写，无条件跳转指令

jr: jump register 的缩写，跳转寄存器指令（函数返回）

jal: jump and link 的缩写，跳转并链接指令（过程调用）

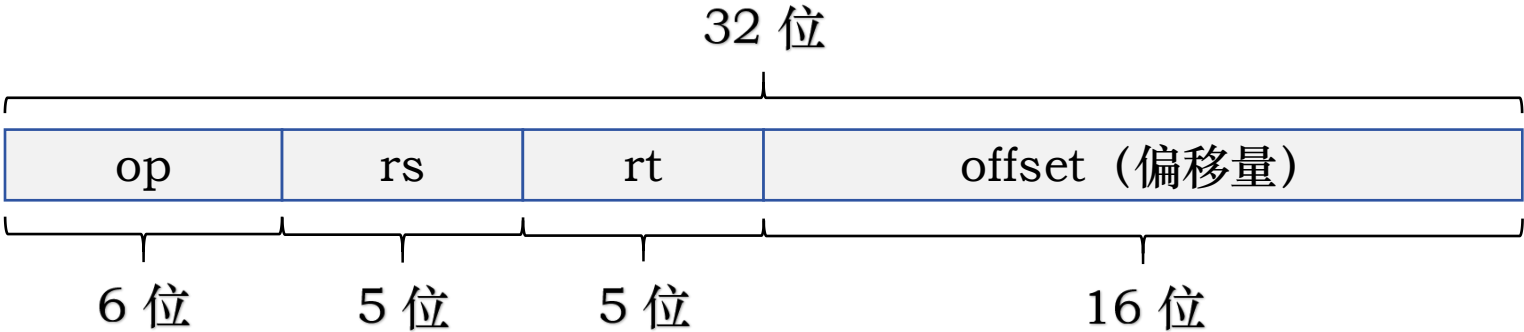
R-型指令格式



jr: \$31

MIPS 指令系统：J-型指令格式（无条件跳转指令）

I-型指令格式



无条件跳转：

beq \$0, \$0, 4

000100	0	0	0000 0000 0000 0100
--------	---	---	---------------------

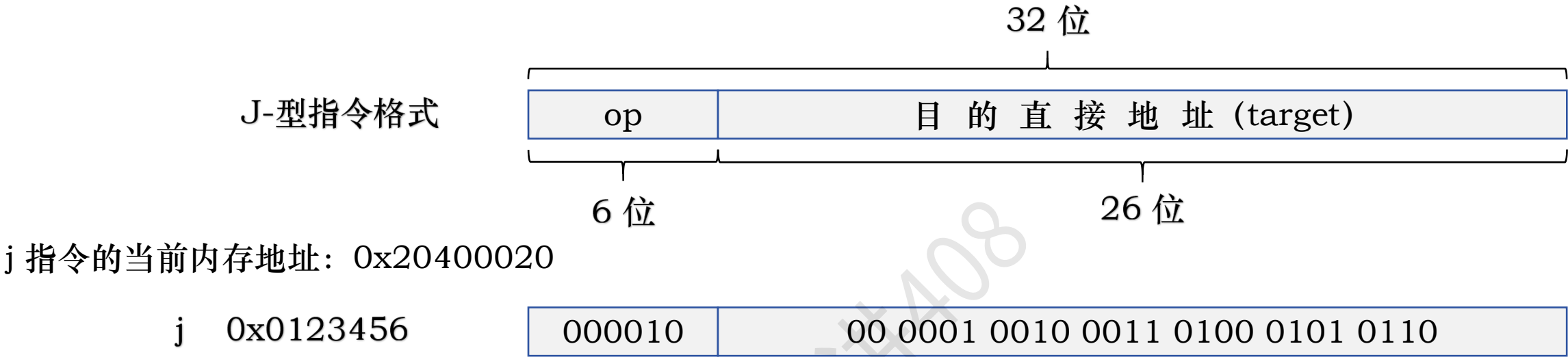
目的内存地址：0x12345678

16 位无符号整数的取值范围：0 ~ 2¹⁶ - 1 (0 ~ 65535)

相对寻址：相对当前的 PC 值计算下一条执行指令的内存地址

16 位补码的取值范围：-2¹⁵ ~ 2¹⁵ - 1 (-32768 ~ 32767)

MIPS 指令系统：J-型指令格式（无条件跳转指令 j） - 第 5 章的第 16 小节



步骤 1：扩展目标地址 $0x0123456 \ll 2 = 0x048D158$

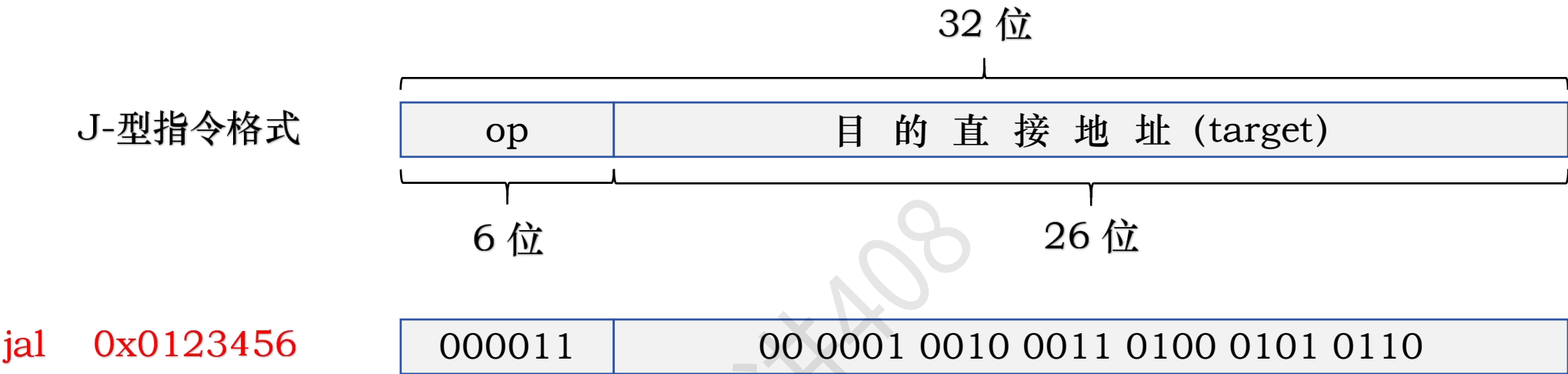
步骤 2：拼接地址 $0x20400020 \& 0xF0000000 = 0x20000000$

目的地址 = $0x20000000 \mid 0x048D158 = 0x2048D158$

目的地址 = (当前指令地址 & 0xF0000000) | (target << 2)

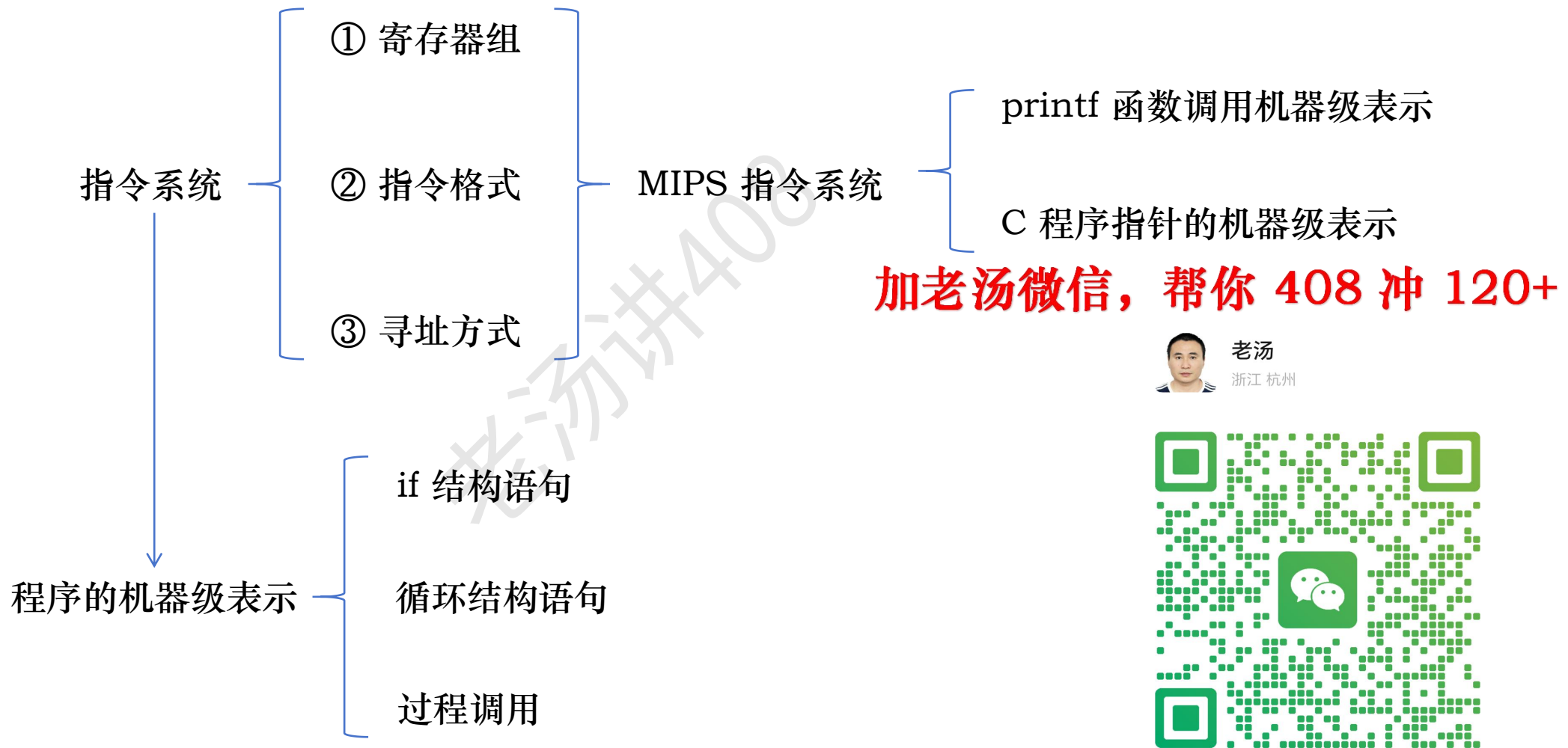
MIPS 指令系统：过程的机器级表示

jal: jump and link 的缩写，跳转和链接指令（过程调用）



目的地址 = (当前指令地址 & 0xF0000000) | (target << 2)

MIPS 指令系统：总结



扫一扫上面的二维码图案，加我为朋友。

MIPS 指令系统：总结（1、寄存器组）

通用寄存器

名称	编号	功能
zero	0	恒为 0
at	1	为汇编程序保留
v0 ~ v1	2 ~ 3	过程调用返回值
a0 ~ a3	4 ~ 7	过程调用参数
t0 ~ t7	8 ~ 15	临时变量，在被调用过程无需保存
s0 ~ s7	16 ~ 23	在被调用过程需保存
t8 ~ t9	24 ~ 25	临时变量，在被调用过程无需保存
k0 ~ k1	26 ~ 27	为 OS 保留
gp	28	全局指针
sp	29	栈指针 (stack pointer)
fp	30	帧指针 (frame pointer)
ra	31	过程调用返回地址

专用寄存器

乘商寄存器：Hi 和 Lo

程序员可见的寄存器

MIPS 指令系统：总结（1、寄存器组）

程序员可见的寄存器
(可编程寄存器)

通用寄存器组

乘商寄存器：Hi 和 Lo

程序计数器 PC

程序员不可见的寄存器
(不可编程寄存器)

指令寄存器 IR

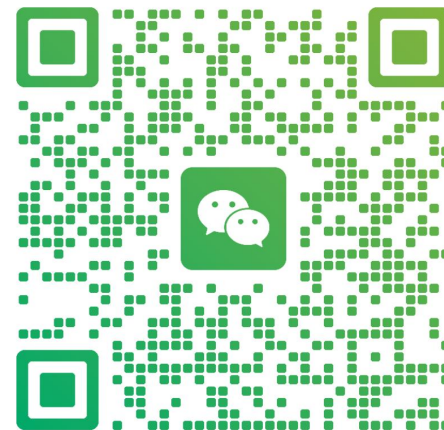
内存地址寄存器 MAR

内存数据寄存器 MDR

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州

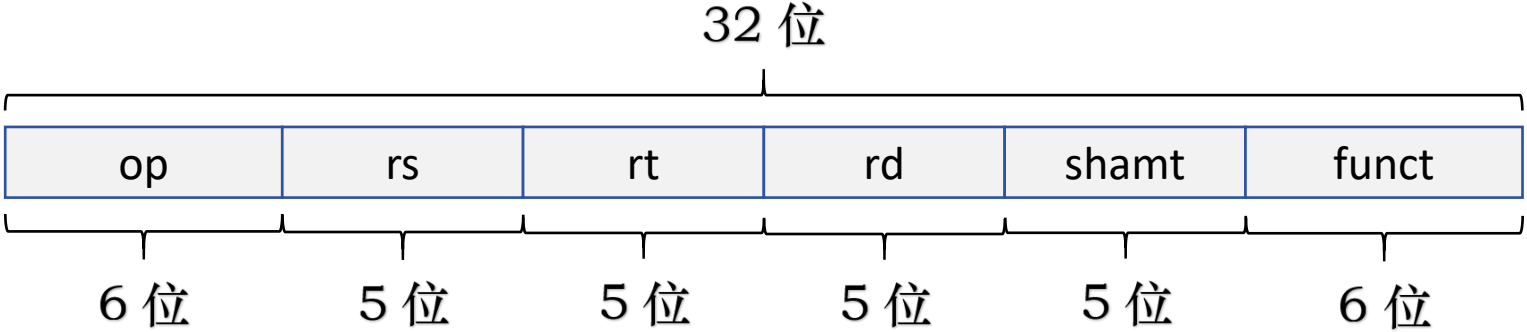


扫一扫上面的二维码图案，加我为朋友。

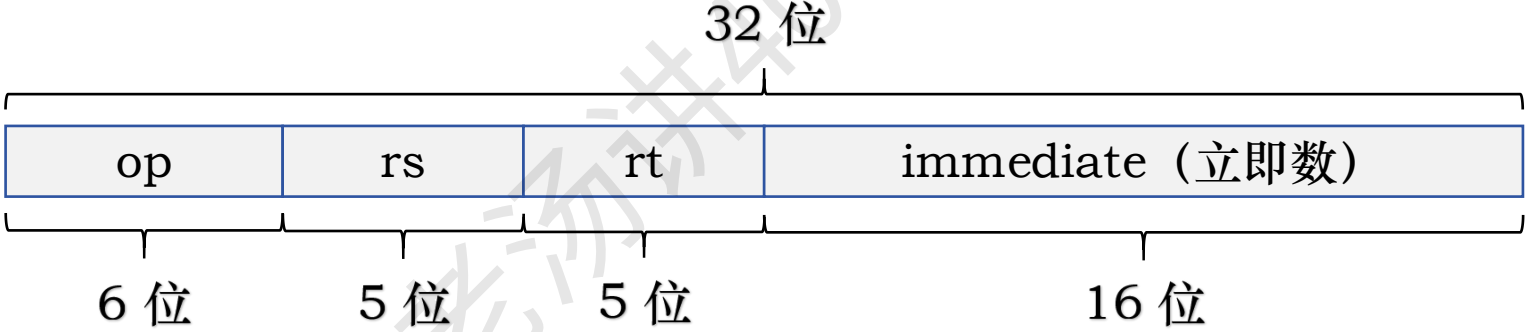
MIPS 指令系统：总结 (2、指令格式)

指令字长 = 32 位

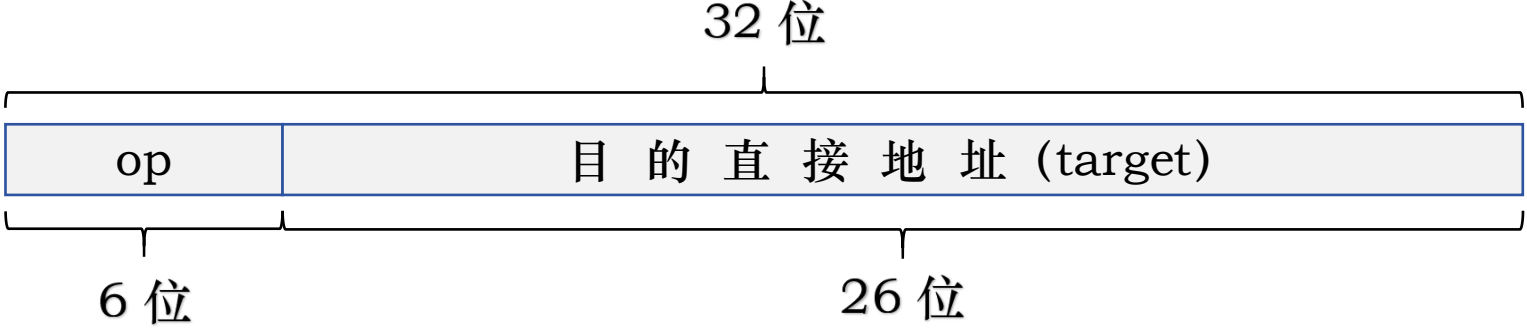
R-型指令格式



I-型指令格式



J-型指令格式



MIPS 指令系统：总结 (3、寻址方式)

① 寄存器寻址

② 立即数寻址

③ 基址寻址

④ PC 相对寻址

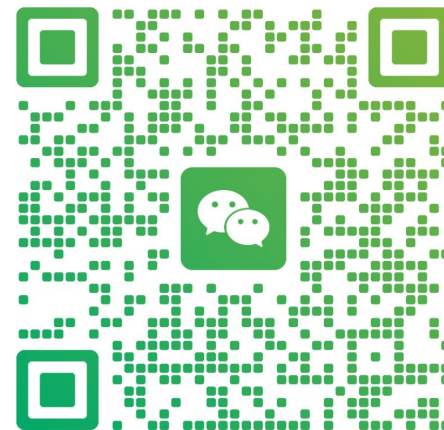
⑤ 伪直接寻址

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

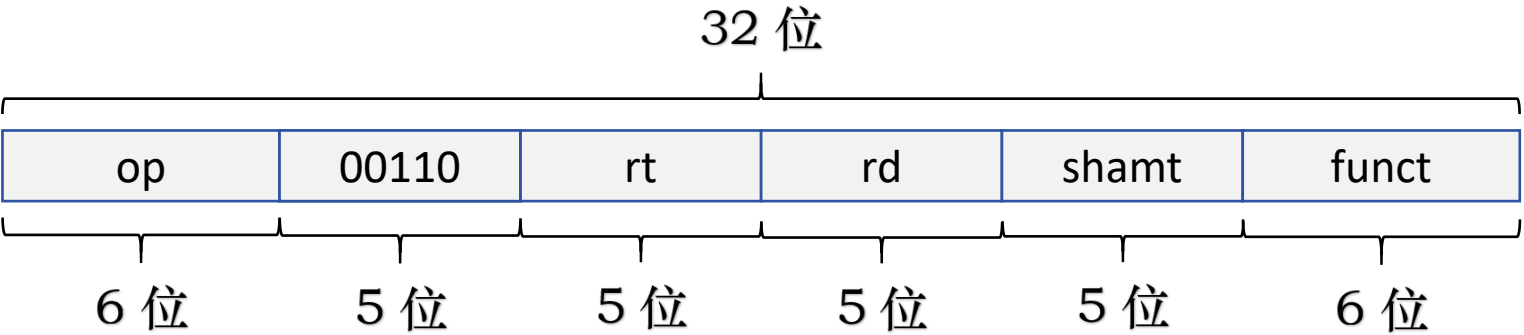
浙江 杭州



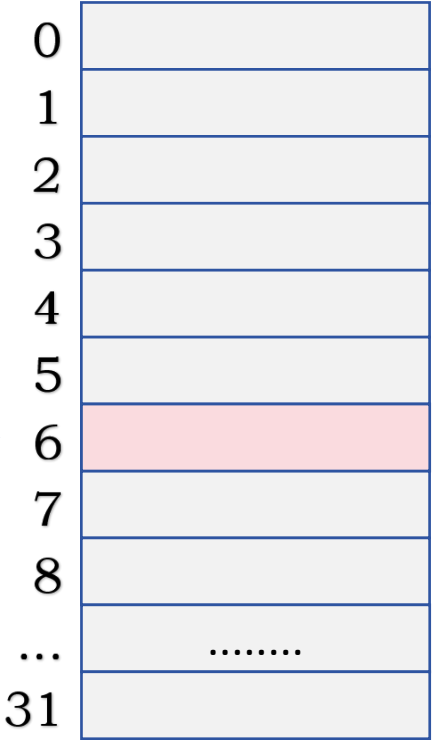
扫一扫上面的二维码图案，加我为朋友。

MIPS 指令系统：总结 (3、寻址方式 - 寄存器寻址)

R-型指令格式



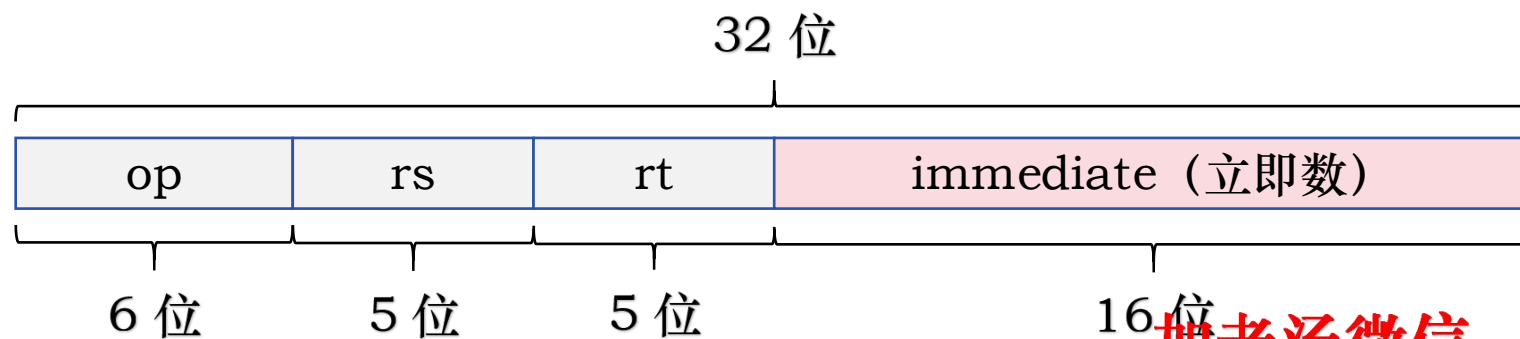
老汤讲408



通用寄存器组

MIPS 指令系统：总结 (3、寻址方式 - 立即数寻址)

I-型指令格式



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州

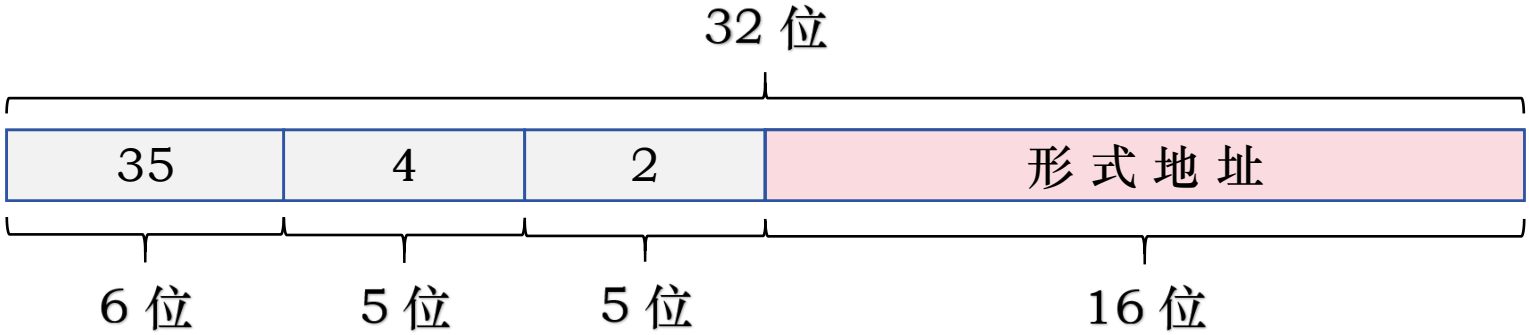


扫一扫上面的二维码图案，加我为朋友。

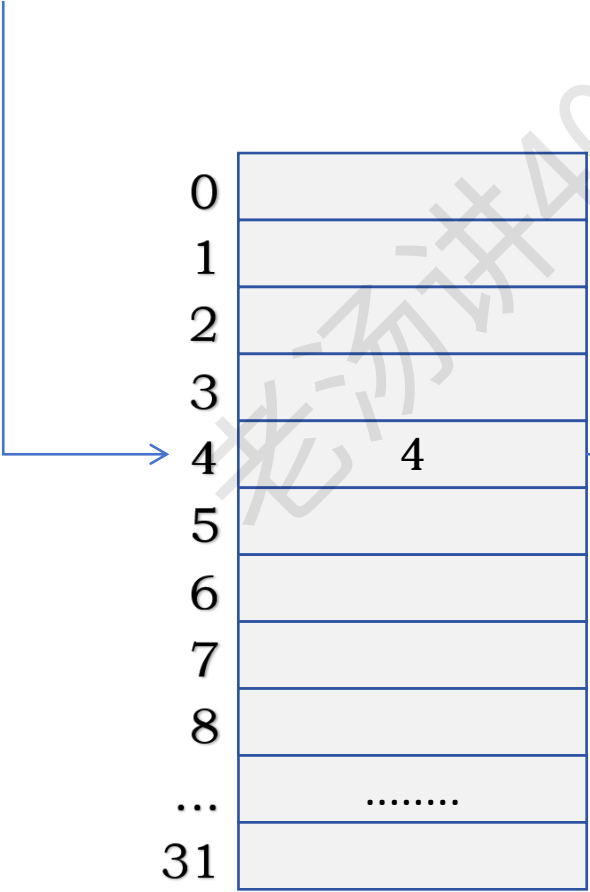
指操作数是位于指令自身中的常数

MIPS 指令系统：总结 (3、寻址方式 - 基址寻址)

I-型指令格式

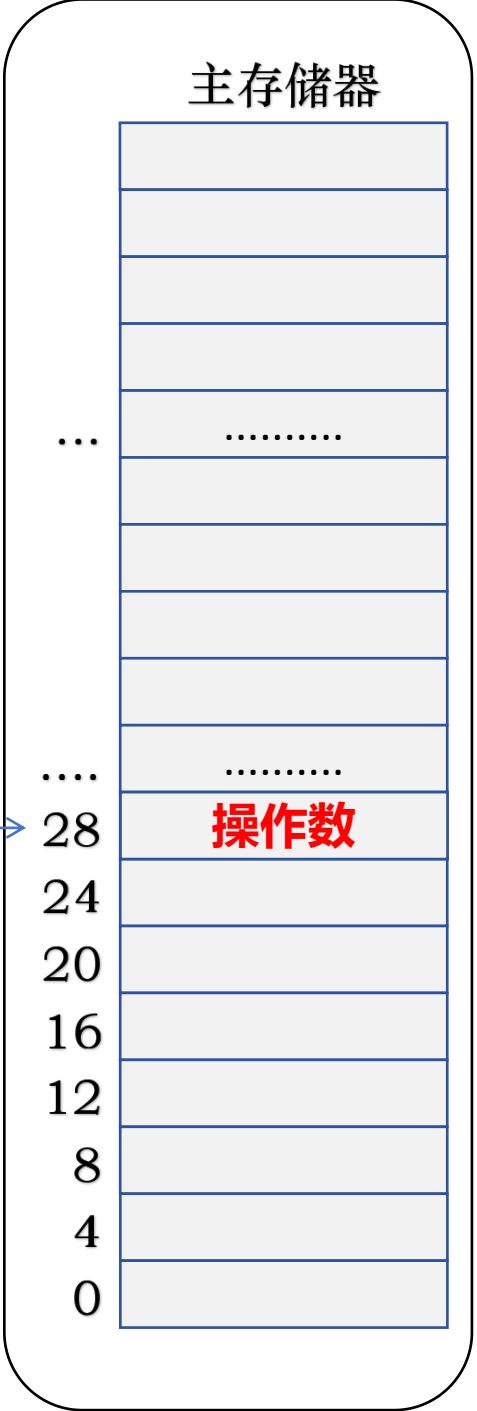


lw \$2, 24(\$4)



通用寄存器组

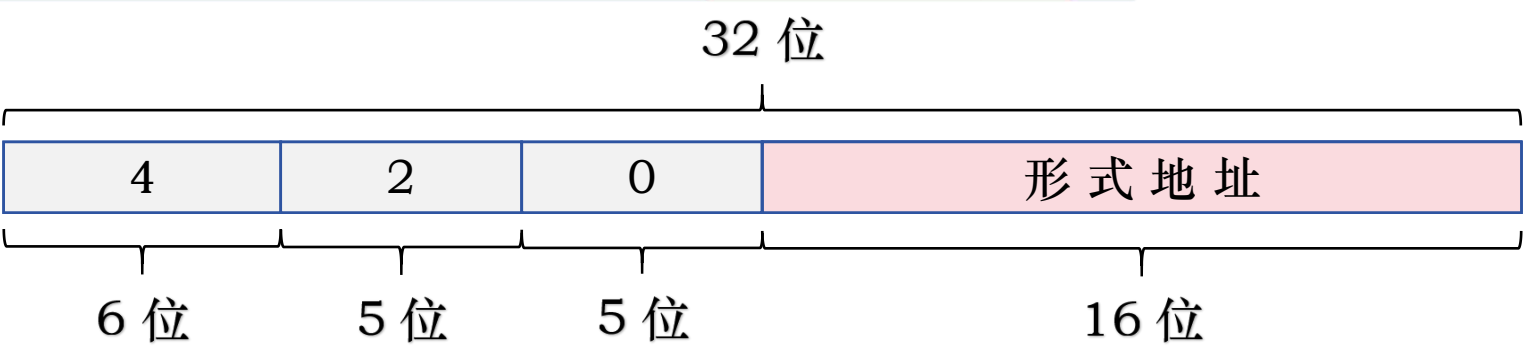
加法器



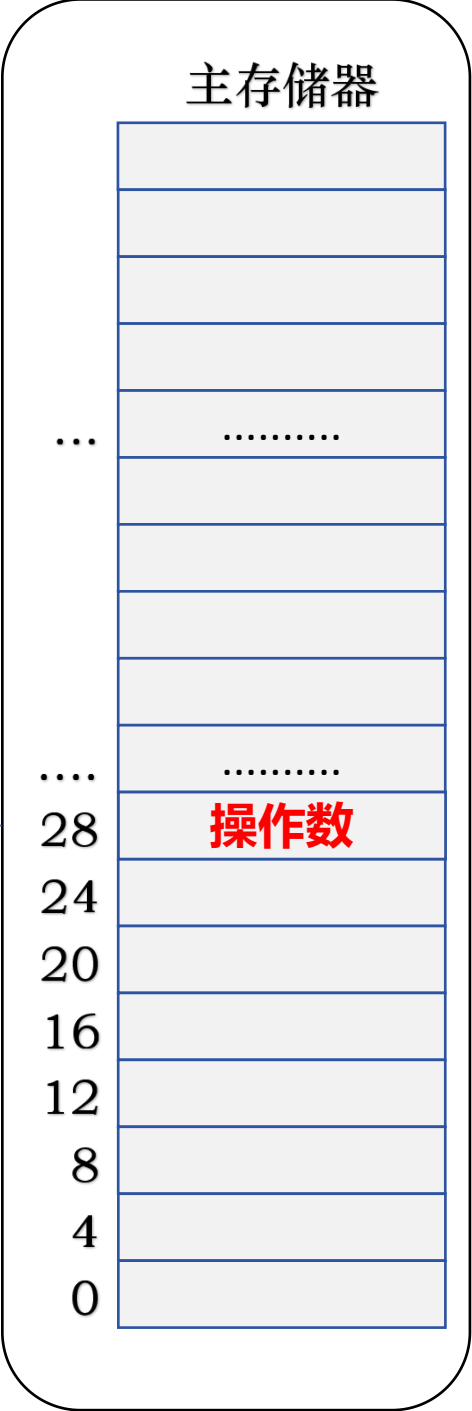
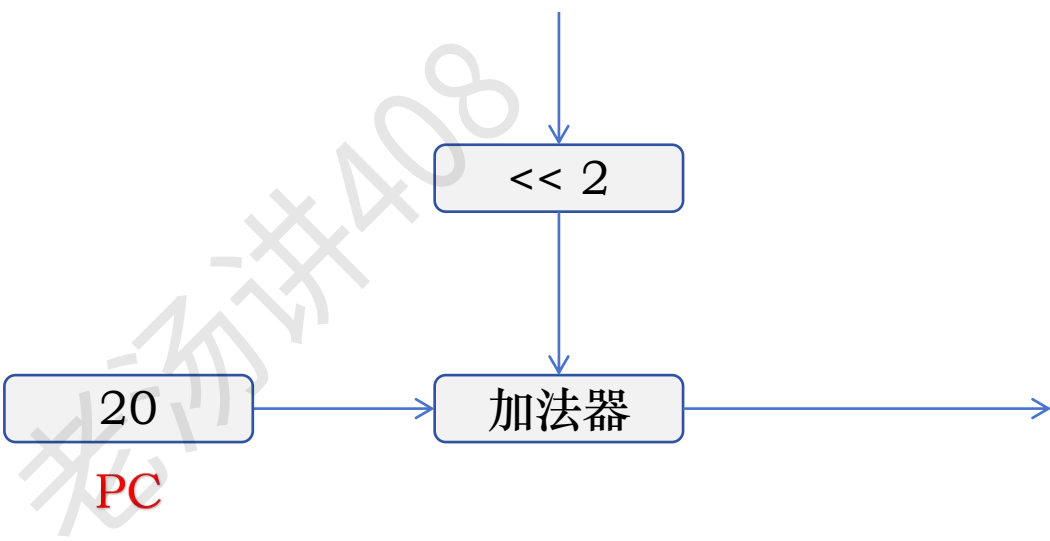
操作数在主存中，
其地址是基址寄存器的值和立即数的和

MIPS 指令系统：总结 (3、寻址方式 - PC 相对寻址)

I-型指令格式

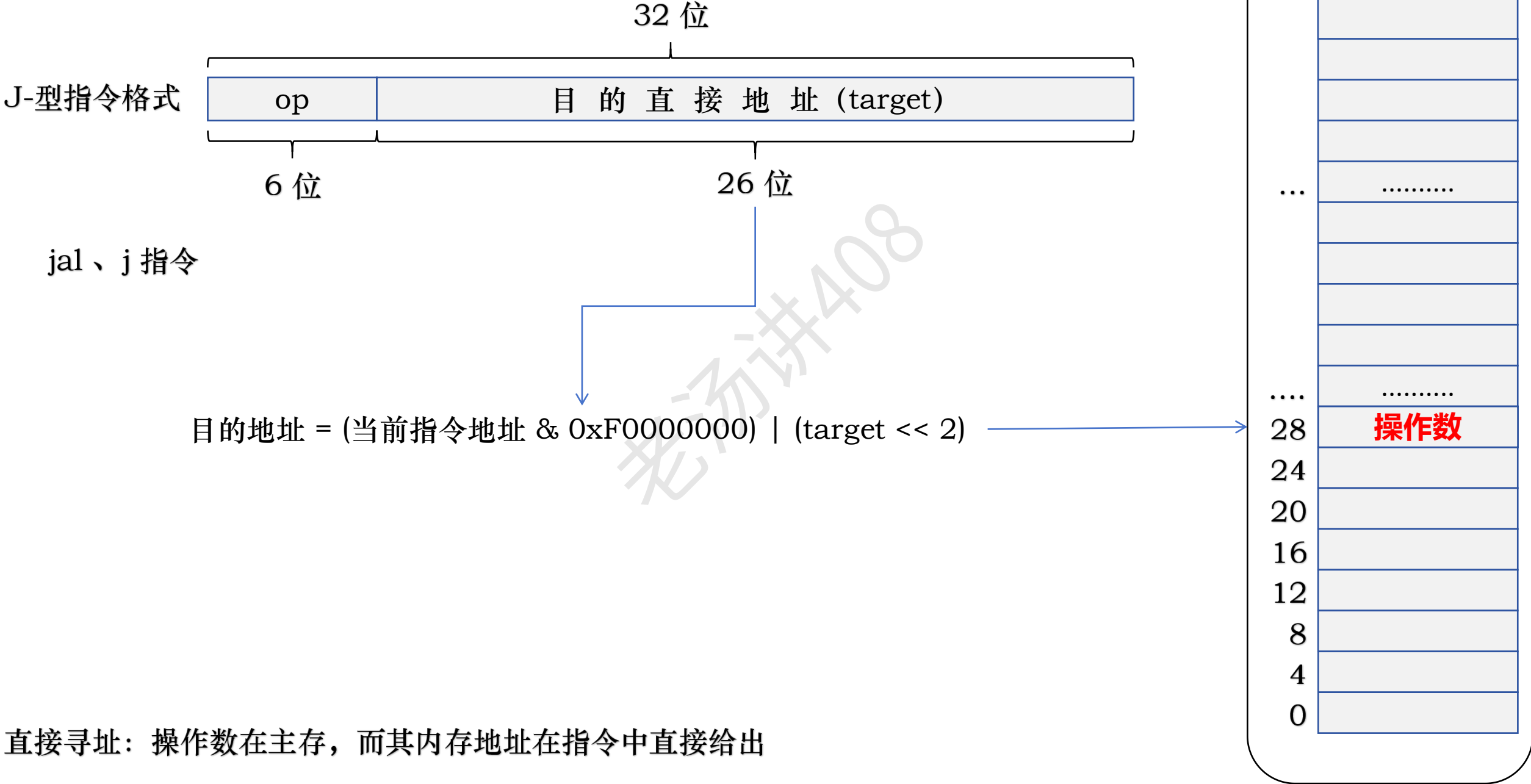


beq \$2, \$0, 8



操作数在主存中，
其地址是 PC 的值和立即数的和

MIPS 指令系统：总结 (3、寻址方式 - 伪直接寻址)



MIPS 指令系统：总结 (3、寻址方式)

① 寄存器寻址

② 立即数寻址

③ 基址寻址

④ PC 相对寻址

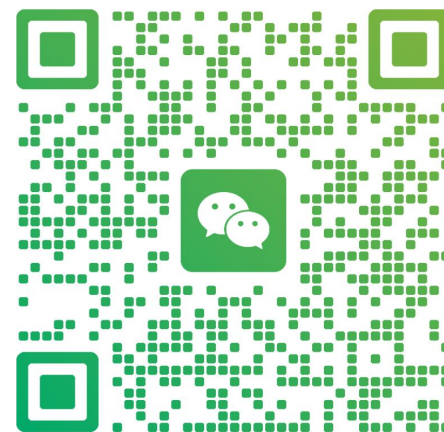
⑤ 伪直接寻址

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

MIPS 指令系统：为什么有多种的寻址方式？

指令执行的时间和空间的权衡

$$6 + 5 + 32 + 32 = 75 + 5 = 80$$

add \$2, 1000000000, 2000000000

add \$3, 1000000000, 2000000000

总共：30 字节

指令执行最快

add \$4, 1000000000, 2000000000

int a = 1000000000;

int b = 2000000000;

int c = a + b;

int d = a + b;

int e = a + b;

li \$3, 1000000000

$$6 + 5 + 32 + 5 = 48$$

li \$4, 2000000000

add \$8, \$3, \$4

$$6 + 5 + 5 + 5 + 3 = 24$$

add \$9, \$3, \$4

add \$2, \$3, \$4

占内存空间少

$$48 \times 2 + 24 \times 3 = 168 \text{ 位} = 21 \text{ 字节}$$

MIPS 指令系统：为什么有多种的寻址方式？

指令执行的时间和空间的权衡

寄存器寻址： 寄存器数量少 → 少量位表示编号 → 指令长度变短 → 占内存空间少

立即数寻址：

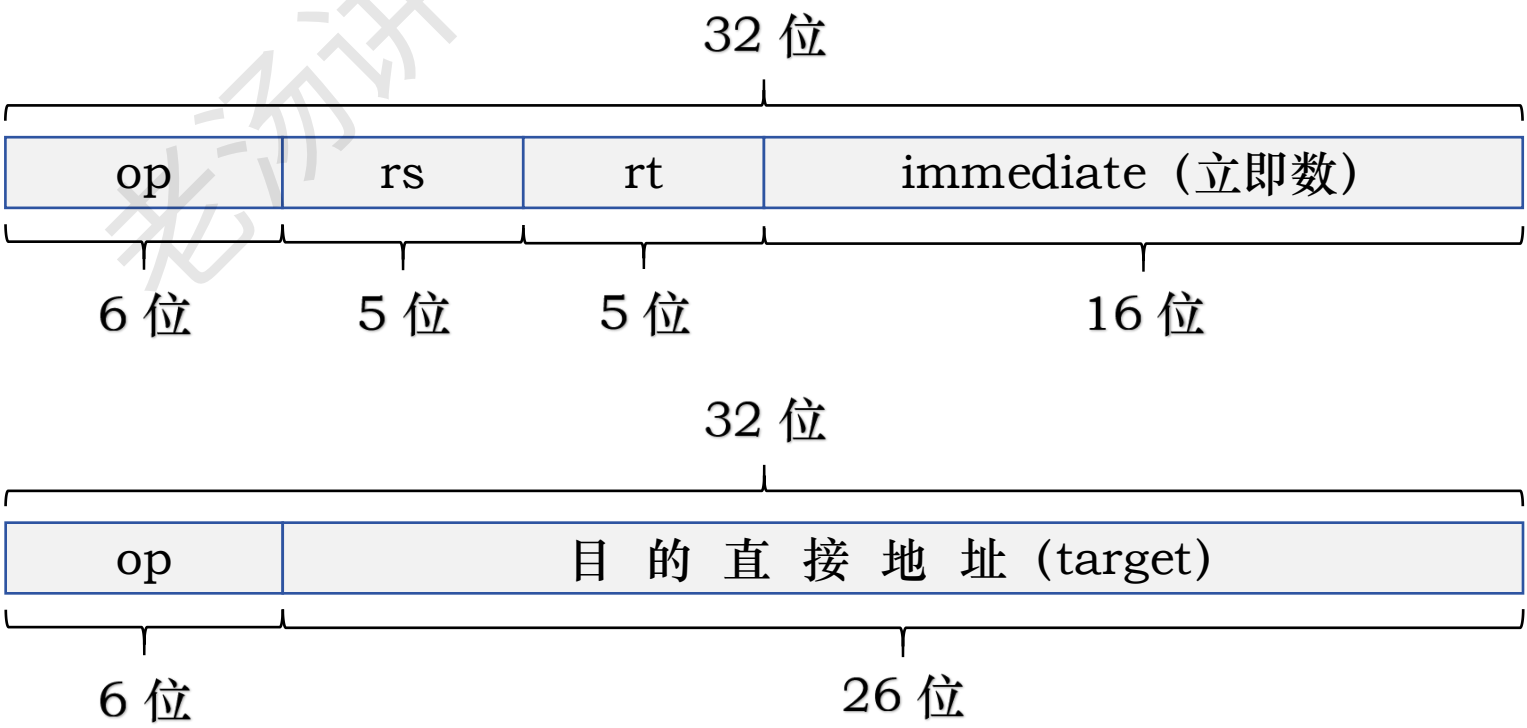
I-型指令格式

立即数比较小

立即数很大 指令变长

能表达的功能受限

J-型指令格式



MIPS 指令系统：总结 (3、寻址方式)

指令执行的时间和空间的权衡

① 寄存器寻址

```
lw $3, 0x12345678
```

② 立即数寻址

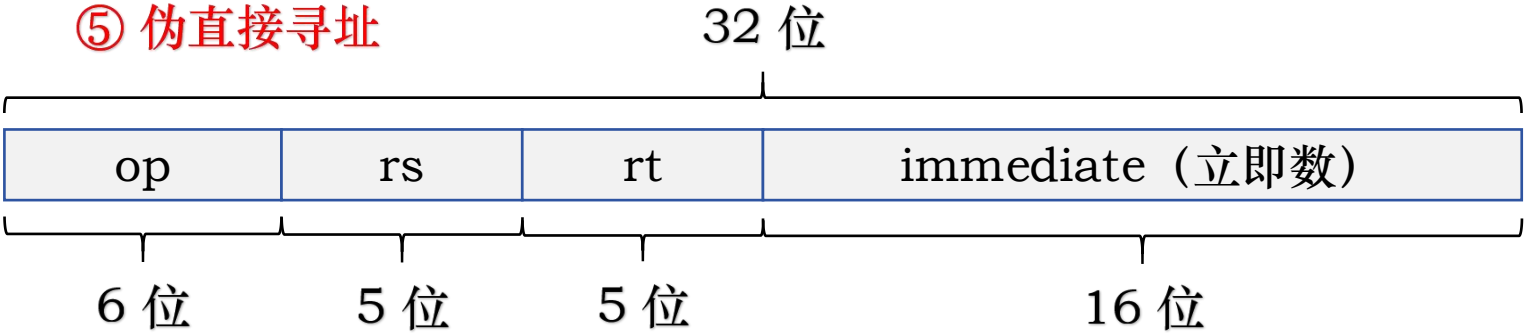
```
lw $3, 24($sp)
```

③ 基址寻址

④ PC 相对寻址

⑤ 伪直接寻址

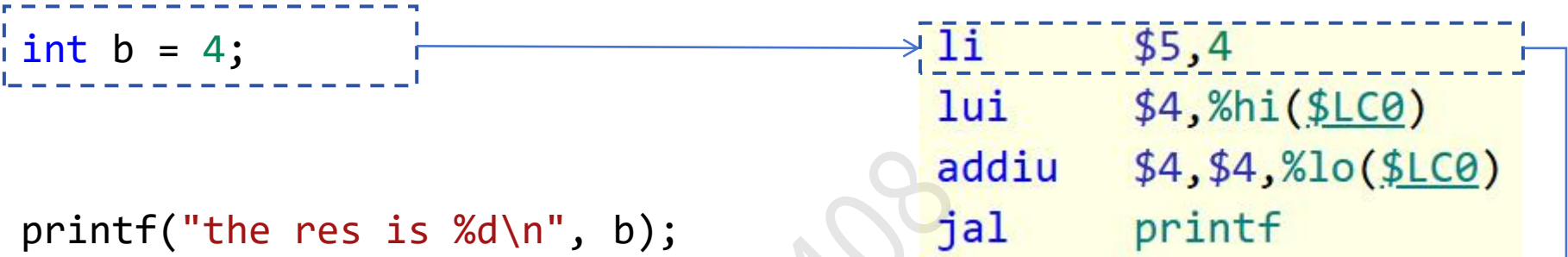
I-型指令格式



MIPS 指令系统: printf 函数调用的机器级表示

li: load immediate 的缩写, 读取立即数到指定寄存器

伪指令



0010 0100 0000 0101 0000 0000 0000 0100

addiu \$0 \$5 4

I-型指令格式

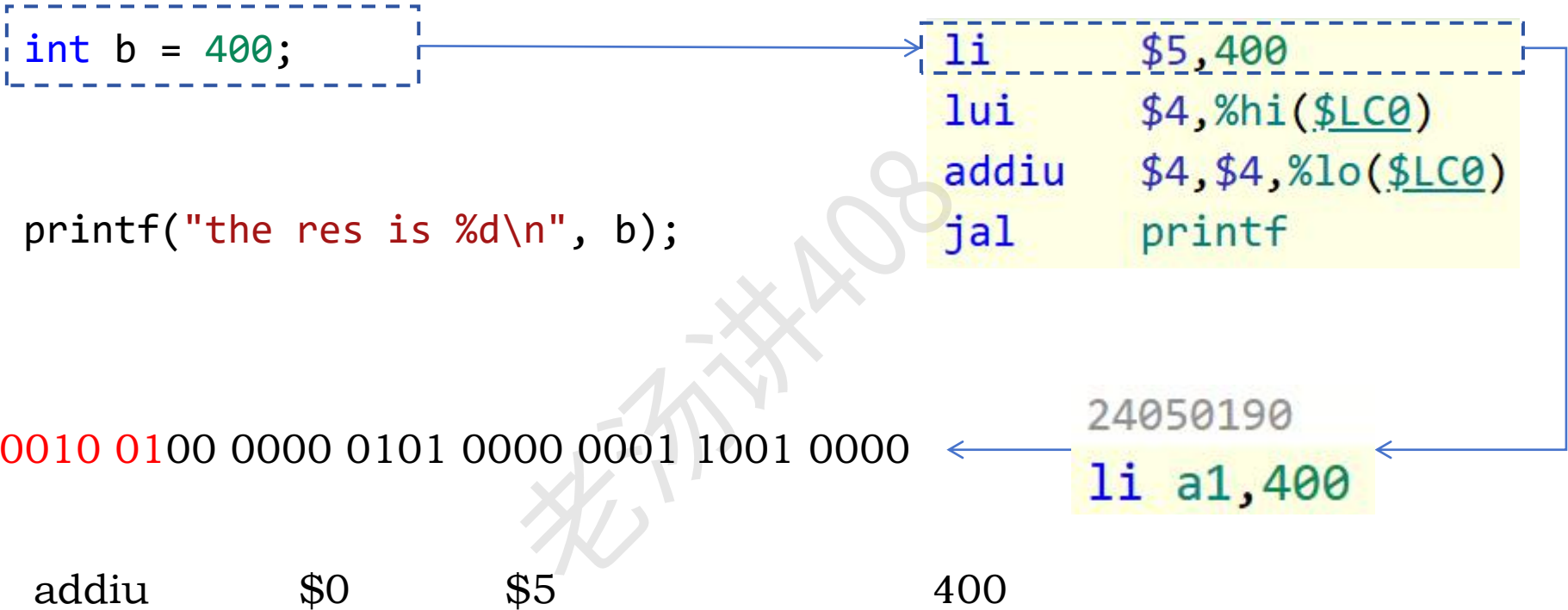
001001	00000	00101	0000 0000 0000 0100
--------	-------	-------	---------------------

强指令的可读性

$\$5 = \$0 + 4$

根据要加载的立即数的大小, 实现 li 的指令是不一样的

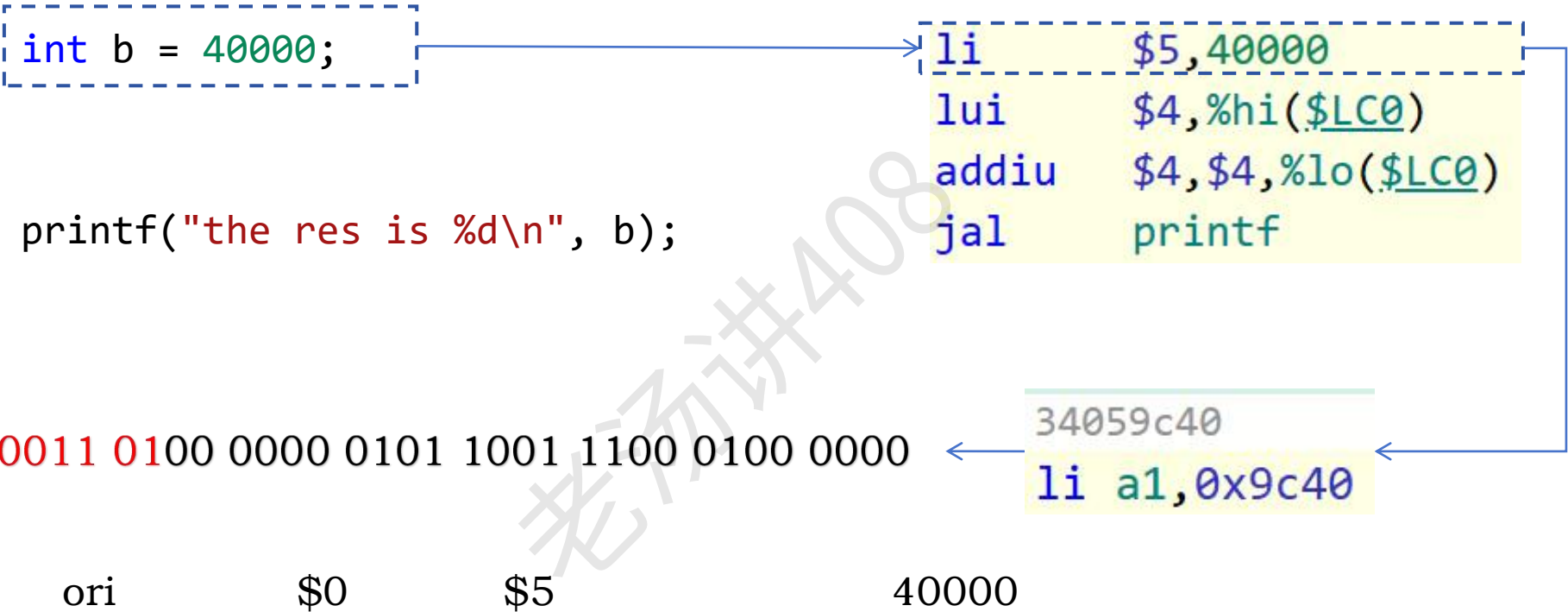
MIPS 指令系统: printf 函数调用的机器级表示



I-型指令格式

001001	00000	00101	0000 0001 1001 0000
--------	-------	-------	---------------------

MIPS 指令系统: printf 函数调用的机器级表示



I-型指令格式

001101	00000	00101	1001 1100 0100 0000
--------	-------	-------	---------------------

$\$5 = \$0 \mid 40000$

MIPS 指令系统：printf 函数调用的机器级表示

lui: load upper immediate 的缩写, 读取立即数高 16 位指令
设置寄存器中常数的高 16 位

```
int b = 400000;
```

0110 0001 1010 1000 0000

```
printf("the res is %d\n", b);
```

```
li      $5,393216
addiu   $5,$5,6784
lui     $4,%hi($LC0)
addiu   $4,$4,%lo($LC0)
jal     printf
```

0011 1100 0000 0101 0000 0000 0000 0110

3c050006

```
lui    a1, 0x6
```

24a51a80

```
addiu    a1, a1, 6784
```

lui

\$0

\$5

6

I-型指令格式

001111

00000

00101

0000 0000 0000 0110

\$5

0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	1	1	0	1	0	1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

高 16 位

低 16 位

MIPS 指令系统: printf 函数调用的机器级表示

```
#include<stdio.h>
```

```
int main() {  
    int b = 400000;  
    printf("the res is %d\n", b);  
    return 0;  
}
```

参数 b 放到 5 号寄存器

```
$LC0: 32 位主存地址: 0x12345678  
        .ascii "the res is %d\012\000"  
main:  
        addiu    $sp,$sp,-32  
        sw       $31,28($sp)  
        {  
            li     $5,393216          #  
            addiu  $5,$5,6784  
        }  
        {  
            lui    $4,%hi($LC0)  
            addiu  $4,$4,%lo($LC0)  
        }  
        jal      printf  
        nop
```

```
        move     $2,$0  
        lw       $31,28($sp)  
        addiu    $sp,$sp,32  
        jr       $31  
        nop
```


MIPS 指令系统：C 指针的机器级表示

```
#include<stdio.h>
```

```
long swap_add(long *xp, long *yp) {  
    long x = *xp;  
    long y = *yp;  
    *xp = y;  
    *yp = x;  
    return x + y;  
}
```

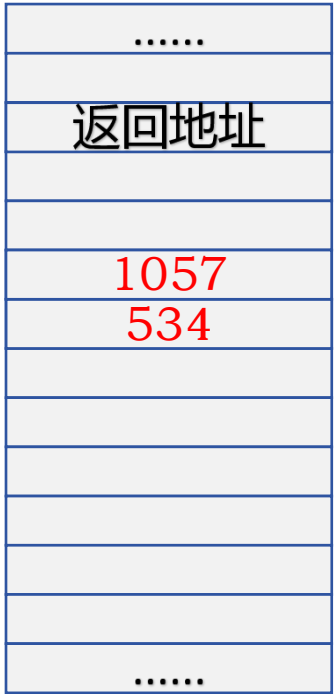
```
int main() {  
    long arg1 = 534;  
    long arg2 = 1057;  
    long res = swap_add(&arg1, &arg2);  
    long diff = arg1 - arg2;  
    printf("res = %d, diff = %d\n", res, diff);  
    return 0;  
}
```

通用寄存器组

	
2	1057
3	
4	0x12345664
5	0x12345668
	
\$sp	0x12345650

主存储器

\$sp → 0x12345678
0x12345674
0x12345670
0x1234566c
0x12345668
0x12345664
0x12345660
0x1234565c
0x12345658
0x12345654
0x12345650
0x1234564c



main:

```
→ addiu $sp,$sp,-40  
    sw   $31,36($sp)  
    li   $2,534  
    sw   $2,24($sp)  
    li   $2,1057  
    sw   $2,28($sp)  
    addiu $5,$sp,28  
    addiu $4,$sp,24  
    jal  swap_add
```


MIPS 指令系统：C 指针的机器级表示

```
#include<stdio.h>
```

```
long swap_add(long *xp, long *yp) {  
    → long x = *xp;  
    long y = *yp;  
    *xp = y;  
    *yp = x;  
    return x + y;  
}
```

所谓的指针，本质上就是内存地址而已

```
int main() {  
    long arg1 = 534;  
    long arg2 = 1057;  
    long res = swap_add(&arg1, &arg2);  
    long diff = arg1 - arg2;  
    printf("res = %d, diff = %d\n", res, diff);  
    return 0;  
}
```

通用寄存器组

	
2	1591
3	534
4	0x12345664
5	0x12345668
	
\$sp	0x12345650

0x12345678
0x12345674
0x12345670
0x1234566c
→ 0x12345668
→ 0x12345664
0x12345660
0x1234565c
0x12345658
0x12345654
\$sp → 0x12345650
0x1234564c

主存储器

.....
返回地址
534
1057
.....

```
swap_add:  
    → lw      $3, 0($4)  
    lw      $2, 0($5)  
    nop  
    sw      $2, 0($4)  
    sw      $3, 0($5)  
    addu    $2, $3, $2  
    jr      $31
```